

OCL2Cypher: A Semantics-Preserving Transformation from UML/OCL Models to Property Graphs

Quoc-Bao Hoang

Duc-Quyen Nguyen

Hoang-Viet Trinh La

Duc-Hanh Dang

VNU University of Engineering and Technology

VNU Town, Hoa Lac, Hanoi, Vietnam

{23020012,25025048,24025181,hanhdd}@vnu.edu.vn

Abstract

This report formalises OCL2Cypher, a transformation from resolved, typed OCL invariants to read-only Cypher queries. A finite runtime instance M of a supported UML metamodel MM is observed through a property graph G according to a representation specification Σ . The transformation passes through a source-oriented Core calculus, a representation-aware query calculus Q , an abstract target artifact, and a serializer with explicit parameter and result contracts. Every partial boundary has explicit success and failure outcomes, with completeness proved on its declared well-formed domain.

The main theorem states that, under metamodel conformance, representation adequacy, configuration well-formedness, feature definedness, and target realisability, the formal target-text semantics returns exactly the stable identifiers of the objects on which OCL evaluation is non-true. A separate one-way refinement transfers this result to successful executions of a conforming Neo4j backend. Supporting lemmas establish environment extension, typed operator congruence, multiplicity-preserving collection behaviour, finite folds, stable identifiers, witness transport, and canonical production selection. Ordered-end navigation is admitted through order-erasing Set/Bag observations only where the surrounding expression is permutation invariant.

An exhaustive constructor and invariant crosswalk establishes closure of the claimed formal fragment. Manual proofs cover its complete constructor inventory; a Lean development independently checks the central preservation argument for a representative executable subfragment. Mathematical validity, mechanised evidence, implementation coverage, and backend conformance thus remain distinct assurance layers connected by explicit premises.

Keywords: OCL; Cypher; model transformation; semantic preservation; property graphs; mechanised proof.

1 Introduction

1.1 Modelling Context

A UML class diagram describes the structural vocabulary of a domain: its classes, attributes, associations, multiplicities, and inheritance relationships. Such a diagram is rarely sufficient to characterize every valid state of a system. The Object Constraint Language (OCL) complements UML with textual invariants whose meaning is evaluated over instances of the diagram [14, 2]. An invariant selects a context class, binds `self` to each instance of that class, and evaluates a Boolean expression formed from attributes, navigation, collections, and iterators.

Property graphs offer a natural physical representation for interconnected model instances. Nodes can represent objects, relationships can represent links, and properties can store identifiers and scalar values. Neo4j and Cypher consequently provide an attractive execution platform for model validation. The apparent correspondence is nevertheless not direct. OCL is defined over the logical structure of a UML instance, whereas Cypher observes a chosen physical graph encoding. A generated query may be syntactically valid and executable while following a relationship in the wrong direction, dropping Bag multiplicity, losing an inherited instance, or treating an undefined value as an ordinary empty result.

OCL2Cypher addresses this semantic gap. It compiles an invariant into a read-only graph query and makes the assumptions connecting the UML instance, its graph representation, and the generated query explicit. The research problem is therefore not merely how to print Cypher text. It is how to ensure that the text observes exactly the same violations as the source invariant.

1.2 Semantic Difficulties

Several differences must be controlled simultaneously.

- UML association ends are logical and bidirectionally navigable, whereas graph relationships have a stored physical direction.
- UML multiplicity and uniqueness determine whether navigation produces one value, a Set, or a Bag; Cypher evaluation naturally produces bags of rows.
- OCL distinguishes false, undefined evaluation, an empty collection, and a whole-collection undefined value. These distinctions cannot be replaced indiscriminately by Cypher `null`.
- Iterators introduce lexical binders. Their realization must preserve declaration identity and prevent capture by generated Cypher aliases.
- `allInstances` and context evaluation are inheritance-aware, while a graph scan depends on the labels and type relations selected by the representation.
- Correct validation returns identities of context objects. Row multiplicity and intermediate witnesses must not introduce either extra identifiers or missing identifiers.

These difficulties are representation-sensitive. A translation rule cannot choose a graph label, property key, or relationship direction from OCL syntax alone. OCL2Cypher therefore uses a schema-level representation specification Σ shared by graph observation and query generation.

1.3 Research Gap and Questions

Existing OCL engines evaluate constraints directly, and several compilers map OCL to relational or graph-oriented execution languages. Model transformations have also been implemented with ATL and validated through tests. These techniques provide valuable executable evidence, but successful execution on finite examples does not establish that every successful translation preserves its source meaning.

This report studies four questions.

RQ-F1. How can the supported OCL-to-Cypher transformation be specified as deterministic, syntax-directed mathematical functions rather than as informal code-generation patterns or implementation procedures?

RQ-F2. Which typing, representation, collection, scope, and result properties must be preserved at each transformation boundary?

RQ-F3. Under which explicit premises does the generated query return exactly the stable identifiers of the violating source objects?

RQ-F4. On which independently defined input domain is translation guaranteed to succeed?

1.4 Approach

The formal input is not a raw character string. It is an OCL invariant in resolved and typed abstract syntax satisfying $WF_{resolved}(MM, inv)$, relative to a supported UML metamodel MM . Parsing raw OCL text is a prototype front-end concern and is kept outside the main semantic theorem. The mathematical transformation is factored as

$$inv \xrightarrow{\text{Front}_{MM}} (K, e) \xrightarrow{T_\Sigma} q \xrightarrow{\text{Realize}_\Sigma} p \xrightarrow{\text{Serialize}} z = (s, \Delta, \pi_{\text{gen}}).$$

Core term e records source-oriented typed computation. The Q term q makes representation-sensitive observations explicit. The artifact $p \in \text{Target}_\Sigma$ is an abstract, scoped realization; it is not a Cypher abstract syntax model. Serialization produces Cypher text s , a result contract Δ , and generated parameters π_{gen} .

A separate representation branch relates a finite runtime instance M to a property graph G :

$$M \xrightarrow{\text{Build}_\Sigma} G.$$

The end-to-end theorem does not require this particular builder. It requires the weaker observational condition $\text{Adequate}_\Sigma(M, G)$, which can be established by a certified builder or by another independent argument.

1.5 Contributions

The report makes four contributions.

1. It gives syntax-directed mathematical algorithms for lowering resolved OCL to Core, translating Core to Q , realizing Q as Target_Σ , and serializing that target to a canonical Cypher package. They are functional presentations of transformation rules, not descriptions of Java methods.
2. It defines the source, intermediate, and target semantic observations, including Set support, Bag occurrences, typed bottom, representation adequacy, lexical scope, and stable-ID projection.
3. It decomposes correctness into frontend preservation, mutual Core-to- Q preservation, violation projection, target realization, serialization transport, and a conditional end-to-end theorem. For $\mathcal{F}_{\text{paper}}$, every published constructor is linked to its typing rule, semantic clause, transformation equation, and an induction discharge route. Several homomorphic constructors may share one generic lemma; semantically distinct constructors receive dedicated cases.
4. It records the distinct assurance levels of the hand proofs, the Lean-checked slice, the ER2REL derivation, and finite Neo4j execution evidence.

The central observable statement is

$$\boxed{\text{Viol}_O(M, inv) = \text{Viol}_C(G, s; \pi, \Delta)}.$$

It states equality of stable-identifier sets, not textual equality of programs or equality of every intermediate value.

1.6 Scope and Non-Claims

The conference-paper profile $\mathcal{F}_{\text{paper}}$ contains finite, well-formed binary UML schemas with concrete or abstract classes, acyclic inheritance, scalar attributes, ordinary/aggregate/composite associations, multiplicities, uniqueness and order metadata, scalar qualifier tuples, and leaf association classes. Its expression language contains admitted primitive and object values, flat Set and Bag collections, inheritance-aware attribute and association navigation, let and conditional expressions, selection, rejection, scalar collection, universal and existential iteration, emptiness, cardinality, type operations, and certified numeric operations. Navigation from an ordered end is admitted only when erasing its order into Set/Bag cannot be observed by the enclosing expression; in particular, collection equality requires order-free provance.

The main theorem is a successful-translation soundness theorem over exactly this grammar and its admission judgment. A separate translation completeness theorem quantifies over an independently specified domain whose terms have every required representation, lexical, numeric, and carrier certificate. The formal target and text results do not by themselves prove that every concrete Neo4j release implements the selected semantics. Concrete execution is reported as finite evidence or used through an explicit executor-refinement premise.

1.7 Report Organization

Section 2 defines UML instances, OCL invariants, property graphs, representation adequacy, and the formal problem. Section 3 defines the syntax, typing, and semantics of the source, Core, Q , and target calculi. Section 4 presents each algorithm immediately before its preservation result and representative proof cases; the exhaustive coverage of the remaining induction obligations is in the appendices. Section 5 gives an ER2REL derivation and finite evaluation evidence. Section 6 positions the work and states the mechanization boundary. Section 7 records limitations and concludes. The appendices contain the complete semantic clauses, algorithm coverage, induction-case discharge map, and traceability ledgers.

2 Basic Concepts and Formal Problem

2.1 Supported UML Metamodels

UML is a family of modelling languages rather than a single finite grammar. The transformation therefore does not quantify over every UML construct. The grammar and well-formedness conditions below give the normative definition of the admitted declaration families.

The supported abstract fragment contains exactly the declarations observed by the admitted OCL expressions:

$$\begin{aligned}
 MM &::= \bar{d}, \\
 d &::= \text{class } K \{\text{abstract} = b\} [\text{extends } \bar{K}] \{\bar{p}\} \\
 &\quad | \text{association}_{\chi} \alpha \{r_1, r_2\} \\
 &\quad | \text{associationClass } K : \alpha, \\
 p &::= \text{attr } a : \sigma \\
 &\quad | \text{role } r : K[\ell..u] \{\text{unique} = b, \text{ordered} = b, \bar{q}\}, \\
 \sigma &::= \text{Boolean} \mid \text{Integer} \mid \text{Real} \mid \text{String} \mid K, \\
 \chi &::= \text{ordinary} \mid \text{aggregate} \mid \text{composite}, \\
 u &::= n \mid *, \quad n, \ell \in \mathbb{N}, \quad b \in \{\text{T}, \text{F}\}.
 \end{aligned}$$

A bar denotes a finite sequence. Declarations carry stable keys; surface names are presentation attributes and are not used as binder identities. Attribute observations are scalar; their type σ may be primitive or object-valued. A qualifier q is a scalar typed declaration attached to an association end. The association kind χ is metadata: all three kinds use the same navigation semantics, while a composite association additionally contributes the containment-validity conditions below.

Definition 2.1 (Supported UML domain) $\text{WF}_{\mathcal{F}}(MM)$ requires declaration-key uniqueness, existing referenced classifiers, acyclic generalization, scalar attribute declarations, valid multiplicities, two consistent ends for every admitted binary association, well-typed finite scalar qualifier tuples, and consistent association-class participants. Qualified ends have lower bound zero; an association class has no proper subtype in the admitted executable profile. For composite associations, every part has at most one composite owner and the containment relation is acyclic. The supported domain is

$$\text{UML}_{\mathcal{F}} = \{MM \mid MM \text{ is generated by the grammar and } \text{WF}_{\mathcal{F}}(MM)\}.$$

This definition treats MM itself as the domain metamodel/class diagram. It does not introduce a second object called a UML metamodel at another meta-level. Ecore or another concrete storage format may encode MM in an implementation, but that representation is not part of the mathematical domain.

2.2 Finite Runtime Instances

An invariant is defined over MM but evaluated on a state conforming to it. That state is a finite runtime instance model, not another metamodel.

Definition 2.2 (Runtime UML instance) For $MM \in \text{UML}_{\mathcal{F}}$, a runtime instance is a tuple

$$M = (O, L, \text{type}_M, \text{slot}_M, \text{ends}_M, \text{id}_M),$$

where O is a finite set of objects, L a finite set of link occurrences, type_M assigns a most-specific declared class to every object, slot_M assigns typed attribute values, ends_M assigns the participating objects and qualifier values to each link occurrence, and $\text{id}_M : O \rightarrow \text{ID}$ assigns stable identifiers.

From this tuple we derive the observations used by OCL:

$$\begin{aligned} \text{Inst}_M(K) &= \{o \in O \mid \text{type}_M(o) \preceq_{MM} K\}, \\ \text{slot}_M(o, a) &= \text{the typed value of attribute } a \text{ at } o, \\ \text{Nav}_M(o, r, \bar{q}) &= \text{the endpoint occurrences reached through } r \text{ with ordered qualifiers } \bar{q}. \end{aligned}$$

The subtype relation $\preceq_{MM}$ is reflexive and transitive. Navigation is logical: its result is independent of any eventual direction chosen for a stored graph relationship. For an ordered end, the admitted observer erases source order and returns the Set support or Bag occurrence multiset selected by the end's uniqueness flag. The expression-admission judgment permits that observer only in permutation-invariant contexts and rejects collection equality when either operand has ordered-navigation provenance.

$\text{ValidInstance}(MM, M)$ states conformance: all objects and links use declared classifiers and ends, no object has an abstract dynamic class, attribute and qualifier values have declared types, unique ends have no duplicate target occurrence, multiplicities are satisfied (including no occurrence at an upper-zero end), and composite ownership is acyclic and single-valued. $\text{WF}_{MM}(M)$ adds the deterministic assumptions consumed by the theorem: id_M is injective, upper-one observations are unambiguous, and every observation is finite. These predicates are source premises; query generation cannot establish them.

2.3 OCL Invariants and Source Observation

The user-facing front end may accept OCL text. The proof begins after parsing, name resolution, and type elaboration, with an abstract-syntax invariant whose declaration references and binder references are identities, not unresolved strings. Formally,

$$\text{inv} = (K, \varphi), \quad MM; \text{self} : K \vdash_O \varphi : \text{Boolean}.$$

The admitted grammar is given in Sect. 3.2. An environment ρ maps declaration identities to typed semantic values. Evaluation is written $\llbracket \varphi \rrbracket_O^{M, \rho}$.

The Boolean carrier is

$$D_B^\perp = \{\top, \text{F}, \perp_B\}.$$

Validation is fail-closed at the semantic level: false and Boolean bottom both identify a violating context object. Compiler or runtime failure is different; it produces no violation observation.

Definition 2.3 (Source violation observation) For $\text{inv} = (K, \varphi)$,

$$\text{Viol}_O(M, \text{inv}) = \{\text{id}_M(o) \mid o \in \text{Inst}_M(K) \wedge \llbracket \varphi \rrbracket_O^{M, [\text{self} \mapsto o]} \neq \top\}.$$

The definition is independent of the compiler. Consequently, equality with a target result is a substantive property rather than a restatement of how translation was defined.

2.4 Property Graphs and Cypher

A property graph is a tuple

$$G = (V, E, \text{src}, \text{tgt}, \text{lab}_V, \text{lab}_E, \text{prop}),$$

with finite node and relationship sets, endpoint functions, finite label sets, and partial property maps. Cypher evaluates graph patterns into bags of rows, then transforms those rows with filters, projections, aggregation, and distinctness. The report uses a finite read-only Cypher 5 profile. Mutation, procedures, and unbounded traversal are outside the target language.

A graph does not have a unique UML interpretation. The schema-level representation specification

$$\Sigma = (\Sigma_{\text{id}}, \Sigma_{\text{type}}, \Sigma_{\text{attr}}, \Sigma_{\text{assoc}}, \Sigma_{\text{inh}}, \Sigma_{\text{coll}}, \Sigma_{\text{qual}}, \Sigma_{\text{codec}})$$

selects stable-ID properties, class/type observations, attribute slots, association-end patterns, physical association kind and direction, inheritance representation, collection/duplicate policy, qualifier encoding, and scalar codecs. Σ_{qual} and Σ_{codec} refine the qualifier and scalar encoding information that the main paper leaves implicit inside its schematic six-component representation contract; they do not introduce a different representation specification. Σ describes how declared UML features are observed in a graph; it is not a runtime snapshot and is not a replacement for MM .

The closed schema predicate is $\text{WF}_{\text{RepSpec}}(MM, \Sigma)$. It requires $\Sigma \in \text{RepSpec}(MM, \text{PGMM})$, a finite catalogue satisfying every catalogue-only PGMM invariant, and exactly one type-correct entry for every class, scalar attribute, navigable end, identity observation, and type operation that the admitted OCL language may request. Every entry

must refer only to declarations, graph types, relationship ends, properties, and codecs owned by MM and Σ ; its result kind, direction, multiplicity, qualifier order, and codec are determined by the referenced declaration. Each selected codec carries the F-1 left-inverse and typed-bottom-separation laws on its representable domain. When MM is fixed, the one-argument notation $WF_{RepSpec}(\Sigma)$ is only a shorthand. Catalogue-local OCL constraints alone are insufficient because they constrain entries that exist but do not establish total observer coverage.

The resulting graph observers are

$$\begin{aligned} & ObjObs_{\Sigma}(G), \quad id_G(g), \quad type_G(g), \\ & slot_G(g, a), \quad Nav_G^{\Sigma}(g, r, \bar{q}), \quad Extent_G^{\Sigma}(K). \end{aligned}$$

Only nodes in $ObjObs_{\Sigma}(G)$ represent source objects. Metadata, slot, type, or reification nodes are excluded.

2.5 Observational Representation Adequacy

Let $\gamma : O \rightarrow ObjObs_{\Sigma}(G)$ map source objects to their graph representatives. Its lifting $\hat{\gamma}$ fixes primitive values and typed bottom, maps objects through γ , maps Set support pointwise, and maps every Bag occurrence without collapsing multiplicity.

Definition 2.4 (Representation adequacy) $Adequate_{\Sigma}(M, G)$ holds iff $WF_{PGMM}(G)$ and there is a bijection

$$\gamma : Obj(M) \cong ObjObs_{\Sigma}(G)$$

such that, for every admitted class K , object o , scalar attribute a , ordinary or association-class end r , and well-typed ordered qualifier tuple $\bar{q}$,

$$\begin{aligned} id_G(\gamma(o)) &= id_M(o), & type_G(\gamma(o)) &= type_M(o), \\ type_M(o) \preceq_{MM} K &\iff conforms_G(\gamma(o), K), & \gamma[Inst_M(K)] &= Extent_G^{\Sigma}(K), \\ slot_G(\gamma(o), a) &= \hat{\gamma}(slot_M(o, a)), & NavOne_G^{\Sigma}(\gamma(o), r, \hat{\gamma}(\bar{q})) &= \hat{\gamma}(NavOne_M(o, r, \bar{q})), \\ NavMany_G^{\Sigma}(\gamma(o), r, \hat{\gamma}(\bar{q})) &= \hat{\gamma}(NavMany_M(o, r, \bar{q})), & ACNav_G^{\Sigma}(\gamma(o), r, \hat{\gamma}(\bar{q})) &= \hat{\gamma}(ACNav_M(o, r, \bar{q})). \end{aligned}$$

Moreover, for every receiver/qualifier bucket the graph-row occurrence multiset is the γ -image of the source link-occurrence multiset. Bottom receivers or qualifiers, absent scalar slots, zero upper-one links, and zero upper-many links follow the typed-bottom/defined-empty cases of the source semantics exactly. An upper-zero end is an upper-one observation whose valid occurrence bucket is necessarily empty, hence it returns typed bottom. For an upper-many end the lifted result is a Set precisely when the end is unique and otherwise is a Bag; association-class navigation uses link-occurrence identity rather than only the endpoint pair. Ordered source occurrences are related only through their order-erased support or occurrence multiset, exactly as required by the admitted navigation semantics.

Adequacy is deliberately observational. It permits different physical graph layouts when they expose the same admitted UML observations. The canonical builder establishes the stronger, witness-bearing predicate

$$ValidRep_{\Sigma}(MM, M, G, \gamma),$$

which records its concrete tagged nodes, slots, links, qualifiers, and association-class objects.

Proposition 2.5 (Canonical builder implies adequacy) If M is finite and well formed and the canonical construction establishes $ValidRep_{\Sigma}(MM, M, G, \gamma)$, then $Adequate_{\Sigma}(M, G)$.

Proof. The canonical finite construction uses injective length-prefixed keys for every node and relationship family, creates exactly one observable graph object per source object, preserves the ordinary/aggregate/composite relationship kind, and excludes untraced semantic elements. An exhaustive invariant crosswalk proves all 56 named PGMM constraints, hence $WF_{PGMM}(G)$. Typed slot construction plus codec left-inversion proves attribute correspondence; the link-occurrence and association-class bijections prove qualified navigation and occurrence multiplicity; copied direct generalization edges prove type conformance and extent equality. The disjoint bottom/absence/empty cases discharge the final adequacy clause, so γ witnesses Definition 2.4. $\square$

After fixing an adequacy witness, write

$$v \simeq_{\Sigma} w \iff w = \hat{\gamma}(v)$$

and lift the relation pointwise to environments:

$$\rho \simeq_{\Sigma} \eta \iff \text{dom}(\rho) = \text{dom}(\eta) \wedge \forall x \in \text{dom}(\rho) : \rho(x) \simeq_{\Sigma} \eta(x).$$

2.6 Notation Summary

Table 1 fixes the symbols used most often in the remaining development. Subscripts O, C, Q, T , and 5 distinguish source OCL, Core, query, abstract-target, and text-level meanings; none denotes an implementation function unless stated explicitly.

Table 1: Principal notation.

Symbol	Meaning
MM, M, G	Supported UML metamodel, finite conforming runtime instance, and representing property graph.
Σ, Ω, Ξ	Representation specification, finite realization-certificate environment, and Core-to- Q translation environment.
$WF_{RepSpec}(MM, \Sigma)$	Closed, total, and type-correct representation catalogue for the admitted observations.
$Adequate_{\Sigma}(M, G)$	Bijection-backed equality of all admitted source and graph observations.
$\mathcal{A}_5$	Abstract target artifact in the supported read-only Cypher 5 profile.
Obs_Q	Stable-ID observation of a successfully evaluated Q plan.
$Viol_O, Viol_C$	Source and decoded formal target-text violation-ID sets.
$TextSem_5(s, G, \pi)$	Formal semantics of serialized target text and its physical parameters.
$Exec_{Neo4j}(s, G, \pi)$	Concrete Neo4j execution relation, used only through the refinement contract of Definition 4.15.

2.7 Formal Compilation Problem

The compiler is partial because malformed, ill-typed, unsupported, or uncertified inputs are explicitly rejected:

$$\text{Compile}_{MM, \Sigma} : \text{Inv}_{OCL}(MM) \rightarrow \text{CypherPackage}.$$

A successful package is

$$z = (s, \Delta, \pi_{\text{gen}}),$$

where s is Cypher text, Δ is a parameter/result contract, and π_{gen} contains compiler-owned physical bindings.

Two properties must be distinguished.

Successful-translation soundness. If compilation, realization, execution, and decoding succeed under the stated premises, the source and target violation-ID observations are equal.

Translation completeness. For every invariant in an independently defined domain $\text{Inv}_{val}(MM, \Sigma)$, there exists a successful artifact. The domain may mention admitted syntax, typing, representation capabilities, lexical admissibility, and numeric/carrier certificates. It must not be defined by successful compiler execution.

The first property answers whether produced queries are correct. The second answers whether the transformation produces a query for every member of its advertised domain. Neither property implies the other.

3 Formal Languages, Types, and Semantics

This section defines every semantic object used by the algorithms in Sect. 4. The order is intentional: a translation is not used to define the meaning of its source language. Each language is first given a grammar, then a typing judgment, and finally an interpretation.

3.1 Common Domains and Collections

The admitted value types are

$$\tau ::= \text{Boolean} \mid \text{Integer} \mid \text{Real} \mid \text{String} \mid \text{Class}(K) \mid \text{Set}(\tau) \mid \text{Bag}(\tau).$$

For every non-collection type τ , D_{τ} is its defined mathematical domain and

$$D_{\tau}^{\perp} = D_{\tau} \cup \{\perp_{\tau}\}.$$

Bottom is typed: $\perp_{\tau}$ is not interchangeable with $\perp_{\sigma}$ when $\tau \neq \sigma$. Integer and Real denote mathematical values; their eventual realization by finite database carriers requires separate certificates.

The Boolean domain is $D_B^\perp = \{\top, \text{F}, \perp_B\}$. Negation fixes bottom. Conjunction is false-dominant, disjunction is true-dominant, and implication is $\beta_1 \Rightarrow \beta_2 = \neg\beta_1 \vee \beta_2$. These tables are listed in Appendix A.

Typed equality is total on each admitted carrier. Two same-typed bottoms are equal, exactly one bottom is unequal to a defined value, and two defined values use their type-specific equality. Consequently equality and inequality return $\top$ or F , never $\perp_B$. Numeric comparison remains a distinct, strict operation with its own bottom and carrier obligations. Object equality compares the resolved class key and stable identifier; Set and Bag equality use typed support and typed multiplicity, respectively.

For a collection kind $\kappa \in \{\text{Set}, \text{Bag}\}$, $\text{Coll}_\kappa(D_\tau^\perp)$ contains finite defined collections and a typed whole-collection bottom. A Set is observed by its support; a Bag is observed by its finite multiplicity function. Write $\text{Occ}(C)$ for one representative of each Set element or for every Bag occurrence. Three cases remain distinct:

$$\perp_{\text{Coll}}, \quad \emptyset, \quad C \text{ containing an occurrence } \perp_\tau.$$

An environment is a finite map from declaration identities to typed values. $\rho[x \mapsto v]$ extends or shadows the binding for the identity x ; it does not perform substitution by a textual variable name.

3.2 Resolved and Typed OCL Abstract Syntax

The formal source is an admitted, resolved subset of OMG OCL abstract syntax. For readability we write a compact mathematical grammar:

$$\begin{aligned} a ::= & c_\tau \mid x \mid \text{LetO}(x : \tau, a, a) \mid \text{IfO}(a, a, a) \\ & \mid \text{AttrO}(a, \mathbf{a}) \mid \text{NavOneO}(a, \mathbf{r}, \bar{a}) \mid \text{NavManyO}_\kappa(a, \mathbf{r}, \bar{a}) \\ & \mid \text{UnO}_{op}(a) \mid \text{BinO}_{op}(a, a) \mid \text{LitO}_\kappa(\bar{a}) \\ & \mid \text{SelectO}(a, x, a) \mid \text{RejectO}(a, x, a) \mid \text{CollectO}(a, x, a) \\ & \mid \text{ForAllO}(a, x, a) \mid \text{ExistsO}(a, x, a) \\ & \mid \text{AllInstancesO}(K) \mid \text{TypeTestO}(a, K) \mid \text{TypeCastO}(a, K). \end{aligned}$$

Here $\mathbf{a}$ is a resolved attribute key, $\mathbf{r}$ is a resolved navigation key, and $\bar{a}$ is the ordered tuple of qualifier expressions. The navigation key is tagged as an ordinary association end or an association-class participant path and retains the matching qualifier declarations; the compact grammar therefore does not require a second association-class navigation syntax. x is a resolved variable declaration identity. Multiplicity and uniqueness determine whether a property call is represented by NavOneO or NavManyO_κ . Surface names, dot/arrow spelling, and operator precedence have already served their parsing purpose and are not semantic fields of this syntax. An ordered association end may occur only when the resolved provenance analysis proves that every enclosing use is permutation-invariant. Its order is then erased into the Set/Bag observation selected by uniqueness; collection equality is admitted only for operands whose provenance contains no ordered-end navigation.

Typing has the form

$$MM; \Gamma \vdash_O a : \tau.$$

Representative rules are

$$\frac{\Gamma(x) = \tau}{MM; \Gamma \vdash_O x : \tau}, \quad \frac{MM; \Gamma \vdash_O a : \text{Class}(C) \quad \text{owner}(\mathbf{a}) \succeq C}{MM; \Gamma \vdash_O \text{AttrO}(a, \mathbf{a}) : \text{type}(\mathbf{a})},$$

and

$$\frac{MM; \Gamma \vdash_O s : \text{Coll}_\kappa(\tau) \quad MM; \Gamma, x : \tau \vdash_O b : \text{Boolean}}{MM; \Gamma \vdash_O \text{ForAllO}(s, x, b) : \text{Boolean}}.$$

Every property, class, and operation reference has exactly one resolved declaration; every binder occurrence points to its declaration identity.

The semantics $\llbracket a \rrbracket_O^{M, \rho}$ is compositional. Literals and variables denote their value and environment lookup. Attribute and navigation terms use slot_M and Nav_M . Let extends the environment once; conditionals evaluate only the selected branch when the guard is defined. Collection operations act occurrence-wise. Selection retains the source collection kind, while admitted scalar-body `collect` produces a Bag with one result occurrence per source occurrence. Universal and existential iteration use the folds defined below. An upper-zero navigation is a scalar observation with an empty valid occurrence bucket and therefore yields typed bottom. An ordered many-valued end is observed after erasing position, so Set support or Bag multiplicity is preserved but no position is exposed. Appendix A gives every clause.

3.3 Core Calculus

Core removes OMG metaclass detail and surface syntactic sugar while retaining types, declaration identities, collection kinds, and source-oriented observers. Expression and plan sorts are mutually defined:

$$\begin{aligned}
e ::= & c_\tau \mid \perp_\tau \mid x \mid \text{Let}(x, e, e) \mid \text{If}(e, e, e) \\
& \mid \text{Attr}(e, \mathbf{a}) \mid \text{NavOne}(e, \mathbf{r}, \bar{e}) \mid \text{Coerce}_h(e) \\
& \mid \text{TypeTest}(e, K, b) \mid \text{TypeCast}(e, K) \mid \text{Un}_{op}(e) \mid \text{Bin}_{op}(e, e) \\
& \mid \text{Lit}_\kappa(\bar{e}) \mid \text{Materialize}(P) \mid \text{ForAll}(P, x, e) \mid \text{Exists}(P, x, e), \\
P ::= & \text{From}(e) \mid \text{Scan}_K \mid \text{NavMany}_\kappa(e, \mathbf{r}, \bar{e}) \\
& \mid \text{Select}(P, x, e) \mid \text{Reject}(P, x, e) \mid \text{Collect}(P, x, e) \mid \text{LetP}(x, e, P).
\end{aligned}$$

The plan sort exposes occurrence-producing computations so that Set support and Bag multiplicity are not hidden inside scalar expressions. Here h is an admitted coercion witness and b selects exact-type or conformance testing. A Core coercion is inserted only where resolved OCL typing supplies a unique witness; it is not an untyped conversion fallback.

Typing judgments are

$$MM; \Gamma \vdash_{\text{Core}} e : \tau, \quad MM; \Gamma \vdash_{\text{Core}} P : (\kappa, \tau).$$

They are mutually inductive. For example,

$$\frac{MM; \Gamma \vdash_{\text{Core}} P : (\kappa, \tau) \quad MM; \Gamma, x : \tau \vdash_{\text{Core}} b : \text{Boolean}}{MM; \Gamma \vdash_{\text{Core}} \text{Select}(P, x, b) : (\kappa, \tau)}.$$

The denotations

$$\llbracket e \rrbracket_{\text{Core}}^{M, \rho}, \quad \llbracket P \rrbracket_{\text{Core}}^{M, \rho}$$

are mutually recursive. A plan denotes either a whole-collection bottom or a finite occurrence collection. Selection evaluates its predicate once per occurrence under $\rho[x \mapsto v]$ and keeps exactly true occurrences; rejection keeps false and bottom occurrences, following the USE local-bottom policy. Collection maps the body over occurrences without silently removing Bag duplicates.

3.4 Representation-Aware Q Calculus

Core does not choose graph access paths. Q makes those choices explicit while retaining a typed, backend-independent query meaning:

$$\begin{aligned}
q ::= & c_{\tau_Q} \mid p_{\tau_Q} \mid \text{QBottom}_{\tau_Q} \mid x \mid \text{QCoerce}_h(q) \mid \text{QLet}(x, q, q) \\
& \mid \text{QIf}(q, q, q) \mid \text{ReadAttribute}(q, \mathbf{a}) \\
& \mid \text{NavigateOne}(q, \mathbf{r}, \bar{q}) \mid \text{NavigateAssociationClassOne}(q, A, \mathbf{r}, \bar{q}) \\
& \mid \text{QTypeTest}(q, K, b) \mid \text{QTypeCast}(q, K) \mid \text{QUn}_{op}(q) \mid \text{QBin}_{op}(q, q) \\
& \mid \text{QCollection}_\kappa(\bar{q}) \mid \text{MaterializeQ}(P_Q) \\
& \mid \text{ForAll3}(P_Q, x, q) \mid \text{Exists3}(P_Q, x, q), \\
P_Q ::= & \text{FromCollection}(q) \mid \text{ScanClass}_K \mid \text{NavigateMany}_\kappa(q, \mathbf{r}, \bar{q}) \\
& \mid \text{NavigateAssociationClass}_\kappa(q, A, \mathbf{r}, \bar{q}) \\
& \mid \text{Filter}_\kappa(P_Q, x, q) \mid \text{Project}_\kappa(P_Q, x, q) \mid \text{CollectQ}(P_Q, x, q) \\
& \mid \text{PlanLet}(x, q, P_Q) \mid \text{DistinctQ}(P_Q).
\end{aligned}$$

A Q query root is

$$\text{QQuery}(m, \sigma, K, x, q, P_Q, \text{key}_\Sigma, \bar{u}),$$

where m is value or violation mode, σ is the result shape, K and x identify an optional context, key_Σ names the representation contract, and $\bar{u}$ declares logical parameters. These root fields are the mathematical roles sometimes called the Q model and Q parameters; they do not form a separate runtime model language.

The transformation domain of this report admits exactly the violation-root instance of that record:

$$\text{Violations}(K, q; \text{key}_\Sigma, \bar{u}) = \text{QQuery}(\text{violation}, \text{IdSet}, K, \text{self}, q, \emptyset, \text{key}_\Sigma, \bar{u}).$$

We write $\text{Violations}(K, q)$ when the representation key and declared parameters are fixed by the surrounding compilation judgment. Value-mode and arbitrary plan-root records are not members of $\mathcal{F}_{\text{paper}}$ and therefore are not silently covered by the theorems below.

The representation-indexed type map is

$$\text{Ty}_\Sigma : \text{Type}_{\text{Core}} \rightarrow \text{Type}_Q.$$

Typing judgments $\Sigma; \Gamma \vdash_Q q : \tau_Q$ and $\Sigma; \Gamma \vdash_Q P_Q : (\kappa, \tau_Q)$ require every graph observer to have a unique catalogue entry in Σ .

The denotations

$$\llbracket q \rrbracket_Q^{G, \eta}, \quad \llbracket P_Q \rrbracket_Q^{G, \eta}$$

use the graph observers from Sect. 2. Their structural clauses parallel Core, but attribute access, navigation, and scans use the representation choices fixed by Σ .

The displayed mutual grammar is the normative and exhaustive Q inventory. Parameterized alternatives with identical typing and semantic equations share one proof schema, while association-class navigation and typed collection operations remain distinct cases. Project preserves its source plan's result kind (deduplicating by typed equality for Set), whereas Collect always returns Bag and retains every occurrence. Here $k \in \{\text{SELECT}, \text{REJECT}\}$; a QFilter retains true occurrences for SELECT and retains false or bottom occurrences for REJECT.

For $C = \llbracket P_Q \rrbracket_Q^{G, \eta}$ and $B(v) = \llbracket b \rrbracket_Q^{G, \eta[x \mapsto v]}$,

$$\text{ForAll3}(C, x, b) = \begin{cases} \perp_B, & C = \perp_{\text{Coll}}, \\ \text{F}, & \exists v \in \text{Occ}(C) : B(v) = \text{F}, \\ \perp_B, & \nexists v : B(v) = \text{F} \wedge \exists v : B(v) = \perp_B, \\ \text{T}, & \text{otherwise}, \end{cases}$$

and

$$\text{Exists3}(C, x, b) = \begin{cases} \perp_B, & C = \perp_{\text{Coll}}, \\ \text{T}, & \exists v \in \text{Occ}(C) : B(v) = \text{T}, \\ \perp_B, & \nexists v : B(v) = \text{T} \wedge \exists v : B(v) = \perp_B, \\ \text{F}, & \text{otherwise}. \end{cases}$$

Thus universal iteration over an empty defined collection is true and existential iteration is false.

For a Boolean query body q and context class K , $\text{Violations}(K, q)$ scans $\text{Extent}_G^\Sigma(K)$, binds `self`, retains non-true body results, projects stable IDs, and removes duplicates:

$$\text{Obs}_Q(G, K, q) = \llbracket \text{Violations}(K, q) \rrbracket_Q^G.$$

3.5 Abstract Target Target_Σ

The target is an abstract realization IR admitted by the selected Cypher profile. It intentionally abstracts from the full grammar and semantics of arbitrary Cypher. Its relevant sorts are

$$\begin{aligned} t &::= \text{TConst}(c) \mid \text{TParam}(u) \mid \text{TVar}(y) \\ &\quad \mid \text{TBottom}(\tau) \mid \text{TCoerce}_h(t) \mid \text{TLet}(y, t, t) \\ &\quad \mid \text{TCase}(t, t, t) \mid \text{TOp}(op, \bar{t}) \\ &\quad \mid \text{TObserveAttr}(t, \omega_a) \mid \text{TObserveOne}(t, \bar{t}, \omega_r) \\ &\quad \mid \text{TTypeTest}(t, \omega_K, b) \mid \text{TTypeCast}(t, \omega_K) \\ &\quad \mid \text{TList}_\kappa(\bar{t}) \mid \text{TMaterialize}(d) \\ &\quad \mid \text{TForAll3}(d, y, t) \mid \text{TExists3}(d, y, t), \\ d &::= \text{TFrom}(t) \mid \text{TScan}(\omega_K) \\ &\quad \mid \text{TObserveMany}_\kappa(t, \bar{t}, \omega_r) \mid \text{TFilter}_k(d, y, t) \\ &\quad \mid \text{TProject}_\kappa(d, y, t) \\ &\quad \mid \text{TCollect}(d, y, t) \mid \text{TPlanLet}(y, t, d) \\ &\quad \mid \text{TDistinct}(d), \\ p &::= \text{TQuery}(m, K, y, t, \omega_K, \omega_{id}, \\ &\quad \text{resultAlias}, \\ &\quad \Delta_\pi, \pi_{\text{gen}}). \end{aligned}$$

The metavariables $\omega_a, \omega_r, \omega_K$ are concrete observer descriptors obtained from Σ . They contain typed structural metadata (source key, kind, direction, result end, properties, codecs, and result kind), not raw Cypher fragments and not an assumed semantic equality. Rule 06 selects one fixed target template from this metadata; theorem R5O proves that every such template denotes the corresponding Q observer on every adequate graph. Fresh target variables y are drawn from a namespace disjoint from logical parameter identities. Every artifact produced in $\mathcal{F}_{\text{paper}}$ has $m = \text{violation}$; the field remains explicit only so that the artifact records its observation mode.

$\text{WF}_{\text{Target}, \Sigma}(p)$ is the least judgment generated by the target typing and scope rules in Appendix A. In particular it requires admitted types, lexical scope, read-only constructs, valid observer descriptors, disjoint public/generated parameter domains, and a result shape matching the contract. The denotation $A_5(p, G, \pi)$ is the finite row-bag semantics defined constructor by constructor in that appendix, with explicit success or error:

$$A_5(p, G, \pi) \in \{\text{Success}(R), \text{Failure}(d)\}.$$

A failure is not an empty row bag and is not typed bottom.

For the public theorem it is convenient to expose a target witness p through two mathematical observations:

$$B_T^p : \text{Extent}_G^\Sigma(K) \rightarrow D_B^\perp, \quad \text{Ret}^p : \text{ID} \rightarrow \text{Prop}.$$

They denote, respectively, evaluation of the realized Boolean body and the stable-ID membership predicate induced by the violation-mode TQuery root. Their defining equations, including the restriction to $\text{Extent}_G^\Sigma(K)$, are given in Appendix A.

3.6 Serialized Package and Text Semantics

Serialization returns

$$\text{Serialize}(p) = \text{Success}(z), \quad z = (s, \Delta, \pi_{\text{gen}}).$$

The witness relation used by the main paper is made explicit as

$$\text{Witness}_\Sigma(p; G, K, q, s, \Delta, \pi_{\text{gen}}).$$

It holds exactly when p is a well-typed realization of $\text{Violations}(K, q)$ under Σ , serialization of p produces the displayed package, and G supplies the interpretation domain for its target observations. Equivalently, its non-semantic definition is

$$\begin{aligned} & \text{Witness}_\Sigma(p; G, K, q, s, \Delta, \pi_{\text{gen}}) \\ \iff & \text{Realize}_\Sigma^{IR}(\text{Violations}(K, q)) = \text{Success}(p) \\ & \wedge \text{Serialize}(p) = \text{Success}(s, \Delta, \pi_{\text{gen}}) \\ & \wedge \text{WF}_{\text{Target}, \Sigma}(p) \wedge \text{Extent}_G^\Sigma(K) \text{ is defined.} \end{aligned}$$

The graph is not an input to realization. The witness records provenance and artifact correspondence; semantic obligations R1–R3 are proved separately rather than hidden in its definition. The contract

$$\Delta = (\text{Dom}_{\text{pub}}, \text{idCol}, \text{decode})$$

defines admissible public parameters, the returned identifier column, and the decoder from a successful raw result to a finite subset of ID. Runtime parameters satisfy

$$\pi_{\text{public}} \in \text{Dom}_{\text{pub}}, \quad \text{dom}(\pi_{\text{public}}) \cap \text{dom}(\pi_{\text{gen}}) = \emptyset, \quad \pi = \pi_{\text{public}} \uplus \pi_{\text{gen}}.$$

The artifact contains two non-textual side channels: decoder contract Δ_π and generated-value map π_{gen} . Let $\text{Syn}(p)$ erase those values while retaining their parameter identities, result alias, and all executable target syntax. Thus $A_5(p, G, \pi) = A_5(\text{Syn}(p), G, \pi)$; the erased maps are consumed by the package interface, not by target expression evaluation.

Let parse_5 be the deterministic parser for the canonical generated text profile. It returns executable syntax $c = \text{Syn}(p)$ rather than inventing the erased side-channel values. Formal text semantics is

$$\text{TextSem}_5(s, G, \pi) = \begin{cases} \text{Failure}(d), & \text{parse}_5(s) = \text{Failure}(d), \\ A_5(c, G, \pi), & \text{parse}_5(s) = \text{Success}(c). \end{cases}$$

After successful execution and decoding,

$$\text{Viol}_C(G, s; \pi, \Delta) = J$$

iff the text semantics succeeds and decode returns J . A concrete executor $\text{Exec}_{\text{Neo4j}}$ is related to this formal semantics only by the one-way generated-profile refinement of Definition 4.15; finite deployment evidence does not itself establish that relation.

4 Transformation Algorithms and Correctness

The four transformation boundaries are defined here as partial mathematical functions over the abstract grammars of Sect. 3. The word *algorithm* therefore means a finite, syntax-directed presentation of a transformation relation. It does not denote a Java method, an object model, an exception protocol, or an order of API calls. As in a conventional rule-based specification, each displayed case has a source pattern, premises, and one target term; recursion invokes the cases for strict subterms.

The proof roadmap is the following composition; solid arrows are proved transformations or observations, whereas the dashed arrow is the separately stated concrete-executor refinement:

$$\begin{aligned} \text{OCL}_{\text{AST}} &\longrightarrow \text{Core} \longrightarrow Q \longrightarrow \text{Obs}_Q, \\ \text{Obs}_Q &\longrightarrow \text{Target}_\Sigma \longrightarrow \text{TextSem}_5 \xrightarrow{\text{external}} \text{Exec}_{\text{Neo4j}}. \end{aligned}$$

Preservation is established locally at each solid boundary and then composed; no later-stage equality is used to prove an earlier stage.

For explicitness, partiality is represented by the mathematical result type

$$\text{Result}(X) = \text{Success}(X) \mid \text{Failure}(\bar{d}), \quad |\bar{d}| \geq 1.$$

Here $\bar{d}$ is only an abstract nonempty explanation of why no rule is applicable; it is not a Java diagnostic object. A failed subderivation makes the enclosing mathematical function undefined, represented by Failure. Transformation failure, runtime bottom, and an empty violation set are three different outcomes.

Equivalently, the four stages may be read as deterministic judgments

$$MM; \Gamma \vdash_N a \Downarrow e, \quad \Sigma; \Gamma \vdash e \Downarrow_T q, \quad \Sigma; \Xi \vdash q \Downarrow_R p, \quad p \Downarrow_S (s, \Delta, \pi_{\text{gen}}).$$

Every equation in Algorithms 1–4 is the functional orientation of one or more such inference rules. Because the source constructors are disjoint and all resolution choices are unique, the judgments are syntax directed and have at most one conclusion. For readability, the listings display the payload equation of a successful case and omit the outer Success constructor. If a stated premise or a recursive payload equation is unavailable, the lifted function yields Failure; no listing clause silently supplies a default value.

Proposition 4.1 (Rule–algorithm equivalence) *For each boundary $B \in \{N, T, R, S\}$, the corresponding algorithm returns $\text{Success}(y)$ on input x iff the corresponding judgment displayed above has a finite derivation with input x and output y . The result y is unique.*

Proof. From a derivation to an algorithm result, use induction on the last applied rule: the constructor selects the same displayed case and the induction hypotheses supply its recursive results. In the other direction, use structural induction on the input term: every successful case supplies the premises and conclusion of the corresponding rule. Disjoint constructors, unique resolved declarations, deterministic freshness, and canonical printing give uniqueness. $\square$

Granularity of the correctness proof. The unit of correctness is a complete transformation function, not an individual rule. Its theorem is proved by structural induction (mutual structural induction for expression/plan pairs), so the grammar generates one proof obligation for each constructor branch. This does not require one separately numbered lemma per branch. Constructors that are semantic homomorphisms are discharged by instantiating a shared congruence or environment-extension lemma. Constructors with materially different semantics—in particular typed bottom, Set/Bag occurrence behaviour, selection, collection, distinctness, three-valued universal/existential folds, navigation, and violation projection—have dedicated arguments. Appendices B and C jointly show that every constructor obligation has one such discharge route; they do not assert a one-rule–one-lemma correspondence.

Theorem 4.2 (Termination and determinism of the formal transformation) *On the independently admitted source and backend domains, every recursive stage terminates and has at most one successful result. Consequently every admitted source invariant has one unique Core image, one unique Q image, one unique target image under the fixed least-fresh and constant policies, and one unique canonical serialized package.*

Proof. For frontend mutual recursion use the lexicographic measure $(\text{height}(a), b)$, where $b = 1$ for a plan request and $b = 0$ for an expression request. The fallback plan-to-expression equation decreases b ; all expression-to-plan calls decrease syntax height. Every other frontend call is on a strict child. Core-to- Q and Q -to-target mutual calls are on strict expression/plan children, and serialization is structural on the finite target tree. Thus all stages terminate.

For determinism, source constructors and target constructors are disjoint; the frontend fallback applies only when no preceding plan pattern matches. Every resolved declaration has one identity, every observer catalogue lookup is unique, leastFresh is a function, the inline/parameter policy is fixed, and every lexical codec and production is canonical. Induction on the same measures therefore gives at most one result at every recursive call and hence at each root. $\square$

4.1 Public Pipeline

The mathematical pipeline is

$$\text{Front}_{MM}(\text{inv}) = \text{Success}(K, e), \quad (1)$$

$$T_{\Sigma}(e) = \text{Success}(q), \quad (2)$$

$$\text{Realize}_{\Sigma}^{IR}(\text{Violations}(K, q)) = \text{Success}(p), \quad (3)$$

$$\text{Serialize}(p) = \text{Success}(s, \Delta, \pi_{\text{gen}}). \quad (4)$$

The main paper exposes the last two lines as one public boundary. Using its notation, define

$$\text{Realize}_{\Sigma}(v) = \text{bind}(\text{Realize}_{\Sigma}^{IR}(v), \text{Serialize}),$$

where bind propagates a formal failure and otherwise applies the second function. Hence the main paper's equation $\text{Realize}_{\Sigma}(v) = \text{Success}(s, \Delta, \pi_{\text{gen}})$ and this report's two internal equations denote the same transformation boundary. The split only permits separate proofs for target-IR realization and text serialization.

$\text{Compile}_{MM, \Sigma}$ is the composition of Front_{MM} , T_{Σ} , and the public Realize_{Σ} . The algorithms inspect MM , resolved terms, and the schema-level catalogue in Σ . They do not inspect the runtime instance M or graph G . Those objects occur in semantics and adequacy premises, not as hidden compiler inputs.

4.2 Algorithm 1: Resolved OCL to Core

Front_{MM} checks that the resolved OCL invariant belongs to the admitted profile and lowers its body by the structural function $\text{lower}_{MM, \Gamma}$. Parser correctness is not an obligation of this algorithm because its input is already abstract syntax.

At an implementation-facing interface, parsing, resolution, and type elaboration may be fused with this frontend. In the formal calculus they are represented by the premises $\text{inv} \in \text{Inv}(MM)$ and $\text{WF}_{\text{resolved}}(MM, \text{inv})$: all required sources and static types are present, and declaration, signature, and binder references are uniquely resolved. The algorithm below is therefore the admission-and-lowering component of the main paper's public Front_{MM} , not an algorithm over raw OCL text.

Listing 1: Formal resolved OCL-to-Core transformation.

```

1 Front-MM(K, phi) = (K, Lower-MM({self:Class(K)}, phi))
2
3 Lower-MM(Gamma, c_tau) = Const(tau, c)
4 Lower-MM(Gamma, x) = Var(x)
5 Lower-MM(Gamma, Let0(x:tau, a1, a2)) =
6   Let(x, Lower-MM(Gamma, a1), Lower-MM(Gamma+x:tau, a2))
7 Lower-MM(Gamma, If0(c, t, f)) =
8   If(Lower-MM(Gamma, c), Lower-MM(Gamma, t), Lower-MM(Gamma, f))
9 Lower-MM(Gamma, Attr0(r, a)) = Attr(Lower-MM(Gamma, r), a)
10 Lower-MM(Gamma, NavOne0(r, end, qs)) =
11   NavOne(Lower-MM(Gamma, r), end, map(Lower-MM(Gamma), qs))
12 Lower-MM(Gamma, NavMany0(kind, r, end, qs)) =
13   Materialize(NavMany(kind, Lower-MM(Gamma, r), end,
14     map(Lower-MM(Gamma), qs)))
15 Lower-MM(Gamma, Un0(op, a)) = Un(op, Lower-MM(Gamma, a))
16 Lower-MM(Gamma, Bin0(op, a1, a2)) =
17   Bin(op, Lower-MM(Gamma, a1), Lower-MM(Gamma, a2))
18 Lower-MM(Gamma, Lit0(kind, [a_i])) =
19   Lit(kind, [Lower-MM(Gamma, a_i)])
20 Lower-MM(Gamma, Select0(src, x, b)) =
21   Materialize(Select(LowerPlan-MM(Gamma, src), x,
22     Lower-MM(Gamma+x:elem(src), b)))
23 Lower-MM(Gamma, Reject0(src, x, b)) =
24   Materialize(Reject(LowerPlan-MM(Gamma, src), x,
25     Lower-MM(Gamma+x:elem(src), b)))
26 Lower-MM(Gamma, Collect0(src, x, b)) =
27   Materialize(Collect(LowerPlan-MM(Gamma, src), x,
28     Lower-MM(Gamma+x:elem(src), b)))
29 Lower-MM(Gamma, ForAll0(src, x, b)) =
30   ForAll(LowerPlan-MM(Gamma, src), x, Lower-MM(Gamma+x:elem(src), b))

```

```

31 Lower-MM(Gamma,ExistsO(src,x,b)) =
32   Exists(LowerPlan-MM(Gamma,src),x,Lower-MM(Gamma+x:elem(src),b))
33 Lower-MM(Gamma,AllInstancesO(K)) = Materialize(Scan(K))
34 Lower-MM(Gamma,TypeTestO(a,K)) =
35   TypeTest(Lower-MM(Gamma,a),K,testKind(a,K))
36 Lower-MM(Gamma,TypeCastO(a,K)) =
37   TypeCast(Lower-MM(Gamma,a),K)
38
39 LowerPlan-MM(Gamma,NavManyO(kind,r,end,qs)) =
40   NavMany(kind,Lower-MM(Gamma,r),end,map(Lower-MM(Gamma),qs))
41 LowerPlan-MM(Gamma,AllInstancesO(K)) = Scan(K)
42 LowerPlan-MM(Gamma,LitO(kind,[a_i])) =
43   From(Lit(kind,[Lower-MM(Gamma,a_i)]))
44 LowerPlan-MM(Gamma,SelectO(src,x,b)) =
45   Select(LowerPlan-MM(Gamma,src),x,Lower-MM(Gamma+x:elem(src),b))
46 LowerPlan-MM(Gamma,RejectO(src,x,b)) =
47   Reject(LowerPlan-MM(Gamma,src),x,Lower-MM(Gamma+x:elem(src),b))
48 LowerPlan-MM(Gamma,CollectO(src,x,b)) =
49   Collect(LowerPlan-MM(Gamma,src),x,Lower-MM(Gamma+x:elem(src),b))
50 LowerPlan-MM(Gamma,LetO(x:tau,a,P)) =
51   LetP(x,Lower-MM(Gamma,a),LowerPlan-MM(Gamma+x:tau,P))
52 LowerPlan-MM(Gamma,a) = From(Lower-MM(Gamma,a))
53   when none of the preceding plan patterns matches
54
55 At every typed child boundary, insert Coerce(h,e) exactly when the resolved
56 typing derivation contains the unique admitted coercion witness h.

```

These equations are defined only when the source typing rule and the admission side conditions for the row in Appendix B hold. For example, $\Gamma(x) = \tau$ is required for a variable, branch types must agree for an if expression, and the declared upper bound determines whether a navigation uses the one- or many-valued clause. If a premise is absent, there is no successful derivation for that case. Each recursive call is on a strict abstract-syntax child, so termination follows from tree height. Constructor dispatch and resolved declaration identity make the result deterministic.

Lemma 4.3 (Frontend type and scope preservation) *If $MM; \Gamma \vdash_O a : \tau$ and $\text{lower}_{MM,\Gamma}(a) = \text{Success}(e)$, then $MM; \Gamma \vdash_{\text{Core}} e : \tau$ and every free Core binder is in $\text{dom}(\Gamma)$.*

Proof. We prove the strengthened statement

$$MM; \Gamma \vdash_O a : \tau \wedge \text{lower}_{MM,\Gamma}(a) = \text{Success}(e) \implies MM; \Gamma \vdash_{\text{Core}} e : \tau \wedge \text{FV}(e) \subseteq \text{dom}(\Gamma)$$

by structural induction on a .

Variable base case. If $a = \text{VariableO}(x)$, inversion of source typing gives $\Gamma(x) = \tau$. The matching algorithm equation returns $e = \text{Var}(x)$. The Core variable rule gives $MM; \Gamma \vdash_{\text{Core}} \text{Var}(x) : \tau$, and $\text{FV}(e) = \{x\} \subseteq \text{dom}(\Gamma)$. Literal leaves have no free binders and follow directly from their declared types; source bottom is dynamic and therefore has no surface leaf case.

Binding case. Let $a = \text{LetO}(x : \tau_1, a_1, a_2)$. Inversion of source typing yields

$$MM; \Gamma \vdash_O a_1 : \tau_1, \quad MM; \Gamma, x : \tau_1 \vdash_O a_2 : \tau_2.$$

Successful lowering yields $e = \text{Let}(x, e_1, e_2)$ with both recursive calls successful. The two induction hypotheses give the corresponding Core judgments under Γ and $\Gamma, x : \tau_1$. The Core let rule therefore gives $MM; \Gamma \vdash_{\text{Core}} e : \tau_2$. Moreover,

$$\text{FV}(e) = \text{FV}(e_1) \cup (\text{FV}(e_2) \setminus \{x\}) \subseteq \text{dom}(\Gamma).$$

The declaration identity copied into the body is the resolved identity of x , so this argument also excludes capture by a same-spelled declaration. Iterator binders use the same environment-extension step, once for their source plan and once for the body under the iterator declaration.

Non-binding composite cases. For each remaining constructor, typing inversion types every strict child; the induction hypotheses type their images and bound their free variables. The matching Core typing rule reconstructs the parent type, while the free variables of the parent are the union of those of its children. For an observer-bearing constructor, the resolved declaration and multiplicity premise select the corresponding attribute, to-one, or to-many Core rule. These cases exhaust the source grammar listed in Appendix B. $\square$

Theorem 4.4 (Frontend semantic preservation) Let $inv = (K, \varphi)$ satisfy $WF_{resolved}(MM, inv)$, and suppose the frontend succeeds:

$$\text{Front}_{MM}(inv) = \text{Success}(K, e).$$

If $\text{ValidInstance}(MM, M)$ and $WF_{MM}(M)$, then for every $o \in \text{Inst}_M(K)$,

$$\llbracket \varphi \rrbracket_O^{M, [\text{self} \mapsto o]} = \llbracket e \rrbracket_{\text{Core}}^{M, [\text{self} \mapsto o]}.$$

Proof. The proof is by structural induction on the resolved source body.

Base case. Let $a = \text{Literal}(v : \tau)$. Algorithm 1 returns $\text{Const}_\tau(v)$. By the two literal semantic clauses,

$$\llbracket a \rrbracket_O^{M, \rho} = v = \llbracket \text{Const}_\tau(v) \rrbracket_{\text{Core}}^{M, \rho}.$$

Inductive let case. Let $a = \text{LetO}(x : \tau, a_1, a_2)$. The algorithm produces $\text{Let}(x, e_1, e_2)$, where the induction hypothesis gives

$$\llbracket a_1 \rrbracket_O^{M, \rho} = \llbracket e_1 \rrbracket_{\text{Core}}^{M, \rho} = v.$$

Because the source and Core body use the same declaration identity x , the two body evaluations receive the identical finite environment $\rho[x \mapsto v]$. Applying the second induction hypothesis in that environment,

$$\llbracket a_2 \rrbracket_O^{M, \rho[x \mapsto v]} = \llbracket e_2 \rrbracket_{\text{Core}}^{M, \rho[x \mapsto v]}.$$

Unfolding the two let clauses proves the case. The remaining families are discharged by the generic congruence lemmas or the specialized navigation, collection, iterator, and bottom cases in Appendix C. $\square$

4.3 Algorithm 2: Core to Q

The translation is mutually recursive because expressions may materialize plans and iterator expressions contain plans:

$$T_\Sigma^E : \text{Expr}_{\text{Core}} \rightarrow \text{Result}(\text{Expr}_Q), \quad T_\Sigma^P : \text{Plan}_{\text{Core}} \rightarrow \text{Result}(\text{Plan}_Q).$$

Listing 2: Formal mutual Core-to-Q transformation.

```

1 T-E-SIGMA(e) =
2   Const/Bottom => corresponding typed Q leaf
3   Var(x) => QParameterExpr(x) if parameter(x), else QVariable(x)
4   Coerce(h,e) => QCoerce(h,T-E(e))
5   Let(x,e1,e2) => QLet(x,T-E(e1),T-E(e2))
6   If(c,t,f) => QIf(T-E(c),T-E(t),T-E(f))
7   Attr(r,a) => ReadAttribute(T-E(r),a)
8   NavOne(r,end,qs) =>
9     if ordinary(end) then NavigateOne(T-E(r),end,map(T-E,qs))
10    else NavigateAssociationClassOne(T-E(r),assoc(end),source(end),map(T-E,qs))
11   TypeTest(r,K,b) => QTypeTest(T-E(r),K,b)
12   TypeCast(r,K) => QTypeCast(T-E(r),K)
13   Un(op,e1) => QUn(op,T-E(e1))
14   Bin(op,e1,e2) => QBin(op,T-E(e1),T-E(e2))
15   Lit(kind,items) => QCollection(kind,map(T-E,items))
16   Materialize(P) => MaterializeQ(T-P(P))
17   ForAll(P,x,b) => ForAll3(T-P(P),x,T-E(b))
18   Exists(P,x,b) => Exists3(T-P(P),x,T-E(b))
19
20 T-P-SIGMA(P) =
21   From(e) => FromCollection(T-E(e))
22   Scan(K) => ScanClass(K)
23   NavMany(kind,r,end,qs) =>
24     if ordinary(end) then NavigateMany(kind,T-E(r),end,map(T-E,qs))
25     else NavigateAssociationClass(kind,T-E(r),assoc(end),source(end),map(T-E,qs))
26   Select(P,x,b) => Filter(SELECT,T-P(P),x,T-E(b))
27   Reject(P,x,b) => Filter(REJECT,T-P(P),x,T-E(b))
28   Collect(P,x,b) => CollectQ(T-P(P),x,T-E(b))
29   LetP(x,e,P) => PlanLet(x,T-E(e),T-P(P))

```

Each observer-bearing equation has the premise that Σ contains one type-correct descriptor for the resolved declaration. If it is absent, ambiguous, or incompatible with the collection kind, the equation has no successful conclusion; no default graph pattern is introduced.

Lemma 4.5 (Core-to- Q typing) *Let $\text{Ty}_\Sigma(\Gamma)$ map each type in Γ pointwise. The mutual translation satisfies both of the following statements:*

1. *if $MM; \Gamma \vdash_{\text{Core}} e : \tau$ and $T_\Sigma^E(e) = \text{Success}(q)$, then $\Sigma; \text{Ty}_\Sigma(\Gamma) \vdash_Q q : \text{Ty}_\Sigma(\tau)$ and $\text{FV}(q) = \text{FV}(e)$;*
2. *if $MM; \Gamma \vdash_{\text{Core}} P : (\kappa, \tau)$ and $T_\Sigma^P(P) = \text{Success}(P_Q)$, then $\Sigma; \text{Ty}_\Sigma(\Gamma) \vdash_Q P_Q : (\kappa, \text{Ty}_\Sigma(\tau))$ and $\text{FV}(P_Q) = \text{FV}(P)$.*

Proof. We use mutual structural induction on Core expressions and plans. Constant, bottom, and variable leaves follow from the pointwise type map; in particular, the variable equation copies the resolved binder identity rather than its display name.

Plan case with a binder. Consider $P = \text{Select}(P_0, x : \tau, b)$. Inversion of Core typing gives

$$MM; \Gamma \vdash_{\text{Core}} P_0 : (\kappa, \tau), \quad MM; \Gamma, x : \tau \vdash_{\text{Core}} b : \text{Boolean}.$$

The algorithm returns

$$P_Q = \text{Filter}(T_\Sigma^P(P_0), x, T_\Sigma^E(b)).$$

The mutual induction hypotheses type the first child as $(\kappa, \text{Ty}_\Sigma(\tau))$ and the predicate as Boolean under $\text{Ty}_\Sigma(\Gamma), x : \text{Ty}_\Sigma(\tau)$. The Q filter rule therefore types P_Q with the same collection kind and element type. For scope,

$$\text{FV}(P_Q) = \text{FV}(T_\Sigma^P(P_0)) \cup (\text{FV}(T_\Sigma^E(b)) \setminus \{x\}) = \text{FV}(P).$$

The Reject, Collect, and PlanLet cases use the same displayed environment extension, with the filter policy or declared result type substituted in their respective conclusion rules.

Observer case. For $e = \text{Attr}(e_0, \mathbf{a})$, the expression induction hypothesis types the receiver. Successful dispatch supplies the unique Σ descriptor for $\mathbf{a}$; its type-correctness certificate states that its graph result type is exactly Ty_Σ of the resolved UML attribute type. The Q read-attribute rule then gives the required type and introduces no binder. Navigation observers use the corresponding certified element and collection-kind clauses. All remaining non-binding constructors apply the mutual induction hypotheses to their strict children and then their syntax-directed Q typing rule. The cases are exhaustive by Appendix B. $\square$

Theorem 4.6 (Mutual Core-to- Q semantic preservation) *Assume*

$$\text{Adequate}_\Sigma(M, G) \quad \text{and} \quad \rho \simeq_\Sigma \eta.$$

For every expression translation satisfying

$$\begin{aligned} T_\Sigma^E(e) &= \text{Success}(q), \\ \llbracket e \rrbracket_{\text{Core}}^{M, \rho} &\simeq_\Sigma \llbracket q \rrbracket_Q^{G, \eta}. \end{aligned}$$

For every successful plan translation $T_\Sigma^P(P) = \text{Success}(P_Q)$, the exact plan statement is

$$\hat{\gamma}(\llbracket P \rrbracket_{\text{Core}}^{M, \rho}) = \llbracket P_Q \rrbracket_Q^{G, \eta},$$

where the lifted map sends whole-collection bottom to whole-collection bottom, maps Set support pointwise, and maps every indexed Bag occurrence.

Proof. Use mutual structural induction on e and P .

Base case. For $e = \text{Const}_\tau(v)$, the translation is the corresponding typed Q constant. The lifting $\hat{\gamma}$ fixes admitted primitive values and maps object constants by γ , hence the two values correspond directly. The variable and typed-bottom leaves follow from environment correspondence and the definition of $\hat{\gamma}$.

Inductive navigation case. Let the source and translated nodes be

$$e = \text{NavOne}(e_0, \mathbf{r}, \bar{e}), \quad q = \text{NavigateOne}(q_0, \mathbf{r}, \bar{q}).$$

By the induction hypothesis, the receiver values correspond. If the receiver is bottom, both strict navigation clauses return the corresponding typed bottom. Otherwise the navigation clause of Definition 2.4 gives

$$\hat{\gamma}(\text{Nav}_M(v, \mathbf{r}, \llbracket \bar{e} \rrbracket)) = \text{Nav}_G^\Sigma(\gamma(v), \mathbf{r}, \hat{\gamma}(\llbracket \bar{e} \rrbracket)).$$

The upper-one well-formedness premise makes both observations either the same paired object or absence, which both semantics represent by the corresponding typed bottom. Therefore the navigation results correspond. The many-valued case applies the same equation occurrence-wise and then uses either Set-support or Bag-multiplicity preservation. The child induction hypotheses map the ordered qualifier tuple componentwise, and codec reflection makes source and graph qualifier matching equivalent.

The generic unary and binary cases use operand induction hypotheses and the typed operator-correspondence lemma. Let and iterator cases use the environment-extension lemma. Select/reject filtering, collect, distinct, scan, three-valued fold, qualifier, association-class, and whole-bottom cases are expanded in Appendix C. $\square$

4.4 Violation Projection

Lemma 4.7 (Violation-projection preservation) *Under the hypotheses of Theorems 4.4 and 4.6,*

$$\text{Viol}_O(M, \text{inv}) = \text{Obs}_Q(G, K, q).$$

Proof. The two preceding preservation theorems classify each paired body result identically as true, false, or Boolean bottom. Apply Lemma C.10 to the adequacy bijection for the context extent. It gives equality after non-true selection, stable-ID projection, and distinct support, which are precisely the two displayed observations. $\square$

4.5 Algorithm 3: Q to Target_Σ

Before realization, a deterministic normalization removes derived Q forms in favour of the canonical constructors in Sect. 3. It is idempotent, type-preserving, and denotation-preserving. The auxiliary mathematical environment $\Xi = (\text{scope}, \text{fresh}, \chi, \pi_{\text{gen}})$ records the current scope, the finite set of used target identities, the binder map, and generated logical bindings. It is proof state carried by the recursive definition, not the state of a Java object.

The finite certificate environment Ω records the carrier, numeric, and cardinality facts required by the constructors occurring in a query. The two independently defined inductive admission judgments are

$$\text{C5Realisable}_\Gamma(q, \Sigma, \Omega) \quad \text{and} \quad \text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega).$$

Their leaf rules require the appropriate codecs and catalogue lookups, their composite rules require derivations for all children, and the root rule requires WF_Q^Σ , $\text{WF}_{\text{RepSpec}}(MM, \Sigma)$, LexAdmissible_5 , the context extent, stable-ID, non-true, and one-column result certificates. $\text{ObserverAdequate}_5$ is deliberately not a certificate field: theorem R50 below derives it from Q well-formedness and representation-specification well-formedness. Thus the judgment does not mean “the implementation happened to succeed.” Appendix B gives its constructor coverage.

Write $\text{Certificates}_\Omega(q, \Sigma)$ for the finite conjunction of the carrier, numeric, cardinality, codec, context-extent, stable-ID, three-valued non-true, and result-shape certificates selected by a derivation for q . This predicate contains no preservation equation and no claim about the success of a concrete executor. In particular, R1–R3 below are theorem conclusions, not members of $\text{Certificates}_\Omega$.

For the published grammar in Sect. 3, every constructor is already canonical, so

$$\text{Norm}_Q(q) = q.$$

Hence termination, idempotence, typing preservation, and denotation preservation of Norm_Q follow immediately. This explicit identity definition ensures that the realization theorem uses exactly the published formal grammar.

Theorem 4.8 (R50: adequacy of the fixed graph-observer templates) *If*

$$\text{WF}_Q^\Sigma(q) \quad \text{and} \quad \text{WF}_{\text{RepSpec}}(MM, \Sigma),$$

then $\text{ObserverAdequate}_5(q, \Sigma)$. Equivalently, for every graph observer reachable from q , the fixed Rule 06 target template selected by its unique catalogue signature denotes the corresponding Q observer on every runtime pair satisfying $\text{Adequate}_\Sigma(M, G)$.

Proof. Fix an arbitrary adequate M, G and eliminate the existential adequacy witness γ . Catalogue totality and signature uniqueness give one template for each reachable observer. A finite case split covers context, allInstances, stable identity, scalar attribute, ordinary qualified navigation, association-class navigation, and type test/cast. Context follows from binder agreement; allInstances follows from the unique direct-type edge and copied inheritance closure; identity and attribute follow from A-ID, A-ATTR, codec left-inversion, and the absence/bottom split. For navigation, relationship occurrences and matched rows are in a multiplicity-preserving bijection; qualifier predicates are componentwise

equivalent by codec injectivity, while upper-one, Set, and Bag packaging use A-NAV1, A-NAVM, and A-MULT. The association-class case uses the occurrence node, rather than the participant pair, as identity. Type tests use reachability through the finite copied inheritance relation, and casts add only the lazy success/bottom branch. Bottom receiver and qualifier guards are outer lazy cases in every applicable template. These cases exhaust the graph-reading Q constructors, and structural traversal of q only unions their already-proved obligations. $\square$

Listing 3: Formal Q-to-target realization.

```

1 RealizeIR-SIGMA(qroot) = p,
2   where q0 = Norm-Q(qroot),
3         K = contextClass(q0), x = contextBinder(q0),
4         y = leastFresh(name(x), emptyXi.fresh),
5         Xi0 = extend(emptyXi, x, y),
6         (targetBody, Xi1) = Realize-E-SIGMA(Xi0, body(q0)),
7         p = TQuery(mode(q0), K, y, targetBody,
8                   observerSigma(K), idObserverSigma(K),
9                   resultAliasSigma(K), contractSigma(K), Xi1.piGen)
10
11 Realize-E-SIGMA(Xi, q) =
12   Constant(v) => Leaf-SIGMA(Xi, v)
13   Parameter(p) => (TParam(publicName(p)), addPublic(Xi, p))
14   QBottom(tau) => (TBottom(tau), Xi)
15   Var(x) => (TVar(Xi.chi(x)), Xi)
16   QCoerce(h, q) =>
17     let (t, Xi1) = Realize-E-SIGMA(Xi, q)
18     in (TCoerce(h, t), Xi1)
19   QLet(x, q1, q2) =>
20     let (t1, Xi1) = Realize-E-SIGMA(Xi, q1),
21         y = leastFresh(name(x), Xi1.fresh),
22         (t2, Xi2) = Realize-E-SIGMA(extend(Xi1, x, y), q2)
23     in (TLet(y, t1, t2), Xi2)
24   QIf(c, t, f) =>
25     let (tc, Xi1) = Realize-E-SIGMA(Xi, c),
26         (tt, Xi2) = Realize-E-SIGMA(Xi1, t),
27         (tf, Xi3) = Realize-E-SIGMA(Xi2, f)
28     in (TCase(tc, tt, tf), Xi3)
29   ReadAttribute(q, a) =>
30     let (t, Xi1) = Realize-E-SIGMA(Xi, q)
31     in (TObserveAttr(t, observerSigma(a)), Xi1)
32   NavigateOne(q, r, qs) =>
33     let (t, Xi1) = Realize-E-SIGMA(Xi, q),
34         (tqs, Xi2) = mapState(Realize-E-SIGMA, Xi1, qs)
35     in (TObserveOne(t, tqs, observerSigma(r)), Xi2)
36   NavigateAssociationClassOne(q, A, r, qs) =>
37     let (t, Xi1) = Realize-E-SIGMA(Xi, q),
38         (tqs, Xi2) = mapState(Realize-E-SIGMA, Xi1, qs)
39     in (TObserveOne(t, tqs, observerSigma(A, r)), Xi2)
40   QTypeTest(q, K, b) =>
41     let (t, Xi1) = Realize-E-SIGMA(Xi, q)
42     in (TTypeTest(t, observerSigma(K), b), Xi1)
43   QTypeCast(q, K) =>
44     let (t, Xi1) = Realize-E-SIGMA(Xi, q)
45     in (TTypeCast(t, observerSigma(K)), Xi1)
46   QUn(op, q) =>
47     let (t, Xi1) = Realize-E-SIGMA(Xi, q)
48     in (TOp(certSigma(op), [t]), Xi1)
49   QBin(op, q1, q2) =>
50     let (t1, Xi1) = Realize-E-SIGMA(Xi, q1),
51         (t2, Xi2) = Realize-E-SIGMA(Xi1, q2)
52     in (TOp(certSigma(op), [t1, t2]), Xi2)
53   QCollection(kind, items) =>
54     let (ts, Xi1) = mapState(Realize-E-SIGMA, Xi, items)
55     in (TList(kind, ts), Xi1)

```

```

56 MaterializeQ(P) =>
57   let (tp,Xi1) = Realize-P-SIGMA(Xi,P)
58   in (TMaterialize(tp),Xi1)
59 ForAll3(P,x,b) =>
60   let (tp,Xi1) = Realize-P-SIGMA(Xi,P),
61   y = leastFresh(name(x),Xi1.fresh),
62   (tb,Xi2) = Realize-E-SIGMA(extend(Xi1,x,y),b)
63   in (TForAll3(tp,y,tb),Xi2)
64 Exists3(P,x,b) =>
65   let (tp,Xi1) = Realize-P-SIGMA(Xi,P),
66   y = leastFresh(name(x),Xi1.fresh),
67   (tb,Xi2) = Realize-E-SIGMA(extend(Xi1,x,y),b)
68   in (TExists3(tp,y,tb),Xi2)
69
70 Realize-P-SIGMA(Xi,P) =
71   FromCollection(q) =>
72     let (t,Xi1) = Realize-E-SIGMA(Xi,q)
73     in (TFrom(t),Xi1)
74   ScanClass(K) => (TScan(observerSigma(K)),Xi)
75   NavigateMany(kind,q,r,qs) =>
76     let (t,Xi1) = Realize-E-SIGMA(Xi,q),
77     (tqs,Xi2) = mapState(Realize-E-SIGMA,Xi1,qs)
78     in (TObserveMany(kind,t,tqs,observerSigma(r)),Xi2)
79   NavigateAssociationClass(kind,q,A,r,qs) =>
80     let (t,Xi1) = Realize-E-SIGMA(Xi,q),
81     (tqs,Xi2) = mapState(Realize-E-SIGMA,Xi1,qs)
82     in (TObserveMany(kind,t,tqs,observerSigma(A,r)),Xi2)
83   Filter(filterKind,P,x,b) =>
84     let (tp,Xi1) = Realize-P-SIGMA(Xi,P),
85     y = leastFresh(name(x),Xi1.fresh),
86     (tb,Xi2) = Realize-E-SIGMA(extend(Xi1,x,y),b)
87     in (TFilter(filterKind,tp,y,tb),Xi2)
88   ProjectQ(kind,P,x,b) =>
89     let (tp,Xi1) = Realize-P-SIGMA(Xi,P),
90     y = leastFresh(name(x),Xi1.fresh),
91     (tb,Xi2) = Realize-E-SIGMA(extend(Xi1,x,y),b)
92     in (TProject(kind,tp,y,tb),Xi2)
93   CollectQ(P,x,b) =>
94     let (tp,Xi1) = Realize-P-SIGMA(Xi,P),
95     y = leastFresh(name(x),Xi1.fresh),
96     (tb,Xi2) = Realize-E-SIGMA(extend(Xi1,x,y),b)
97     in (TCollect(tp,y,tb),Xi2)
98   PlanLet(x,q,P) =>
99     let (tq,Xi1) = Realize-E-SIGMA(Xi,q),
100    y = leastFresh(name(x),Xi1.fresh),
101    (tp,Xi2) = Realize-P-SIGMA(extend(Xi1,x,y),P)
102    in (TPlanLet(y,tq,tp),Xi2)
103   DistinctQ(P) =>
104     let (tp,Xi1) = Realize-P-SIGMA(Xi,P)
105     in (TDistinct(tp),Xi1)
106
107 Leaf-SIGMA(Xi,v) =
108   (TConst(encodeSigma(v)),Xi) if inlineSigma(v),
109   (TParam(u),addGenerated(Xi,u,encodeSigma(v))) otherwise,
110   where u = leastFreshParameter(type(v),Xi.fresh)

```

The displayed equations cover every expression and plan constructor in the published Q grammar. The target semantics of $TForAll3$ and $TExists3$ expands them into the priority-ordered Boolean-bottom cases specified in Appendix A. The equations are cross-checked by the exhaustive constructor table in Appendix B. A variable case has the premise $x \in \text{dom}(\chi)$; observer and primitive cases have the corresponding Σ and carrier certificates. The root checks only source-independent catalogue, profile, contract, and lexical side conditions; $WF_{\text{Target},\Sigma}(p)$ is a postcondition proved by construction, not a premise about the output being built. Without the independent premises no realization rule applies. The function Leaf_Σ applies the one canonical inline-versus-generated-parameter policy fixed by the target profile, so

the constant case is deterministic and every generated parameter is inserted once into π_{gen} . Realization does not serialize text: its mathematical result is the typed target artifact p . Generated values and descriptors are owned by p and are exported as π_{gen} only by serialization.

Theorem 4.9 (Q-to-target well-formedness and preservation) *Let q be well typed and let the public root be certified by $\text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega)$. Every successful realization*

$$\text{Realize}_{\Sigma}^{IR}(\text{Violations}(K, q)) = \text{Success}(p)$$

satisfies $\text{WF}_{\text{Target}, \Sigma}(p) \wedge \text{LexWF}_5(p)$. The structural component follows from the target constructors and the lexical component from the root's LexAdmissible_5 premise and injective codecs. Theorem 4.8 supplies the observer equations; the mutual induction proves that every recursive expression and plan result denotes the same value or occurrence sequence as its Q input for every runtime state covered by the certificates in Ω . At the public violation root the construction establishes

$$\mathbf{R1} : \forall g \in \text{Extent}_{\Sigma}^{\Sigma}(K), B_Q(g) = B_T^p(g),$$

$$\mathbf{R2} : \forall j \in \text{ID}, \text{Ret}^p(j) \iff \exists g \in \text{Extent}_{\Sigma}^{\Sigma}(K) : \text{id}_G(g) = j \wedge B_T^p(g) \neq \top.$$

Proof. Use mutual structural induction on normalized Q expressions and plans.

Base case. For a constant v , target admission provides a codec/carrier certificate. The fixed leaf policy creates either its canonical tagged target value or a fresh generated parameter with that value in π_{gen} . Codec left-inversion or parameter lookup therefore yields v ; the target node is closed relative to the declared/generated parameter contract and is well scoped.

Inductive let case. Suppose $q = \text{QLet}(x, q_1, q_2)$. The first induction hypothesis gives a well-formed target t_1 with the same denotation as q_1 . The least-fresh rule selects y outside the finite live-name set and extends χ with $x \mapsto y$. The environment-extension lemma relates evaluation of q_2 under $x \mapsto v$ to evaluation of t_2 under $y \mapsto \hat{v}$; freshness prevents capture. The second induction hypothesis gives equal body denotations. The target let evaluates t_1 once and then t_2 , so it equals the Q let and is well scoped.

Observer cases use Theorem 4.8; collection, iterator, numeric, and query-root cases use the specialized lemmas recorded in Appendix C. At the violation root, body preservation gives R1 and the explicit non-true, stable-ID, and distinct construction gives R2. For lexical well-formedness, the same induction uses the LexAdmissible_5 premise at every constructor: fixed productions cover composite nodes, codec-domain premises cover terminals, and leastFresh supplies admissible non-capturing binder and parameter names. Hence the completed finite artifact also satisfies LexWF_5 . $\square$

4.6 Algorithm 4: Target Serialization

Serialization is a syntax-directed mathematical printer over well-formed target artifacts. It threads precedence, prints every binder reference from target identity, and copies rather than recomputes the parameter/result side channels. The following equations specify the canonical text; they do not describe calls to a concrete rendering library.

$\text{LexWF}_5(p)$ means that every target constructor has a production in the admitted Cypher 5 text profile, every identifier and literal lies in its injective codec domain, and every generated name satisfies the profile's lexical and freshness constraints. Observer descriptors do not store raw query templates. Realization has already selected one of the finite fixed Rule 06 target constructors from the descriptor's typed metadata. The serializer therefore prints an ordinary target AST, and the required left-inverse is proved for the fixed grammar production together with the identifier, property, relationship-type, literal, and parameter codecs.

Listing 4: Formal canonical target serialization.

```

1 Serialize(p) = (PrintQuery(p.query), Delta(p), piGen(p))
2
3 PrintExpr(TConst(c), k) = encode(c)
4 PrintExpr(TParam(u), k) = parameterName(u)
5 PrintExpr(TVar(y), k) = identifier(y)
6 PrintExpr(TBottom(tau), k) = encodeBottom(tau)
7 PrintExpr(TCoerce(h, t), k) = emitCoerce(h, PrintExpr(t, 0))
8 PrintExpr(TOp(op, [t_i]), k) = delimited(op, [PrintExpr(t_i, prec(op))])
9 PrintExpr(TLet(y, t1, t2), k) =
10   "head([" identifier(y) " IN [" PrintExpr(t1, 0) " ] | "
11     PrintExpr(t2, 0) "])"
12 PrintExpr(TCase(c, t, f), k) =
13   "CASE WHEN " PrintExpr(c, 0) " THEN " PrintExpr(t, 0)
14   " ELSE " PrintExpr(f, 0) " END"

```

```

15 PrintExpr(TObserveAttr(t,w),k) = printFixedAttr(w,PrintExpr(t,0))
16 PrintExpr(TObserveOne(t,qs,w),k) =
17   printFixedOne(w,PrintExpr(t,0),map(PrintExpr,qs))
18 PrintExpr(TTypeTest(t,w,b),k) = printFixedTypeTest(w,b,PrintExpr(t,0))
19 PrintExpr(TTypeCast(t,w),k) = printFixedTypeCast(w,PrintExpr(t,0))
20 PrintExpr(TList(kind,[t_i]),k) =
21   encodeCollection(kind,[PrintExpr(t_i,0)])
22 PrintExpr(TMaterialize(d),k) = emitMaterialize(PrintPlan(d))
23 PrintExpr(TForAll3(d,y,t),k) = emitForAll3(PrintPlan(d),y,PrintExpr(t,0))
24 PrintExpr(TExists3(d,y,t),k) = emitExists3(PrintPlan(d),y,PrintExpr(t,0))
25
26 PrintPlan(TFrom(t)) = emitFrom(PrintExpr(t,0))
27 PrintPlan(TScan(wK)) = emitScan(wK)
28 PrintPlan(TObserveMany(kind,t,qs,w)) =
29   printFixedMany(kind,w,PrintExpr(t,0),map(PrintExpr,qs))
30 PrintPlan(TFilter(filterKind,d,y,t)) =
31   printFixedFilter(filterKind,PrintPlan(d),y,PrintExpr(t,0))
32 PrintPlan(TProject(kind,d,y,t)) =
33   emitProject(kind,PrintPlan(d),y,PrintExpr(t,0))
34 PrintPlan(TCollect(d,y,t)) =
35   emitCollect(PrintPlan(d),y,PrintExpr(t,0))
36 PrintPlan(TPlanLet(y,t,d)) =
37   emitPlanLet(y,PrintExpr(t,0),PrintPlan(d))
38 PrintPlan(TDistinct(d)) = emitDistinct(PrintPlan(d))
39
40 PrintQuery(TQuery(mode,K,y,t,wK,wId,alias,DeltaP,piGen)) =
41   emitRoot(mode,K,y,PrintExpr(t,0),wK,wId,alias,DeltaP)

```

These equations are defined on $WF_{\text{Target},\Sigma} \cap \text{LexWF}_5$. Precedence and delimiter functions are mathematical components of the canonical text profile; injective codecs fix escaping of identifiers, strings, and parameters. The functions named `emitFrom` through `emitRoot` and the `printFixed*` functions are the finite production table of the canonical profile, not implementation calls or catalogue-provided text. Appendix B prints their grammar productions and inverse-parser obligations. Consequently every target constructor above has one serialization production and there is no prose-only fallback case.

Theorem 4.10 (Serialization transport) *If p is finite, target-well-formed, lexically admissible, and $\text{Serialize}(p) = \text{Success}(s, \Delta, \pi_{\text{gen}})$, then*

$$\text{parse}_5(s) = \text{Success}(\text{Syn}(\text{eraseLoc}(p)))$$

up to internal alpha-renaming, and

$$\text{TextSem}_5(s, G, \pi) = A_5(p, G, \pi).$$

Serialization preserves Δ and π_{gen} exactly.

Proof. Simultaneous structural induction on target expressions, patterns, clauses, and queries.

Base case. A parameter reference is printed as its canonical lexical parameter name. Injectivity of the parameter-name codec makes the parser recover the same parameter identity, and both semantics perform the same lookup.

Inductive composite case. For a typed operator node, the induction hypotheses recover every printed child. Canonical delimiters and precedence parentheses identify one unique parent production, so parsing recovers the same operator tree. Compositional target semantics then gives equal denotation. Binder forms additionally use alpha-invariance; clauses and query roots use the same argument on the syntactic projection. The non-textual maps Δ and π_{gen} are proved equal by direct projection from the serialization package, not by pretending that the text parser reconstructs their values. The exhaustive cases are recorded in Appendix C. $\square$

Remark 4.11 (Boundary of the serialization result) *Theorem 4.10 is unconditional only after a concrete target profile has discharged LexWF_5 . In particular, each identifier, literal, parameter, and observer-template codec must have the stated left-inverse property, and the productions must be unambiguous under the profile parser. Thus the theorem proves every artifact generated within a certified Cypher 5 profile; it does not by itself certify arbitrary Cypher text, derive the certificates from Neo4j documentation, or prove that a particular Neo4j release implements TextSem_5 . The latter is the separate executor-refinement obligation stated in Sect. D.3.*

4.7 Generated-Query Refinement

Let target execution and decoding succeed with $J \subseteq \text{ID}$. The runtime membership obligation is

$$\mathbf{R3} : \quad \forall j \in \text{ID}, j \in J \iff \text{Ret}^p(j).$$

Lemma 4.12 (Formal-text membership) *Let p be a finite violation-mode target with $\text{WF}_{\text{Target}, \Sigma}(p) \wedge \text{LexWF}_5(p)$ and $\text{Serialize}(p) = \text{Success}(s, \Delta, \pi_{\text{gen}})$. If the admissible combined parameter map is π and formal text execution and decoding succeeds with J , then R3 holds for this same p, J .*

Proof. Theorem 4.10 rewrites formal text evaluation of s to $A_5(p, G, \pi)$. By the violation-root equation in Appendix A, successful target execution followed by the copied decoder yields exactly $\{j \mid \text{Ret}^p(j)\}$. Extensionality gives $j \in J \iff \text{Ret}^p(j)$ for arbitrary j . $\square$

Lemma 4.13 (Generated-query refinement) *Suppose q is well typed and the following realization premises hold:*

$$\begin{aligned} & \text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega), \\ & \text{Realize}_{\Sigma}^{IR}(\text{Violations}(K, q)) = \text{Success}(p), \\ & \text{Witness}_{\Sigma}(p; G, K, q, s, \Delta, \pi_{\text{gen}}). \end{aligned}$$

If public and generated parameters are admissible and disjoint and formal text execution and decoding succeed, then

$$\text{Obs}_Q(G, K, q) = \text{Viol}_C(G, s; \pi, \Delta).$$

Proof. Theorem 4.9 supplies R1–R2 for p ; the witness and Theorem 4.10 satisfy the premises of Lemma 4.12, which supplies R3. For arbitrary j , R3 rewrites membership in the decoded result to $\text{Ret}^p(j)$. R2 rewrites this predicate to existence of an extent object with identifier j and a non-true target body. R1 replaces that body by the Q body. This is exactly membership in $\text{Obs}_Q(G, K, q)$. $\square$

4.8 Conditional End-to-End Preservation

Theorem 4.14 (Conditional exact preservation of violation IDs) *Fix a metamodel MM , an instance M , a graph G , a representation specification Σ , and an invariant inv . Assume:*

- P1.** $MM \in \text{UML}_{\mathcal{F}}$, $\text{ValidInstance}(MM, M)$, and $\text{WF}_{MM}(M)$; M is finite; and inv is supported and well typed under MM .
- P2.** $\text{Adequate}_{\Sigma}(M, G)$.
- P3.** $\text{Compile}_{MM, \Sigma}(\text{inv}) = \text{Success}(s, \Delta, \pi_{\text{gen}})$. Take K, e, q from the successful frontend, Core-to- Q , and public-realization equations.
- P4.** The following independently checkable realization premises hold:

$$\begin{aligned} & \text{WF}_{\text{RepSpec}}(MM, \Sigma), \quad \text{Certificates}_{\Omega}(\text{Violations}(K, q), \Sigma), \\ & \text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega). \end{aligned}$$

Moreover, there is a p such that

$$\begin{aligned} & \text{Realize}_{\Sigma}^{IR}(\text{Violations}(K, q)) = \text{Success}(p), \\ & \text{Witness}_{\Sigma}(p; G, K, q, s, \Delta, \pi_{\text{gen}}). \end{aligned}$$

- P5.** $\pi_{\text{public}} \in \text{Dom}_{\text{pub}}$, $\text{dom}(\pi_{\text{public}}) \cap \text{dom}(\pi_{\text{gen}}) = \emptyset$, and $\pi = \pi_{\text{public}} \uplus \pi_{\text{gen}}$. All applicable runtime numeric and carrier certificates cover the run, and $\text{bind}(\text{TextSem}_5(s, G, \pi), \text{decode}_{\Delta}) = \text{Success}(J)$ with $J \subseteq \text{ID}$.

Then

$$\boxed{\text{Viol}_O(M, \text{inv}) = \text{Viol}_C(G, s; \pi, \Delta)}.$$

Proof. Theorem 4.4, Theorem 4.6, and Lemma 4.7 give

$$\text{Viol}_O(M, \text{inv}) = \text{Obs}_Q(G, K, q).$$

From P4, Theorem 4.8 derives observer adequacy and Theorem 4.9 derives R1–R2. From P5, Lemma 4.12 derives R3 for the same p, J ; none of R1–R3 is an assumption. Therefore Theorem 4.10 and Lemma 4.13 give

$$\text{Obs}_Q(G, K, q) = \text{Viol}_C(G, s; \pi, \Delta).$$

Transitivity yields the conclusion. No premise is obtained from the conclusion, so the composition is non-circular. $\square$

Definition 4.15 (One-way refinement by a concrete Neo4j executor) Write

$$\text{Exec}_{\text{Neo4j}} \sqsubseteq_{\text{Gen5}} \text{TextSem}_5$$

iff, for every package $(s, \Delta, \pi_{\text{gen}})$ generated from a C5Realisable root, every adequate graph G , and every admissible combined parameter map π , the following implication holds for every R_f and $J \subseteq \text{ID}$:

$$\begin{aligned} \text{TextSem}_5(s, G, \pi) = \text{Success}(R_f) \wedge \text{decode}_\Delta(R_f) = \text{Success}(J) \\ \implies \exists R_n. \text{Exec}_{\text{Neo4j}}(s, G, \pi) = \text{Success}(R_n) \wedge \text{decode}_\Delta(R_n) = \text{Success}(J). \end{aligned}$$

For a successful concrete run define

$$\begin{aligned} \text{Viol}_C^{\text{Neo4j}}(G, s; \pi, \Delta) = J \iff \exists R_n. \text{Exec}_{\text{Neo4j}}(s, G, \pi) = \text{Success}(R_n) \\ \wedge \text{decode}_\Delta(R_n) = \text{Success}(J). \end{aligned}$$

This is deliberately one-way: the report neither assigns a meaning to failed or out-of-profile executions nor claims equivalence for arbitrary Cypher text.

Corollary 4.16 (Exact violation preservation for a refining Neo4j executor) Under P1–P5 of Theorem 4.14, if $\text{Exec}_{\text{Neo4j}} \sqsubseteq_{\text{Gen5}} \text{TextSem}_5$, then the generated concrete run succeeds and

$$\text{Viol}_O(M, \text{inv}) = \text{Viol}_C^{\text{Neo4j}}(G, s; \pi, \Delta).$$

Proof. P5 and the definition of bind provide an R_f with $\text{TextSem}_5(s, G, \pi) = \text{Success}(R_f)$ and $\text{decode}_\Delta(R_f) = \text{Success}(J)$. Refinement supplies a successful concrete result R_n with the same decoded set J . Hence

$$\text{Viol}_C^{\text{Neo4j}}(G, s; \pi, \Delta) = J.$$

Theorem 4.14 gives $\text{Viol}_O(M, \text{inv}) = J$; transitivity proves the claim. $\square$

4.9 Translation Completeness Boundary

Define the logical translation domain $\text{Inv}_{\text{val}}(MM, \Sigma)$ independently by requiring admitted resolved syntax and typing, recursively admitted children, and one type-correct Σ observer for each referenced model declaration. It does not mention compiler success, Cypher lexical admissibility, or backend numeric/carrier certificates. Those latter requirements belong to the separate backend judgment $\text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega)$.

Theorem 4.17 (Two-level translation completeness) For every invariant in $\text{Inv}_{\text{val}}(MM, \Sigma)$, the logical stages Front_{MM} and T_Σ have a successful image. If the resulting Q body has context class K and its public root additionally has a derivation of $\text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega)$, realization and serialization have a successful formal Cypher 5 artifact.

Proof. We separate logical completeness from backend completeness so that neither domain is defined through success of the function it is intended to justify.

Logical base cases. An admission derivation for a literal contains its resolved primitive type; the matching frontend equation constructs the corresponding typed Core constant, and the matching T_Σ^E equation constructs the corresponding Q constant. For a variable, the admission premise supplies $x \in \text{dom}(\Gamma)$; both transformations copy that resolved identity. Hence the recursive call sequence exists and succeeds in either base case.

Logical composite case. Suppose the final admission rule constructs $C(a_1, \dots, a_n)$. Each premise admits a strict child, so the induction hypotheses produce Core and Q images for all children. The remaining premises of that same admission rule supply exactly the constructor-specific conditions used by the matching equations: equal branch types for `lf`, operand types for operators, resolved multiplicity and collection kind for navigation, or the unique type-correct Σ descriptor for an observer. Therefore the equation for C has no rejecting premise and constructs the parent image.

Logical binder case. For `LetO`(x, a_1, a_2), the admission derivation admits a_1 under Γ and a_2 under $\Gamma, x : \tau$. Applying the induction hypothesis under those two environments yields both children, after which the let equations construct the Core and Q binders. Iterator and plan binders have the same derivation shape. Because identities, rather than spelling, are copied, this step cannot fail through capture. These base, composite, and binder cases cover the logical grammars by the coverage table in Appendix B.

Backend expression/plan cases. We next use mutual induction on the independent C5Realisable derivation. A leaf rule carries the required literal codec, parameter, or finite carrier certificate and therefore constructs its target leaf. A

composite rule first applies the mutual induction hypotheses to its expression and plan children, then uses the operator or observer certificate in its conclusion to construct one well-formed target parent. For a binder, the used-identity set is finite while the target identity namespace is infinite; hence `leastFresh` exists. Extending Ξ with this identity enables the body induction hypothesis and produces a capture-free target binder.

Violation-root case. The final root rule for $\text{Violations}(K, q)$ contains certificates for a context-class extent, three-valued non-true selection, stable-ID projection, distinctness, and the one-ID-column decoder. The expression induction result for q , together with these certificates, makes $\text{Realize}_{\Sigma}^{IR}$ construct a finite $\text{WF}_{\text{Target}, \Sigma}$ query rather than reject.

Serialization totality. The root derivation also supplies LexWF_5 . Structural recursion of `PrintExpr`, `PrintPlan`, and `PrintQuery` is on strict children of the finite target tree. Every target constructor has one profile production and every terminal has a codec in its domain. Thus the printer terminates with some text s and copies the finite side channels Δ and π_{gen} , yielding $\text{Success}(s, \Delta, \pi_{\text{gen}})$. $\square$

This theorem does not say that every OMG OCL expression is translatable, that every mathematical integer or real has a database carrier, or that a concrete Neo4j executor refines the formal text semantics. Those are separate domain or runtime questions. It is a companion-report result and is not used to strengthen the main paper's conditional soundness theorem.

5 Worked ER2REL Derivation and Evidence

This section follows one invariant through every formal boundary and then reports finite implementation observations. It keeps two transformations distinct:

$$M_{ER} : MM_{ER} \xrightarrow{\text{ER2REL/ATL}} M_{REL} : MM_{REL}, \quad REL_K \xrightarrow{\text{OCL2Cypher}} z.$$

The ATL transformation produces a REL instance. OCL2Cypher compiles an invariant defined by MM_{REL} and validates that instance through its graph representation. Agreement of the validation results is not a proof that the ATL transformation is correct for every ER input.

5.1 ER and REL Setting

The ER input contains entities `Customer` and `Order`, their key attributes `customerId` and `orderId`, and relationship `Places`. The ATL rules create three REL relations:

```
Customer : customerIdkey, name,
Order    : orderIdkey, total,
Places   : createdAt, customerIdkey, orderIdkey.
```

The resulting REL instance contains one schema, three relations, and seven attributes. It is the runtime instance M for the validation example; the REL Ecore definition supplies the corresponding MM .

The selected invariant is

```
1 context Relation inv REL_K:
2   self.attrs->exists(a | a.isKey)
```

and requires every relation to have at least one key attribute. Stable identifiers are checked deterministic names:

$$\text{Relation } n \mapsto \text{relation}:n, \quad \text{RELAttribute } (r, a) \mapsto \text{rel-attr}:r/a.$$

Duplicate names in either key domain are rejected instead of being silently merged.

5.2 Resolved Source, Core, and Q

Let

$$\mathbf{r} = \text{relationAttrs} :: \text{attrs}, \quad \mathbf{a} = \text{RELAttribute} :: \text{isKey}.$$

The resolved source body is

$$\text{ExistsO}(\text{NavManyO}_{\text{Set}}(\text{self}_1, \mathbf{r}), a_2, \text{AttrO}(a_2, \mathbf{a})).$$

The subscripts are declaration identities, not textual-name decorations. Front_{MM} produces

$$e = \text{Exists}(\text{NavMany}_{\text{Set}}(\text{Var}(\text{self}_1), \mathbf{r}), a_2, \text{Attr}(\text{Var}(a_2), \mathbf{a})).$$

The navigation typing rule uses the resolved upper bound and uniqueness of `attrs` to select `Set`. The iterator rule extends the environment with $a_2 : \text{Class}(\text{RELAttribute})$.

T_Σ produces

$$q = \text{Exists3}(\text{NavigateMany}_{\text{Set}}(\text{Var}(\text{self}_1), \mathbf{r}), a_2, \text{ReadAttribute}(\text{Var}(a_2), \mathbf{a})).$$

The Q term no longer denotes an unspecified association lookup: the catalogue entries selected by Σ fix the physical relationship pattern for $\mathbf{r}$ and the typed slot observation for $\mathbf{a}$. `Exists3` also fixes true, bottom, false priority and the false result on an empty defined collection.

5.3 Graph Witness and Target Artifact

The canonical graph used by each fixture has 37 nodes and 62 relationships. The observations relevant to this invariant are:

- three observable `Relation` objects in the context extent;
- seven observable `RELAttribute` objects;
- one represented `attrs` link occurrence per logical link;
- one typed Boolean slot for `RELAttribute::isKey`;
- equal stable identifiers on every paired object.

These finite checks instantiate, but do not universally prove, Definition 2.4. The abstract target contains a context scan, a representation-selected relationship match, typed attribute-slot decoding, an explicit three-valued existential fold, a non-true filter, and a distinct stable-ID projection.

The package has no public OCL parameter. Its generated bindings identify the model, context class, association end, and attribute:

<code>__oclModelKey</code>	$\mapsto$	<code>er2rel-valid</code> ,
<code>__oclContextClassKey</code>	$\mapsto$	<code>Relation</code> ,
<code>__oclRole_2</code>	$\mapsto$	<code>attrs</code> ,
<code>__oclAttr_8</code>	$\mapsto$	<code>RELAttribute::isKey</code> .

The result contract is

$$\Delta = (\emptyset, \text{violationId}, \text{decodeIDSet}).$$

A shortened explanatory rendering is

```

1 MATCH (self:Object)-[:ObjectInstanceOf]->(:UmlClass)
2   -[:Extends*0..3]->(:UmlClass {classKey: $contextClass})
3 OPTIONAL MATCH
4   (self)-[:LinkAssociateWith {targetRole: $role}]->(a:Object)
5 OPTIONAL MATCH
6   (a)-[:ObjectHasAttribute]->
7   (slot:AttributeValue {attributeKey: $attribute})
8 WITH self, /* typed Exists3 fold over decoded slots */ AS body
9 WHERE body.bottom = true OR body.value = false
10 RETURN DISTINCT self.use_id AS violationId

```

This display is explanatory, not a replacement for the semantic target artifact. In particular, the generated query retains scope-safe bind-once forms, tagged carriers, and explicit bottom branches that the abbreviation does not show.

5.4 Observed Results

Two fixtures were evaluated. The positive fixture is the actual ATL output. The controlled negative fixture changes only the two derived key flags of `Places`; it is not claimed to be ATL output.

The comparison normalizes tool-specific object names to the checked stable-ID convention. The result shows no extra or missing identifier in these two runs. It is finite evidence for the implementation and selected deployment, not a premise of Theorem 4.14.

Table 2: ER2REL violation-ID observations.

Fixture	Observers	Common result
ATL-produced REL model	USE, Core, Q , decoded Neo4j	$\emptyset$
Controlled no-key model	USE, Core, Q , decoded Neo4j	$\{\text{relation} : \text{Places}\}$

Table 3: Finite evaluation summary.

Subject	Main risk	Core- Q	Rejection	Live result
Library	Collection, bottom, numeric bounds	78/78	1/1	–
CarRental	Domain constraints and admission	19/19	6/6	19/19
DeepHierarchy	Inherited context extent	36/36	–	–
Medical	Navigation and iterators	36/36	–	36/36
Feature matrix	Cross-family combinations	30/30	1/1	30/30
Boundary corpus	Unsupported inputs	–	15/15	–
Total		199/199	23/23	85/85

5.5 Broader Finite Evaluation

The broader corpus addresses three empirical questions: Core- Q agreement, prescribed rejection, and end-to-end execution. Its reported observations are summarized below.

All reported Core- Q comparisons agreed, all designated negatives were rejected at the prescribed boundary, and all successfully decoded live runs returned the expected finite ID set. These counts neither establish constructor coverage nor prove universal executor refinement.

5.6 Threats to Validity

Source and target evaluators share schema and identifier assumptions, creating a risk of correlated defects. Fixed expected-ID cases and comparisons at multiple intermediate boundaries reduce but do not eliminate this risk. The corpus is finite and not industrial scale. Finally, live execution covers a reported configuration rather than every Cypher implementation. These limits are why the empirical results are presented after, and separately from, the mathematical proof.

6 Related Work and Assurance Positioning

6.1 OCL Execution and Compilation

USE evaluates OCL directly over UML instances [9]. SQL-PL4OCL and OCL2PSQL compile supported OCL expressions to relational queries [8, 12]. Nguyen and Clavel give an SMT-based method for proving manually written SQL implementations correct for a substantial OCL subset [13]. These approaches clarify source execution and relational compilation, but do not by themselves relate a property-graph representation, a representation-aware query IR, and the stable-ID validation observation considered here.

Graph-backed model repositories include Neo4EMF and NeoEMF [1, 5]. Mogwai compiles model queries to graph traversals [6, 3], while Gremlin-ATL translates ATL transformations whose expressions use OCL into Gremlin scripts [4]. Other work derives graph representations from UML schemas [7, 11]. Such work motivates executable graph processing; the present report adds an explicit observational adequacy premise and a layered preservation argument.

At the formal boundary, Essential OCL invariants have been related to nested graph constraints [15], and Graphiti checks equivalence between supported Cypher and SQL fragments relative to a database transformer [10]. OCL2Cypher differs in the observation and factorization: it connects resolved OCL validation, a chosen graph representation, Core and Q semantics, target realization, serialization, execution, and stable-ID decoding.

6.2 Why Layered Proof Is Needed

Testing can expose wrong queries, rejected valid inputs, and incorrect result projection, but it cannot quantify over every admitted expression, environment, instance, or graph. Conversely, a calculus proof does not show that a Java compiler or database engine implements that calculus. The assurance chain therefore distinguishes four layers: mathematical definitions and hand proofs; Lean-checked kernels and a vertical slice; implementation tests; and finite executions on a

concrete deployment. Every claim is stated at the weakest boundary that supports it: tests are not called theorems, and a kernel theorem is not presented as full Java refinement.

6.3 Mechanized Scope

The Lean development machine-checks the following mathematical components.

- total tagged codecs and successful-decoding characterizations for Boolean3, Integer, rational Real payloads, String, object identity, and typed bottom;
- Boolean3 operators and universal finite-fold laws;
- selected integer range and exact rational Real properties;
- flat Set-support and Bag-multiplicity kernels;
- a closed inventory of the formal Q syntax;
- a Boolean vertical slice containing literals, variables, not/and/or/implies, if, and let through source, Core, Q , formal target, and violation projection;
- an AST/token-level serialization round trip for the admitted slice;
- conditional pointwise and set-level end-to-end composition, given the explicit stage-refinement witnesses.

The checked modules contain no project axiom, sorry, or admit. Standard Lean principles reported by the axiom audit include propositional extensionality, classical choice for finite extensional collection equality, and quotient soundness where imported library representations require them.

The following boundaries remain hand proved or external:

- the complete frontend, graph, Core-to- Q , realization, and serialization constructor families of the formal calculus;
- the complete recursive Cypher text parser/printer and decoder;
- conformance of a concrete Neo4j executor to the formal text semantics.

Accordingly, the report uses the labels *Lean checked*, *hand proved*, *assumed refinement*, and *finite evidence* literally. Appendix D maps theorem families to their current status.

6.4 Relationship Between the Formal Profiles

$\mathcal{F}_{\text{paper}}$ is the fragment claimed by the conference paper. $\mathcal{F}_5$ is the independently certified target profile used by the formal realization and serialization theorems. Inclusion $\mathcal{F}_{\text{paper}} \subseteq \mathcal{F}_5$ is justified only by the constructor-by-constructor matrix in Appendix B.

The formal target theorem and concrete executor corollary are also distinct:

$$\text{TextSem}_5(s, G, \pi) = A_5(p, G, \pi)$$

is the mathematical serialization result, whereas concrete execution is connected only by the one-way generated-profile relation

$$\text{Exec}_{\text{Neo4j}} \sqsubseteq_{\text{Gen5}} \text{TextSem}_5$$

of Definition 4.15. Under that external premise, Corollary 4.16 transfers exact violation preservation to successful Neo4j runs. The live experiments in Sect. 5 support selected instances of this implication but do not prove it universally.

7 Limitations, Future Work, and Conclusion

7.1 Limitations

The theorem covers exactly the finite OCL_{val} grammar and admission judgment defined in Sects. 2 and 3: scalar attributes, binary associations, flat Set/Bag values, the displayed operators and five iterators, and class invariants over one finite state. UML end `isUnique` determines Set versus Bag navigation. Ordered-end navigation is observed only after order erasure and only in permutation-invariant contexts. Mathematical Integer and Real values are not identified with 64-bit integers or floating point, so realization requires explicit range, intermediate-operation, cardinality, and exact-leaf certificates.

The representation theorem assumes $Adequate_{\Sigma}(M, G)$; compiler success cannot establish that an arbitrary graph represents M . Qualifiers, association classes, inheritance, and codecs are covered through a well-formed, total Σ and an adequate runtime pair. The target is the finite certified read-only Cypher 5 grammar defined in this report; every successful artifact must carry its lexical, carrier, numeric, and result-shape certificates.

Finally, the rule system is hand-proved on its complete stated domain, while Lean checks kernels and a representative vertical slice rather than the full production compiler. The pure theorem ends at $TextSem_5$. Concrete Neo4j preservation follows only through Definition 4.15 and Corollary 4.16; live runs are finite evidence, not a universal or version-independent executor proof.

7.2 Future Work

The immediate priorities are:

1. extend the Lean vertical slice to navigation, plans, iterators, inheritance, and certified numeric operators;
2. machine-check the complete formal Core/ Q constructor inductions;
3. machine-check the complete target serializer/parser and result decoder;
4. formulate and validate a reusable executor-conformance suite;
5. machine-check the order-erasure admission and provenance lemmas;
6. prove semantics-preserving target optimizations over the admitted grammar and evaluate them on industrial-scale models.

7.3 Conclusion

OCL2Cypher is specified as a four-boundary transformation from resolved and typed OCL through Core and Q to an abstract target and canonical Cypher package. Its correctness argument is not based on the appearance of the generated query. It rests on independently defined source semantics, observational representation adequacy, typed structural algorithms, target scope and carrier obligations, and stable-ID result projection.

For every formal run satisfying P1–P5, the report proves

$$Viol_O(M, inv) = Obs_Q(G, K, q) = Viol_C(G, s; \pi, \Delta).$$

Translation completeness is stated separately on the independently admitted and backend-certified domain. Hand proofs, Lean results, and empirical observations support different parts of this chain and are reported without conflation. Within $\mathcal{F}_{paper}$, the hand proof is constructor-complete: each syntax alternative has a typing rule, denotational clause, transformation equation, and discharged induction obligation. This does not mean one lemma per rule: homomorphic alternatives share proved generic lemmas, whereas semantically distinct alternatives have dedicated arguments. What remains partial is mechanization and refinement to a particular implementation or Neo4j release, not the mathematical constructor coverage of the published calculus. This separation makes both the established result and the remaining implementation/refinement work explicit.

If a concrete executor satisfies the one-way generated-profile refinement, Corollary 4.16 additionally yields $Viol_O = Viol_C^{Neo4j}$ for the corresponding successful run. Thus the report makes the pure mathematical endpoint and the remaining deployment obligation explicit rather than conflating them.

How to Read the Appendices

Appendix A fixes the complete formal syntax and semantics used by the theorems. Appendix B records constructor coverage, and Appendix C supplies reusable lemmas and rule-level proof cases. Appendix D then records assurance status and assumptions without enlarging the mathematical claim. Thus a soundness review should follow the main theorem into Appendices A–C; an assurance review can continue with Appendix D and the companion artefact metadata.

A Complete Grammars, Typing, and Semantics

This appendix completes the definitions summarized in Sect. 3. The tables are normative for $\mathcal{F}_{\text{paper}}$. A row in a broader backend inventory does not enlarge this profile unless it also has source syntax, typing, admission, translation, and preservation entries.

A.1 Semantic Operators

For Boolean values $\beta, \beta' \in D_B^\perp$:

$\neg$	T	F	$\perp_B$
	T	F	$\perp_B$
	F	T	$\perp_B$

$\wedge$	T	F	$\perp_B$
T	T	F	$\perp_B$
F	F	F	F
$\perp_B$	$\perp_B$	F	$\perp_B$

and $\vee$ is the dual table with true dominant. Implication is $\neg\beta \vee \beta'$. Typed equality is total on each admitted typed carrier: two same-typed bottoms are equal, exactly one bottom and one defined value are unequal, and two defined values are compared by the type-specific equality relation. Its result is always T or F, never $\perp_B$. Unrelated types are rejected statically rather than compared through an untyped carrier. Other strict primitive operations propagate a typed bottom according to their published bottom policy.

For a defined collection C and predicate $B : \text{Occ}(C) \rightarrow D_B^\perp$,

$$\begin{aligned}
 \text{select}(C, B) &= \text{the occurrences } v \text{ for which } B(v) = \text{T}, \\
 \text{reject}(C, B) &= \text{the occurrences } v \text{ for which } B(v) \in \{\text{F}, \perp_B\}, \\
 \text{project}_\kappa(C, F) &= \begin{cases} \text{typed support of the occurrence-wise image,} & \kappa = \text{Set}, \\ \text{the occurrence-wise image,} & \kappa = \text{Bag}, \end{cases} \\
 \text{collect}(C, F) &= \text{the occurrence-wise image of } C \text{ under } F, \\
 \text{distinct}(C) &= \text{the Set support of } C.
 \end{aligned}$$

If C is whole-bottom, each strict operation returns the appropriately typed whole-bottom. An admitted scalar-body collect always returns a Bag and retains one output occurrence per source occurrence, even when its source is a Set.

The finite operator catalogue of $\mathcal{F}_{\text{paper}}$ is

$$\begin{aligned}
 \mathcal{U} &= \{\text{not, neg, isEmpty, notEmpty, size, sum, typeTest}(K), \text{typeCast}(K)\}, \\
 \mathcal{B} &= \{\text{and, or, implies, eq, neq, <, \leq, >, \geq, +, -, \times, \text{div}\}.
 \end{aligned}$$

Each catalogue entry contains one or more explicitly typed signatures selected uniquely after source elaboration. Boolean entries use the tables above; equality uses total typed equality; comparison and arithmetic are strict and use mathematical Integer/Real operations. Division by zero and a failed admitted cast yield the declared typed bottom. A target numeric certificate later restricts these mathematical results to the selected physical carrier; it does not change their source meaning. No operation outside $\mathcal{U} \cup \mathcal{B}$ belongs to the published fragment.

A.2 Source-to-Core Constructor Semantics

The compact source notation and Core notation use identical declaration identities. The following table gives the admitted source meaning and its Core image. $\mathcal{E}_L(t, \rho)$ abbreviates evaluation in language $L \in \{O, \text{Core}\}$.

For completeness, let $\mathcal{O}_M(a, \rho) = \llbracket a \rrbracket_O^{M, \rho}$. The source interpretation is the unique structural function satisfying

$$\begin{aligned}
\mathcal{O}(c_\tau, \rho) &= c, \\
\mathcal{O}(x, \rho) &= \rho(x), \\
\mathcal{O}(\text{LetO}(x, a, b), \rho) &= \mathcal{O}(b, \rho[x \mapsto \mathcal{O}(a, \rho)]), \\
\mathcal{O}(\text{IfO}(c, t, f), \rho) &= \begin{cases} \mathcal{O}(t, \rho), & \mathcal{O}(c, \rho) = \top, \\ \mathcal{O}(f, \rho), & \mathcal{O}(c, \rho) = \text{F}, \\ \perp_\tau, & \mathcal{O}(c, \rho) = \perp_B, \end{cases} \\
\mathcal{O}(\text{AttrO}(r, a), \rho) &= \text{slot}_M(\mathcal{O}(r, \rho), a), \\
\mathcal{O}(\text{NavOneO}(r, k, \bar{u}), \rho) &= \text{Nav}_M(\mathcal{O}(r, \rho), k, \overline{\mathcal{O}(u_i, \rho)}), \\
\mathcal{O}(\text{NavManyO}_\kappa(r, k, \bar{u}), \rho) &= \text{Nav}_M(\mathcal{O}(r, \rho), k, \overline{\mathcal{O}(u_i, \rho)}), \\
\mathcal{O}(\text{UnO}_{op}(a), \rho) &= \text{prim}_{op}(\mathcal{O}(a, \rho)), \\
\mathcal{O}(\text{BinO}_{op}(a, b), \rho) &= \text{prim}_{op}(\mathcal{O}(a, \rho), \mathcal{O}(b, \rho)), \\
\mathcal{O}(\text{LitO}_\kappa(\bar{a}), \rho) &= \kappa \langle \mathcal{O}(a_i, \rho) \rangle_i, \\
\mathcal{O}(\text{SelectO}(s, x, b), \rho) &= \text{select}(\mathcal{O}(s, \rho), v \mapsto \mathcal{O}(b, \rho[x \mapsto v])), \\
\mathcal{O}(\text{RejectO}(s, x, b), \rho) &= \text{reject}(\mathcal{O}(s, \rho), v \mapsto \mathcal{O}(b, \rho[x \mapsto v])), \\
\mathcal{O}(\text{CollectO}(s, x, b), \rho) &= \text{collect}(\mathcal{O}(s, \rho), v \mapsto \mathcal{O}(b, \rho[x \mapsto v])), \\
\mathcal{O}(\text{ForAllO}(s, x, b), \rho) &= \text{forAll3}(\mathcal{O}(s, \rho), v \mapsto \mathcal{O}(b, \rho[x \mapsto v])), \\
\mathcal{O}(\text{ExistsO}(s, x, b), \rho) &= \text{exists3}(\mathcal{O}(s, \rho), v \mapsto \mathcal{O}(b, \rho[x \mapsto v])), \\
\mathcal{O}(\text{AllInstancesO}(K), \rho) &= \text{Inst}_M(K), \\
\mathcal{O}(\text{TypeTestO}(a, K), \rho) &= \text{typeTest}_K(\mathcal{O}(a, \rho)), \\
\mathcal{O}(\text{TypeCastO}(a, K), \rho) &= \text{typeCast}_K(\mathcal{O}(a, \rho)).
\end{aligned}$$

Every strict receiver equation first propagates its appropriately typed bottom. The displayed equations and the operator catalogue therefore cover every constructor of the source grammar, including all dynamic bottom cases.

Table 4: Resolved OCL and Core semantic clauses.

Family	Denotation	Conditions and Core form
Literal c_τ	$\mathcal{E}_L(c_\tau, \rho) = c_\tau$	Exact admitted value; Core $\text{Const}_\tau(c)$.
Variable x	$\mathcal{E}_L(x, \rho) = \rho(x)$	Declaration identity $x \in \text{dom}(\rho)$.
Typed bottom	$\mathcal{E}_{\text{Core}}(\perp_\tau, \rho) = \perp_\tau$	No admitted surface bottom literal; bottom may arise dynamically.
Let	evaluate initializer to v , then body under $\rho[x \mapsto v]$	Same identity x ; Core Let.
If	true selects then; false selects else; bottom gives result bottom	Branch types equal; unselected branch is not evaluated.
Attribute	$\text{slot}_M(\mathcal{E}_L(r, \rho), \mathbf{a})$	Resolved attribute and receiver; Core Attr.
Upper-one navigation	$\text{Nav}_M(\mathcal{E}_L(r, \rho), \mathbf{r}, \overline{\mathcal{E}_L(u_i, \rho)})$, with absence as typed bottom	Upper bound at most one; upper zero is necessarily absent; ordered qualifiers match their declarations.
Many navigation	finite Set support or Bag occurrences returned by Nav_M	Kind follows resolved uniqueness; ordered qualifiers are evaluated once; source end order is erased only under the admitted permutation-invariance condition.
Unary operation	admitted typed operation applied strictly to the child value	Signature selected by resolved operation identity.
Binary operation	admitted typed operation applied with its published bottom policy	Includes Boolean tables and certified numeric operations.
Collection literal	finite collection of evaluated item occurrences	Homogeneous element type and resolved Set/Bag kind.
Select	$\text{select}(C, v \mapsto \mathcal{E}_L(b, \rho[x \mapsto v]))$	Body Boolean; same declaration identity x .
Reject	$\text{reject}(C, v \mapsto \mathcal{E}_L(b, \rho[x \mapsto v]))$	Body Boolean; keeps false and bottom occurrences.
Collect	occurrence-wise body image as a Bag	Scalar body in $\mathcal{F}_{\text{paper}}$; one result occurrence per source occurrence.

Family	Denotation	Conditions and Core form
ForAll	false, then bottom, then true priority; true on empty defined input	Whole-bottom source gives Boolean bottom.
Exists	true, then bottom, then false priority; false on empty defined input	Whole-bottom source gives Boolean bottom.
AllInstances	$Inst_M(K)$	Includes admitted subclass instances.
Type test/cast	uses $\preceq_{MM}$; failed admitted cast gives typed bottom	Present only in a profile whose admission table contains the operation.

A.3 Core Plan Equations

For readability, write $\mathcal{E}_M(e, \rho) = \llbracket e \rrbracket_{\text{Core}}^{M, \rho}$. The complete Core expression equations are

$$\begin{aligned}
\mathcal{E}_M(c_\tau, \rho) &= c, \\
\mathcal{E}_M(\perp_\tau, \rho) &= \perp_\tau, \\
\mathcal{E}_M(x, \rho) &= \rho(x), \\
\mathcal{E}_M(\text{Let}(x, e_1, e_2), \rho) &= \mathcal{E}_M(e_2, \rho[x \mapsto \mathcal{E}_M(e_1, \rho)]), \\
\mathcal{E}_M(\text{If}(c, t, f), \rho) &= \begin{cases} \mathcal{E}_M(t, \rho), & \mathcal{E}_M(c, \rho) = \text{T}, \\ \mathcal{E}_M(f, \rho), & \mathcal{E}_M(c, \rho) = \text{F}, \\ \perp_\tau, & \mathcal{E}_M(c, \rho) = \perp_B, \end{cases} \\
\mathcal{E}_M(\text{Coerce}_h(e), \rho) &= \text{coerce}_h(\mathcal{E}_M(e, \rho)), \\
\mathcal{E}_M(\text{Attr}(r, a), \rho) &= \text{slot}_M(\mathcal{E}_M(r, \rho), a), \\
\mathcal{E}_M(\text{NavOne}(r, k, \bar{u}), \rho) &= \text{Nav}_M(\mathcal{E}_M(r, \rho), k, \overline{\mathcal{E}_M(u_i, \rho)}), \\
\mathcal{E}_M(\text{TypeTest}(e, K, b), \rho) &= \text{typeTest}_{K, b}(\mathcal{E}_M(e, \rho)), \\
\mathcal{E}_M(\text{TypeCast}(e, K), \rho) &= \text{typeCast}_K(\mathcal{E}_M(e, \rho)), \\
\mathcal{E}_M(\text{Un}_{op}(e), \rho) &= \text{prim}_{op}(\mathcal{E}_M(e, \rho)), \\
\mathcal{E}_M(\text{Bin}_{op}(e_1, e_2), \rho) &= \text{prim}_{op}(\mathcal{E}_M(e_1, \rho), \mathcal{E}_M(e_2, \rho)), \\
\mathcal{E}_M(\text{Lit}_\kappa(\bar{e}), \rho) &= \kappa\langle \mathcal{E}_M(e_i, \rho) \rangle_i, \\
\mathcal{E}_M(\text{Materialize}(P), \rho) &= \mathcal{P}_M(P, \rho), \\
\mathcal{E}_M(\text{ForAll}(P, x, b), \rho) &= \text{forAll3}(\mathcal{P}_M(P, \rho), v \mapsto \mathcal{E}_M(b, \rho[x \mapsto v])), \\
\mathcal{E}_M(\text{Exists}(P, x, b), \rho) &= \text{exists3}(\mathcal{P}_M(P, \rho), v \mapsto \mathcal{E}_M(b, \rho[x \mapsto v])).
\end{aligned}$$

Together with the source equations, these definitions make frontend preservation an equality between independently defined functions.

For readability, write $\mathcal{P}_M(P, \rho) = \llbracket P \rrbracket_{\text{Core}}^{M, \rho}$. The mutually recursive equations are:

$$\begin{aligned}
\mathcal{P}_M(\text{From}(e), \rho) &= \mathcal{E}_{\text{Core}}(e, \rho), \\
\mathcal{P}_M(\text{Scan}_K, \rho) &= \text{Inst}_M(K), \\
\mathcal{P}_M(\text{NavMany}_\kappa(e, \mathbf{r}, \bar{u}), \rho) &= \text{Nav}_M(\mathcal{E}_{\text{Core}}(e, \rho), \mathbf{r}, \overline{\mathcal{E}_{\text{Core}}(u_i, \rho)}), \\
\mathcal{P}_M(\text{Select}(P, x, b), \rho) &= \text{select}(\mathcal{P}_M(P, \rho), v \mapsto \mathcal{E}_{\text{Core}}(b, \rho[x \mapsto v])), \\
\mathcal{P}_M(\text{Reject}(P, x, b), \rho) &= \text{reject}(\mathcal{P}_M(P, \rho), v \mapsto \mathcal{E}_{\text{Core}}(b, \rho[x \mapsto v])), \\
\mathcal{P}_M(\text{Collect}(P, x, b), \rho) &= \text{collect}(\mathcal{P}_M(P, \rho), v \mapsto \mathcal{E}_{\text{Core}}(b, \rho[x \mapsto v])), \\
\mathcal{P}_M(\text{LetP}(x, e, P), \rho) &= \mathcal{P}_M(P, \rho[x \mapsto \mathcal{E}_{\text{Core}}(e, \rho)]).
\end{aligned}$$

Every equation propagates whole-collection bottom before inspecting occurrences.

A.4 Q Equations

Q has mutually recursive expression and plan semantics. Write $\mathcal{Q}_G(q, \eta) = \llbracket q \rrbracket_Q^{G, \eta}$ and $\mathcal{R}_G(P, \eta) = \llbracket P \rrbracket_Q^{G, \eta}$. The complete expression equations are

$$\begin{aligned}
\mathcal{Q}(c_\tau, \eta) &= c, \\
\mathcal{Q}(p_\tau, \eta) &= \eta(p), \\
\mathcal{Q}(\text{QBottom}_\tau, \eta) &= \perp_\tau, \\
\mathcal{Q}(x, \eta) &= \eta(x), \\
\mathcal{Q}(\text{QCoerce}_h(q), \eta) &= \text{coerce}_h(\mathcal{Q}(q, \eta)), \\
\mathcal{Q}(\text{QLet}(x, q_1, q_2), \eta) &= \mathcal{Q}(q_2, \eta[x \mapsto \mathcal{Q}(q_1, \eta)]), \\
\mathcal{Q}(\text{QIf}(c, t, f), \eta) &= \begin{cases} \mathcal{Q}(t, \eta), & \mathcal{Q}(c, \eta) = \top, \\ \mathcal{Q}(f, \eta), & \mathcal{Q}(c, \eta) = \text{F}, \\ \perp_\tau, & \mathcal{Q}(c, \eta) = \perp_B, \end{cases} \\
\llbracket \text{ReadAttribute}(q, \mathbf{a}) \rrbracket_Q^{G, \eta} &= \text{slot}_G(\llbracket q \rrbracket_Q^{G, \eta}, \mathbf{a}), \\
\llbracket \text{NavigateOne}(q, \mathbf{r}, \bar{u}) \rrbracket_Q^{G, \eta} &= \text{Nav}_G^\Sigma(\llbracket q \rrbracket_Q^{G, \eta}, \mathbf{r}, \overline{\llbracket u_i \rrbracket_Q^{G, \eta}}), \\
\llbracket \text{NavigateAssociationClassOne}(q, A, \mathbf{r}, \bar{u}) \rrbracket_Q^{G, \eta} &= \text{ACNav}_{G,1}^\Sigma(\llbracket q \rrbracket_Q^{G, \eta}, A, \mathbf{r}, \overline{\llbracket u_i \rrbracket_Q^{G, \eta}}), \\
\mathcal{Q}(\text{QTypeTest}(q, K, b), \eta) &= \text{typeTest}_{K,b}^\Sigma(G, \mathcal{Q}(q, \eta)), \\
\mathcal{Q}(\text{QTypeCast}(q, K), \eta) &= \text{typeCast}_K^\Sigma(G, \mathcal{Q}(q, \eta)), \\
\mathcal{Q}(\text{QUn}_{op}(q), \eta) &= \text{prim}_{op}^Q(\mathcal{Q}(q, \eta)), \\
\mathcal{Q}(\text{QBin}_{op}(q_1, q_2), \eta) &= \text{prim}_{op}^Q(\mathcal{Q}(q_1, \eta), \mathcal{Q}(q_2, \eta)), \\
\mathcal{Q}(\text{QCollection}_\kappa(\bar{q}), \eta) &= \kappa(\mathcal{Q}(q_i, \eta))_i, \\
\mathcal{Q}(\text{MaterializeQ}(P), \eta) &= \mathcal{R}(P, \eta), \\
\mathcal{Q}(\text{ForAll3}(P, x, b), \eta) &= \text{forAll3}(\mathcal{R}(P, \eta), v \mapsto \mathcal{Q}(b, \eta[x \mapsto v])), \\
\mathcal{Q}(\text{Exists3}(P, x, b), \eta) &= \text{exists3}(\mathcal{R}(P, \eta), v \mapsto \mathcal{Q}(b, \eta[x \mapsto v])).
\end{aligned}$$

The complete plan equations are

$$\begin{aligned}
\mathcal{R}(\text{FromCollection}(q), \eta) &= \mathcal{Q}(q, \eta), \\
\llbracket \text{ScanClass}_K \rrbracket_Q^{G, \eta} &= \text{Extent}_G^\Sigma(K), \\
\llbracket \text{NavigateMany}_\kappa(q, \mathbf{r}, \bar{u}) \rrbracket_Q^{G, \eta} &= \text{Nav}_G^\Sigma(\llbracket q \rrbracket_Q^{G, \eta}, \mathbf{r}, \overline{\llbracket u_i \rrbracket_Q^{G, \eta}}), \\
\llbracket \text{NavigateAssociationClass}_\kappa(q, A, \mathbf{r}, \bar{u}) \rrbracket_Q^{G, \eta} &= \text{ACNav}_{G,*}^\Sigma(\llbracket q \rrbracket_Q^{G, \eta}, A, \mathbf{r}, \overline{\llbracket u_i \rrbracket_Q^{G, \eta}}), \\
\mathcal{R}(\text{Filter}_{\text{SELECT}}(P, x, b), \eta) &= \text{select}(\mathcal{R}(P, \eta), v \mapsto \mathcal{Q}(b, \eta[x \mapsto v])), \\
\mathcal{R}(\text{Filter}_{\text{REJECT}}(P, x, b), \eta) &= \text{reject}(\mathcal{R}(P, \eta), v \mapsto \mathcal{Q}(b, \eta[x \mapsto v])), \\
\mathcal{R}(\text{Project}_\kappa(P, x, b), \eta) &= \text{project}_\kappa(\mathcal{R}(P, \eta), v \mapsto \mathcal{Q}(b, \eta[x \mapsto v])), \\
\mathcal{R}(\text{CollectQ}(P, x, b), \eta) &= \text{collect}(\mathcal{R}(P, \eta), v \mapsto \mathcal{Q}(b, \eta[x \mapsto v])), \\
\mathcal{R}(\text{PlanLet}(x, q, P), \eta) &= \mathcal{R}(P, \eta[x \mapsto \mathcal{Q}(q, \eta)]), \\
\mathcal{R}(\text{DistinctQ}(P), \eta) &= \text{distinct}(\mathcal{R}(P, \eta)).
\end{aligned}$$

All equations propagate typed receiver or whole-collection bottom before inspection. The functions prim_{op}^Q are required by the operator-correspondence certificate to denote the same mathematical operation as prim_{op} . This definition fixes empty, element-bottom, and whole-bottom behaviour independently of the generated Cypher.

A.5 Typing Coverage

The following rows generate the least typing judgments for the source, Core, and Q grammars after replacing each family name by its language-specific constructor. Thus the table is a compact rule schema, not an informal checklist. Every grammar constructor occurs in exactly one row.

Table 5: Typing obligations by constructor family.

Family	Input obligation	Result type
Literal/bottom	payload belongs to D_τ or is $\perp_\tau$	τ
Parameter	declared public parameter has expected type τ	τ
Variable	$\Gamma(x) = \tau$	τ
Coerce	unique admitted witness $h : \tau \rightsquigarrow \sigma$; child τ	σ
Let	initializer τ ; body typed under $x : \tau$	body type
If	Boolean guard; equal branch types	branch type
Attribute	receiver conforms to owner; declaration is scalar	declared scalar attribute type
NavOne/association-class one	receiver conforms; upper ≤ 1 ; qualifier tuple matches declarations; order admission holds	target object or scalar type
NavMany/association-class many	receiver conforms; upper > 1 or $*$; qualifier tuple matches declarations; order admission holds	$\kappa(\tau)$
Type test/cast	source and target classes are admitted and related as required	Boolean or $\text{Class}(K)$
Unary/binary	unique admitted operation signature	declared result
Collection literal	homogeneous items	$\kappa(\tau)$
Materialize	plan (κ, τ)	$\kappa(\tau)$
Select/Reject	source $\kappa(\tau)$; Boolean body under $x : \tau$	$\kappa(\tau)$
Project	source $\kappa(\tau)$; scalar body σ ; result kind equals source kind κ	$\kappa(\sigma)$
Collect	source $\kappa(\tau)$; scalar body σ	$\text{Bag}(\sigma)$
ForAll/Exists	collection source; Boolean body	Boolean
From forms	expression $\kappa(\tau)$	plan (κ, τ)
Scan	declared class K	$\text{Set}(\text{Class}(K))$
LetP/PlanLet	initializer τ ; body plan typed under $x : \tau$	body plan type

A.6 Target Semantics

The target typing judgments are

$$\Sigma; \Lambda \vdash_T t : \tau, \quad \Sigma; \Lambda \vdash_T d : (\kappa, \tau),$$

where Λ maps target identities to types. They are the least judgments generated by the following complete constructor table.

Table 6: Complete target typing and scope rules.

Constructor	Premises	Conclusion
TConst(c)	c has certified carrier type τ	$t : \tau$
TParam(u)	$u : \tau$ in the parameter contract	$t : \tau$
TVar(y)	$\Lambda(y) = \tau$	$t : \tau$
TBottom(τ)	τ admitted	$t : \tau$
TCoerce $_h$ (t)	$t : \tau$; certified $h : \tau \rightsquigarrow \sigma$	expression of type σ
TLet(y, t_1, t_2)	$t_1 : \tau_1; t_2 : \tau_2$ under $\Lambda, y : \tau_1; y$ fresh	$t : \tau_2$
TCase(c, t, f)	$c : \text{Boolean}; t, f : \tau$	expression of type τ
TOp($op, \bar{t}$)	certified signature $\bar{\tau} \rightarrow \tau; t_i : \tau_i$	expression of type τ
TObserveAttr(t, ω_a)	$t : \text{Class}(K)$; descriptor result τ	expression of type τ
TObserveOne($t, \bar{u}, \omega_r$)	receiver/descriptor agree; qualifiers match in order; upper ≤ 1	declared target type
TTypeTest(t, ω_K, b)	class-typed receiver; exact/conformance descriptor for K	Boolean
TTypeCast(t, ω_K)	class-typed receiver; admitted related target K	$\text{Class}(K)$
TList $_{\kappa}(\bar{t})$	every item has type τ	$\kappa(\tau)$
TMaterialize(d)	$d : (\kappa, \tau)$	$\kappa(\tau)$
TForAll3/TExists3	$d : (\kappa, \tau)$; body Boolean under $y : \tau$	Boolean
TFrom(t)	$t : \kappa(\tau)$	plan (κ, τ)
TScan(ω_K)	certified extent descriptor for K	plan $(\text{Set}, \text{Class}(K))$

Constructor	Premises	Conclusion
$\text{TObserveMany}_\kappa(t, \bar{u}, \omega_r)$	receiver/descriptor agree; qualifiers match; kind certificate κ	plan (κ, τ)
$\text{TFilter}_k(d, y, b)$	$d : (\kappa, \tau); b : \text{Boolean under } y : \tau;$ $k \in \{\text{SELECT}, \text{REJECT}\}$	plan (κ, τ)
$\text{TProject}_\kappa(d, y, t)$	$d : (\kappa, \tau); t : \sigma$ under $y : \tau$; typed equality if $\kappa = \text{Set}$	plan (κ, σ)
$\text{TCollect}(d, y, t)$	$d : (\kappa, \tau); t : \sigma$ under $y : \tau$	plan (Bag, σ)
$\text{TPlanLet}(y, t, d)$	$t : \tau; d : (\kappa, \sigma)$ under $y : \tau; y$ fresh	plan (κ, σ)
$\text{TDistinct}(d)$	$d : (\kappa, \tau)$; total target equality for τ	plan (Set, τ)
TQuery	body Boolean under $y : \text{Class}(K)$ and otherwise closed; certified extent/ID descriptors; one result alias	well-formed query

Let θ be a finite target-variable environment and let $\mathcal{T}_G^\pi(t, \theta)$ and $\mathcal{D}_G^\pi(d, \theta)$ denote target expression and target plan evaluation. Certified descriptors have total logical observations

$$\text{obs}_a(G, v, \omega_a), \quad \text{obs}_1(G, v, \bar{u}, \omega_r), \quad \text{obs}_*(G, v, \bar{u}, \omega_r), \quad \text{obs}_{type}(G, v, \omega_K, b), \quad \text{extent}(G, \omega_K),$$

on their admitted domain. The complete expression equations are

$$\begin{aligned}
\mathcal{T}(\text{TConst}(c), \theta) &= \text{decode}(c), \\
\mathcal{T}(\text{TParam}(u), \theta) &= \pi(u), \\
\mathcal{T}(\text{TVar}(y), \theta) &= \theta(y), \\
\mathcal{T}(\text{TBottom}(\tau), \theta) &= \perp_\tau, \\
\mathcal{T}(\text{TCoerce}_h(t), \theta) &= \text{coerce}_h(\mathcal{T}(t, \theta)), \\
\mathcal{T}(\text{TLet}(y, t_1, t_2), \theta) &= \mathcal{T}(t_2, \theta[y \mapsto \mathcal{T}(t_1, \theta)]), \\
\mathcal{T}(\text{TCase}(c, t, f), \theta) &= \begin{cases} \mathcal{T}(t, \theta), & \mathcal{T}(c, \theta) = \text{T}, \\ \mathcal{T}(f, \theta), & \mathcal{T}(c, \theta) = \text{F}, \\ \perp_\tau, & \mathcal{T}(c, \theta) = \perp_B, \end{cases} \\
\mathcal{T}(\text{TOp}(op, \bar{t}), \theta) &= \text{prim}_{op}^T(\overline{\mathcal{T}(t_i, \theta)}), \\
\mathcal{T}(\text{TObserveAttr}(t, \omega_a), \theta) &= \text{obs}_a(G, \mathcal{T}(t, \theta), \omega_a), \\
\mathcal{T}(\text{TObserveOne}(t, \bar{u}, \omega_r), \theta) &= \text{obs}_1(G, \mathcal{T}(t, \theta), \overline{\mathcal{T}(u_i, \theta)}, \omega_r), \\
\mathcal{T}(\text{TTypeTest}(t, \omega_K, b), \theta) &= \text{obs}_{type}(G, \mathcal{T}(t, \theta), \omega_K, b), \\
\mathcal{T}(\text{TTypeCast}(t, \omega_K), \theta) &= \begin{cases} \mathcal{T}(t, \theta), & \text{obs}_{type}(G, \mathcal{T}(t, \theta), \omega_K, \text{conforms}) = \text{T}, \\ \perp_{\text{Class}(K)}, & \text{otherwise}, \end{cases} \\
\mathcal{T}(\text{TList}_\kappa(\bar{t}), \theta) &= \kappa\langle \mathcal{T}(t_i, \theta) \rangle_i, \\
\mathcal{T}(\text{TMaterialize}(d), \theta) &= \mathcal{D}(d, \theta), \\
\mathcal{T}(\text{TForAll3}(d, y, b), \theta) &= \text{forAll3}(\mathcal{D}(d, \theta), v \mapsto \mathcal{T}(b, \theta[y \mapsto v])), \\
\mathcal{T}(\text{TExists3}(d, y, b), \theta) &= \text{exists3}(\mathcal{D}(d, \theta), v \mapsto \mathcal{T}(b, \theta[y \mapsto v])).
\end{aligned}$$

Here and below G, π are fixed and omitted from the right-hand notation. The complete plan equations are

$$\begin{aligned}
\mathcal{D}(\text{TFrom}(t), \theta) &= \mathcal{T}(t, \theta), \\
\mathcal{D}(\text{TScan}(\omega_K), \theta) &= \text{extent}(G, \omega_K), \\
\mathcal{D}(\text{TObserveMany}_\kappa(t, \bar{u}, \omega_r), \theta) &= \text{obs}_*(G, \mathcal{T}(t, \theta), \overline{\mathcal{T}(u_i, \theta)}, \omega_r), \\
\mathcal{D}(\text{TFilter}_{\text{SELECT}}(d, y, b), \theta) &= \text{select}(\mathcal{D}(d, \theta), v \mapsto \mathcal{T}(b, \theta[y \mapsto v])), \\
\mathcal{D}(\text{TFilter}_{\text{REJECT}}(d, y, b), \theta) &= \text{reject}(\mathcal{D}(d, \theta), v \mapsto \mathcal{T}(b, \theta[y \mapsto v])), \\
\mathcal{D}(\text{TProject}_\kappa(d, y, t), \theta) &= \text{project}_\kappa(\mathcal{D}(d, \theta), v \mapsto \mathcal{T}(t, \theta[y \mapsto v])), \\
\mathcal{D}(\text{TCollect}(d, y, t), \theta) &= \text{collect}(\mathcal{D}(d, \theta), v \mapsto \mathcal{T}(t, \theta[y \mapsto v])), \\
\mathcal{D}(\text{TPlanLet}(y, t, d), \theta) &= \mathcal{D}(d, \theta[y \mapsto \mathcal{T}(t, \theta)]), \\
\mathcal{D}(\text{TDistinct}(d), \theta) &= \text{distinct}(\mathcal{D}(d, \theta)).
\end{aligned}$$

Every strict equation first propagates a typed or whole-collection bottom. Malformed carrier payloads, missing parameters, and descriptors outside their certified domain yield Failure rather than a bottom or an empty collection.

For

$$p = \text{TQuery}(\text{violation}, K, y, t, \omega_K, \omega_{id}, a, \Delta_\pi, \pi_{gen}),$$

put $E = \text{extent}(G, \omega_K)$ and $B_T^p(g) = \mathcal{T}_G^\pi(t, [y \mapsto g])$. Its logical return predicate is

$$\text{Ret}^p(j) \iff \exists g \in E : \text{obs}_{id}(G, g, \omega_{id}) = j \wedge B_T^p(g) \neq \top.$$

$A_5(p, G, \pi)$ evaluates this finite comprehension, applies typed distinct to the IDs, and returns the resulting one-column row bag. Thus its decoder gives exactly $\{j \mid \text{Ret}^p(j)\}$ whenever evaluation succeeds. This equation is the formal target-level origin of R2 and R3; neither is inferred from testing.

Structural induction on the typing derivation gives target progress and result shape: a closed well-formed target either returns the value/row shape assigned by its judgment or an explicit Failure. It never turns a failure into an empty row bag or typed bottom.

B Complete Algorithms and Constructor Coverage

This appendix audits the formal, syntax-directed functions in Sect. 4. Each row denotes a family of transformation-rule instances: it fixes a source constructor, target form, side conditions, and proof obligation. Proposition 4.1 shows that evaluating the displayed mathematical cases and deriving the corresponding rule judgment are equivalent. The tables do not inventory Java branches or methods. There is no catch-all success rule; a constructor outside the tables has no derivation in the published transformation domain.

B.1 Independent Inductive Admission Domains

This subsection gives the definitions on which the two completeness claims quantify. They are defined without mentioning execution of an implementation or success of the transformation functions.

Write $\text{sig}(op, \bar{\tau}; \tau)$ when op has the indicated unique signature in the finite operator catalogue of Appendix A, and write $\text{obs}_\Sigma(k) : \tau \rightsquigarrow \tau'$ when Σ has one type-correct observer for declaration key k . The least source-admission predicate is defined recursively by the following complete equations; every equation also requires the corresponding source typing judgment.

$$\begin{aligned} \text{Adm}_O(c_\tau) &\iff c \in D_\tau, \\ \text{Adm}_O(x) &\iff x \in \text{dom}(\Gamma), \\ \text{Adm}_O(\text{LetO}(x, a, b)) &\iff \text{Adm}_O(a) \wedge \text{Adm}_O(b), \\ \text{Adm}_O(\text{IfO}(c, t, f)) &\iff \bigwedge_{u \in \{c, t, f\}} \text{Adm}_O(u), \\ \text{Adm}_O(\text{AttrO}(r, a)) &\iff \text{Adm}_O(r) \wedge \text{obs}_\Sigma(a), \\ \text{Adm}_O(\text{NavOneO}(r, k, \bar{u})) &\iff \text{Adm}_O(r) \wedge \bigwedge_i \text{Adm}_O(u_i) \wedge \text{obs}_\Sigma(k) \wedge \text{upper}(k) \leq 1 \wedge \text{orderAdmissible}(k), \\ \text{Adm}_O(\text{NavManyO}_\kappa(r, k, \bar{u})) &\iff \text{Adm}_O(r) \wedge \bigwedge_i \text{Adm}_O(u_i) \wedge \text{obs}_\Sigma(k) \wedge \text{kind}(k) = \kappa \wedge \text{orderAdmissible}(k), \\ \text{Adm}_O(\text{UnO}_{op}(a)) &\iff \text{Adm}_O(a) \wedge \text{sig}(op), \\ \text{Adm}_O(\text{BinO}_{op}(a, b)) &\iff \text{Adm}_O(a) \wedge \text{Adm}_O(b) \wedge \text{sig}(op), \\ \text{Adm}_O(\text{LitO}_\kappa(\bar{a})) &\iff \bigwedge_i \text{Adm}_O(a_i), \\ \text{Adm}_O(\text{SelectO}(s, x, b)) &\iff \text{Adm}_O(s) \wedge \text{Adm}_O(b), \\ \text{Adm}_O(\text{RejectO}(s, x, b)) &\iff \text{Adm}_O(s) \wedge \text{Adm}_O(b), \\ \text{Adm}_O(\text{CollectO}(s, x, b)) &\iff \text{Adm}_O(s) \wedge \text{Adm}_O(b) \wedge \text{scalar}(b), \\ \text{Adm}_O(\text{ForAllO}(s, x, b)) &\iff \text{Adm}_O(s) \wedge \text{Adm}_O(b), \\ \text{Adm}_O(\text{ExistsO}(s, x, b)) &\iff \text{Adm}_O(s) \wedge \text{Adm}_O(b), \\ \text{Adm}_O(\text{AllInstancesO}(K)) &\iff K \in MM \wedge \text{obs}_\Sigma(K), \\ \text{Adm}_O(\text{TypeTestO}(a, K)) &\iff \text{Adm}_O(a) \wedge K \in MM, \\ \text{Adm}_O(\text{TypeCastO}(a, K)) &\iff \text{Adm}_O(a) \wedge K \in MM \\ &\quad \wedge \text{admissibleCast}(a, K). \end{aligned}$$

Binder-body occurrences in these equations are checked in the typing context extended by the displayed binder. Thus the apparently compact conjunct $\text{Adm}_O(b)$ includes scope and typing under that extension. The orderAdmissible premise is discharged immediately for an unordered end. For an ordered end it requires the enclosing resolved expression to use the navigation only through permutation-invariant operations and iterators; the provenance check also prevents collection equality from receiving an ordered-navigation operand. The logical completeness domain is now the independent set

$$\text{Inv}_{val}(MM, \Sigma) = \{(K, a) \mid MM; \text{self} : K \vdash_O a : \text{Boolean} \wedge \text{WF}_{resolved}(MM, a) \wedge \text{Adm}_O(a)\}.$$

For backend admission, let $\text{car}_\Omega(\tau)$ mean that the selected target carrier and codec represent every leaf needed by the particular query, $\text{prim}_\Omega(op)$ mean that the operator has a certified target realization on the inferred operand ranges, and $\text{lex}_\Omega(k)$ mean that the descriptor and generated names required by key k have canonical Cypher productions. The mutually recursive least predicates C5E and C5P are:

$$\begin{aligned} \text{C5E}(c_\tau) &\iff \text{car}_\Omega(\tau), \\ \text{C5E}(p_\tau) &\iff p : \tau \text{ occurs exactly once in the public contract}, \\ \text{C5E}(\text{QBottom}_\tau) &\iff \text{car}_\Omega(\tau), \\ \text{C5E}(x) &\iff x \in \text{dom}(\Gamma), \\ \text{C5E}(\text{QCoerce}_h(q)) &\iff \text{C5E}(q) \wedge \text{prim}_\Omega(h), \\ \text{C5E}(\text{QLet}(x, a, b)) &\iff \text{C5E}(a) \wedge \text{C5E}_{\Gamma, x}(b), \\ \text{C5E}(\text{QIf}(c, t, f)) &\iff \text{C5E}(c) \wedge \text{C5E}(t) \wedge \text{C5E}(f), \\ \text{C5E}(\text{ReadAttribute}(q, a)) &\iff \text{C5E}(q) \wedge \text{obs}_\Sigma(a) \wedge \text{lex}_\Omega(a), \\ \text{C5E}(\text{NavigateOne}(q, r, \bar{u})) &\iff \text{C5E}(q) \wedge \bigwedge_i \text{C5E}(u_i) \wedge \text{obs}_\Sigma(r) \wedge \text{lex}_\Omega(r), \\ \text{C5E}(\text{NavigateAssociationClassOne}(q, A, r, \bar{u})) &\iff \text{C5E}(q) \wedge \bigwedge_i \text{C5E}(u_i) \wedge \text{obs}_\Sigma(A, r) \wedge \text{lex}_\Omega(A, r), \\ \text{C5E}(\text{QTypeTest}(q, K, b)) &\iff \text{C5E}(q) \wedge \text{obs}_\Sigma(K) \wedge \text{lex}_\Omega(K), \\ \text{C5E}(\text{QTypeCast}(q, K)) &\iff \text{C5E}(q) \wedge \text{obs}_\Sigma(K) \wedge \text{lex}_\Omega(K), \\ \text{C5E}(\text{QUnop}(q)) &\iff \text{C5E}(q) \wedge \text{prim}_\Omega(op), \\ \text{C5E}(\text{QBinop}(q_1, q_2)) &\iff \text{C5E}(q_1) \wedge \text{C5E}(q_2) \wedge \text{prim}_\Omega(op), \\ \text{C5E}(\text{QCollection}_\kappa(\bar{q})) &\iff \bigwedge_i \text{C5E}(q_i) \wedge \text{car}_\Omega(\kappa), \\ \text{C5E}(\text{MaterializeQ}(P)) &\iff \text{C5P}(P), \\ \text{C5E}(\text{ForAll3}(P, x, b)) &\iff \text{C5P}(P) \wedge \text{C5E}_{\Gamma, x}(b), \\ \text{C5E}(\text{Exists3}(P, x, b)) &\iff \text{C5P}(P) \wedge \text{C5E}_{\Gamma, x}(b), \\ \text{C5P}(\text{FromCollection}(q)) &\iff \text{C5E}(q), \\ \text{C5P}(\text{ScanClass}_K) &\iff \text{obs}_\Sigma(K) \wedge \text{lex}_\Omega(K), \\ \text{C5P}(\text{NavigateMany}_\kappa(q, r, \bar{u})) &\iff \text{C5E}(q) \wedge \bigwedge_i \text{C5E}(u_i) \wedge \text{obs}_\Sigma(r) \wedge \text{lex}_\Omega(r), \\ \text{C5P}(\text{NavigateAssociationClass}_\kappa(q, A, r, \bar{u})) &\iff \text{C5E}(q) \wedge \bigwedge_i \text{C5E}(u_i) \wedge \text{obs}_\Sigma(A, r) \wedge \text{lex}_\Omega(A, r), \\ \text{C5P}(\text{Filter}_\kappa(P, x, b)) &\iff \text{C5P}(P) \wedge \text{C5E}_{\Gamma, x}(b), \\ \text{C5P}(\text{Project}_\kappa(P, x, b)) &\iff \text{C5P}(P) \wedge \text{C5E}_{\Gamma, x}(b) \wedge \text{resultKind}(P) = \kappa \wedge (\kappa = \text{Set} \Rightarrow \text{totalEquality}(\text{type}(b))), \\ \text{C5P}(\text{CollectQ}(P, x, b)) &\iff \text{C5P}(P) \wedge \text{C5E}_{\Gamma, x}(b), \\ \text{C5P}(\text{PlanLet}(x, q, P)) &\iff \text{C5E}(q) \wedge \text{C5P}_{\Gamma, x}(P), \\ \text{C5P}(\text{DistinctQ}(P)) &\iff \text{C5P}(P) \wedge \text{totalEquality}(\text{elem}(P)). \end{aligned}$$

Here $k \in \{\text{SELECT}, \text{REJECT}\}$ and the result kind is the source-plan kind in both cases. The local bottom policy is semantic rather than lexical: SELECT retains exactly true occurrences, whereas REJECT retains false and bottom occurrences. Finally,

$$\text{C5Realisable}_\Gamma(q, \Sigma, \Omega) \iff \Sigma; \Gamma \vdash_Q q : \tau \wedge \text{C5E}_\Gamma(q) \wedge \text{WF}_{RepSpec}(MM, \Sigma).$$

Let nonTrue_Ω mean that the target profile has the certified production for “different from T”, and let $\text{oneldColumn}_\Omega(K)$ mean that the result alias, ID codec, and decoder contract for K form one typed ID column. Admission of a public violation root is the following separate, exact clause:

$$\begin{aligned} \text{C5Realisable}(\text{Violations}(K, q), \Sigma, \Omega) \\ \iff \text{C5Realisable}_{\{\text{self:Class}(K)\}}(q, \Sigma, \Omega) \wedge \text{obs}_\Sigma(K) \wedge \text{obs}_\Sigma(\text{id}(K)) \\ \wedge \text{lex}_\Omega(K) \wedge \text{lex}_\Omega(\text{id}(K)) \wedge \text{LexAdmissible}_5(\text{Violations}(K, q), \Sigma) \wedge \text{nonTrue}_\Omega \wedge \text{oneldColumn}_\Omega(K). \end{aligned}$$

Thus root admission is also structural and independent of whether a program implementing realization happens to terminate or succeed. The predicate contains no $\text{ObserverAdequate}_5$ field. R5O derives that theorem from WF_Q^Σ and $\text{WF}_{RepSpec}(MM, \Sigma)$ for the fixed Rule 06 templates.

B.2 Resolved OCL to Core

Table 7: Frontend branch coverage.

Source family	Core image	Side condition	Proof family
Boolean, Integer, Real, or String literal	typed constant	exact lexical value and admitted type	literal base
Variable	$\text{Var}(x)$	unique declaration identity in Γ	environment lookup
Let	Let	initializer conforms to declared type	environment extension
If	If	Boolean guard; common branch type	conditional congruence
Attribute property call	Attr	unique resolved attribute and receiver conformance	attribute observer
Upper-one property call	NavOne	unique resolved end; upper ≤ 1	to-one navigation
Many property call	NavMany $_{\kappa}$	unique resolved end and collection kind	occurrence navigation
Association-class call	NavOne or NavMany with a tagged association-class navigation key	participant and qualifier identities resolved	association-class observer
Unary operation	Un _{op}	unique admitted signature	unary congruence
Binary operation	Bin _{op}	unique admitted signature and coercion	binary congruence
Collection literal	Lit $_{\kappa}$	flat homogeneous admitted items	collection literal
Select/reject	Select or Reject	collection source, Boolean body, and USE local-bottom policy	filter lemma
Collect	Collect of Bag result kind	admitted scalar body	occurrence image
ForAll/Exists	corresponding Core iterator	Boolean body and declared binder	fold lemma
Collection query operation	canonical operation/plan form	operation is one of isEmpty, notEmpty, size, or sum; collection and numeric certificates as applicable	operation lemma
allInstances	Scan $_K$	resolved admitted class	extent lemma
type test/cast	canonical typed operation	resolved class and admitted cast	subtype lemma

The successful frontend domain is exactly the least predicate generated by the equations above. Absence of a derivation is a transformation failure and does not create CoreBottom. The UML association-end property isUnique remains part of the admitted declaration and selects Set rather than Bag multiplicity for many-valued navigation.

B.3 Core to Q

Table 8: Core-to- Q branch coverage.

Core constructor	Q constructor	Σ /typing obligation	Proof family
Constant, bottom, or variable	corresponding typed leaf	carrier type and binder identity	leaf correspondence
Coerce	QCoerce	unique admitted coercion witness	coercion correspondence
Let	QLet	same binder identity and result type	environment extension
If	QIf	common branch type	conditional congruence
Attr	ReadAttribute	unique slot observer and codec	slot adequacy
NavOne	NavigateOne or NavigateAssociationClassOne	end/association observer, qualifiers, direction, upper-one policy	to-one adequacy
Type test/cast	QTypeTest/QTypeCast	exact/conformance observer and related target class	type-observer adequacy
Un/Bin	QUn/QBin	admitted typed operator	operator correspondence
Lit	QCollection	element carrier and kind	collection literal
Materialize	MaterializeQ	plan/result type compatibility	expression-plan bridge
ForAll/Exists	ForAll3/Exists3	Boolean body and fold policy	three-valued fold
From	FromCollection	collection expression	bridge lemma
Scan	ScanClass	inheritance-aware extent observer	extent adequacy
NavMany	NavigateMany or NavigateAssociationClass	end/association, qualifiers, direction, kind, duplicates	occurrence adequacy
Select/Reject	Filter _{SELECT/REJECT}	Boolean predicate and explicit filter kind	filter lemma
Collect	CollectQ	scalar body; result kind is Bag	occurrence image
LetP	PlanLet	same binder identity	plan environment extension
Violation root	Violations	context extent and stable-ID observer	projection lemma

B.4 Q to Target

Derived Q forms are first reduced by Norm $_Q$. Normalization is structural, terminating, idempotent, and denotation-preserving. The target branch table applies to the normalized image.

Table 9: Q -to-target realization coverage.

Q family	Target family	Certificate/side condition	Proof family
Constant	tagged constant or generated parameter	codec and finite carrier	codec left inverse
Public parameter	typed target parameter	exact public contract entry and domain separation	parameter lookup
Bottom	tagged bottom	exact logical type	bottom separation
Variable	target variable	binder in χ	scope lookup
QCoerce	guarded target coercion	certified carrier conversion	coercion refinement
QLet	scope-safe TLet/bind-once form	least fresh target identity	alpha/scope lemma
QIf	TCase	Boolean3 guard and equal branch carrier	case congruence
ReadAttribute	typed slot pattern and decode	unique observer and codec	observer refinement
NavigateOne	graph pattern plus upper-one check	direction/end/ambiguity policy	observer refinement
Assoc.-class one	association-object pattern plus upper-one check	participant role, occurrence key, qualifiers	association-class refinement
QTypeTest/QTypeCast	fixed type-observer case expression	direct/conformance observer and lazy bottom guard	type-observer refinement
Typed primitive	target operator/case expansion	local numeric or scalar certificate	primitive refinement
QCollection	tagged occurrence list	element carrier and Set/Bag policy	carrier induction
MaterializeQ	list/row materialization	plan descriptor shape	bridge refinement
ForAll3/Exists3	ordered TCase fold	Boolean3 carrier and finite occurrences	fold refinement
ScanClass	class/type graph pattern	extent observer	scan refinement
NavigateMany	row-producing pattern	occurrence observer and duplicate policy	navigation refinement
NavigateAssociationClass	association-object row pattern	participant role, occurrence identity, duplicate policy	association-class refinement
Filter	policy-tagged filtered descriptor	predicate carrier and SELECT/REJECT kind	filter refinement
Project	kind-preserving projected descriptor	scalar output carrier; typed support for Set	projection refinement
CollectQ	projected Bag descriptor	scalar output carrier and one row per source occurrence	collect refinement
PlanLet	correlated scope-safe descriptor	fresh identity	scope refinement
DistinctQ	distinct typed projection	total structural equality	support refinement
Violation root	read-only query root	context, non-true, ID, result contract	R1–R2

Realization rejects missing observers/codecs, unsafe names, unavailable numeric/cardinality certificates, unsupported collection shape, target constructors outside the read-only profile, and any term whose child realization fails.

B.5 Target Serialization

Table 10: Serialization production coverage.

Target family	Canonical text form	Required invariant	Proof family
Literal, parameter, or variable	canonical lexical token	injective codec and valid identity	lexical inverse
Typed bottom/null payload	canonical map/null form	type tag and shape	carrier inverse
Unary/binary operator	delimited precedence form	admitted typed operator	structural inverse
Let/bind-once	scoped list-comprehension/head form	fresh alias and one occurrence of value	alpha inverse
Case	canonical CASE form	ordered alternatives	structural inverse
List/comprehension	bracketed canonical form	scoped iterator alias	recursive inverse
Graph observer	canonical pattern	admitted descriptor	pattern inverse
Filter, project, or distinct	admitted clause sequence	row-shape invariant	clause inverse
Query root	read-only canonical query	result alias/contract agree	query inverse
Parameter side channel	copied map	public/generated disjointness	contract preservation
Result contract	copied Δ	one stable-ID result column	decoder preservation

Serialization is total only on finite $WF_{\text{Target}, \Sigma} \cap \text{Lex}WF_5$. Lexical rejection is a serialization failure, not an empty query result.

The canonical text profile is the image of the following unambiguous meta-grammar. Each capitalized metavariable expands to concrete Cypher 5 text. Attr_ω , One_ω , Many_ω , and Scan_ω are fixed Rule 06 productions selected from the

typed metadata of ω ; no descriptor stores raw query text.

$$\begin{aligned}
E_5 &::= Enc(c) \mid \$U \mid Y \mid Bot_\tau \mid Coerce_h(E_5) \mid head([Y \text{ IN } [E_5] \mid E_5]) \\
&\mid CASE \text{ WHEN } E_5 \text{ THEN } E_5 \text{ ELSE } E_5 \text{ END} \\
&\mid Op(E_5, \dots, E_5) \mid Attr_\omega(E_5) \mid One_\omega(E_5, [E_5, \dots, E_5]) \\
&\mid TypeTest_{\omega,b}(E_5) \mid TypeCast_\omega(E_5) \\
&\mid List_\kappa([E_5, \dots, E_5]) \mid Materialize(P_5) \\
&\mid ForAll3(P_5, Y, E_5) \mid Exists3(P_5, Y, E_5), \\
P_5 &::= From(E_5) \mid Scan_\omega \mid Many_{\kappa,\omega}(E_5, [E_5, \dots, E_5]) \\
&\mid Filter_k(P_5, Y, E_5) \mid Project_\kappa(P_5, Y, E_5) \mid Collect(P_5, Y, E_5) \\
&\mid PlanLet(Y, E_5, P_5) \mid Distinct(P_5), \\
Q_5 &::= Root(m, K, Y, E_5, \\
&\quad \omega_K, \omega_{id}, Alias).
\end{aligned}$$

Here the names $List_\kappa$, $Materialize$, $ForAll3$, $Exists3$, and $Root$ denote the concrete, fixed Cypher clause/comprehension templates printed by Algorithm 4; they are not tokens emitted literally. Their delimiters, keywords, and mandatory parentheses make the productions disjoint. The lexical codecs for $Enc(c)$, U , Y , $Alias$ and each symbolic field copied from a descriptor are injective and have explicit left inverses.

$LexWF_5$ is the least structural predicate with one clause for each production above: a leaf is admitted exactly when its codec accepts it; a composite is admitted exactly when all children are admitted, all binder names are fresh and legal, and every symbolic field used by its fixed production is in the corresponding lexical codec domain. Consequently structural induction gives both total printing and unique parsing on this image. Text outside this canonical image is irrelevant to the serializer theorem even if accepted by a general Cypher parser.

B.6 Stage-Totally Domains

The logical domain $Inv_{val}(MM, \Sigma)$ contains exactly the resolved and typed source invariants generated by the source grammar whose constructor rows and recursive children satisfy frontend admission, and whose referenced model declarations have unique type-correct observers in Σ . Structural induction over this admission derivation proves frontend and Core-to- Q existence.

The backend domain $C5Realisable(Violations(K, q), \Sigma, \Omega)$ independently contains:

$$WF_Q^\Sigma(q), \quad WF_{RepSpec}(MM, \Sigma), \quad LexAdmissible_5(q, \Sigma),$$

together with every required carrier, numeric, and cardinality certificate in Ω . R5O derives observer adequacy for (q, Σ) rather than assuming it. Mutual induction on this derivation proves realization existence; target well-formedness and lexical admissibility then prove serialization existence. Neither domain is defined by observing implementation success.

B.7 Exact Formal Q Inventory

The mutual grammar in Sect. 3 is finite and exhaustive. Its constructors partition into the following proof groups:

Table 11: Exhaustive formal Q constructor inventory.

Proof group	Formal constructors
Leaf/carrier	$c_{\tau_Q}, p_{\tau_Q}, QBottom_{\tau_Q}, x$
Coercion, binding, and control	$QCoerce, QLet, QIf$
Graph observers	$ReadAttribute, NavigateOne, NavigateAssociationClassOne, QTypeTest, QTypeCast$
Primitive operators	$QUn_{op}, QBin_{op}$
Expression-plan bridge and folds	$MaterializeQ, Exists3, ForAll3$
Collection literal	$QCollection_\kappa$
Plans	$FromCollection, ScanClass, NavigateMany, NavigateAssociationClass, Filter_k, Project_\kappa, CollectQ, PlanLet, DistinctQ$

R-5 performs mutual induction over exactly these expression and plan cases; the association-class, filter-kind, project, and distinct branches are not hidden in a generic fallback. Parameterized Set/Bag and SELECT/REJECT alternatives instantiate their stated constructor schemas and retain their separate semantic side conditions.

Proposition B.1 (Constructor-coverage closure) *Within $\mathcal{F}_{\text{paper}}$:*

1. *every resolved-source constructor family in the admitted grammar, including the separate SELECT and REJECT policies, occurs in a frontend equation and has an induction discharge route;*
2. *every admitted Core expression and plan constructor occurs in a Core-to-Q equation and is covered by the mutual induction;*
3. *every expression and plan constructor in the formal Q grammar, together with the violation root, is covered by the R-5 mutual induction; and*
4. *every constructor in $\text{TargetSigma}_{\text{exec5}}$, together with the query root and two copied side channels, occurs in a canonical serialization production and is covered by the simultaneous inversion proof.*

No transformation branch is reachable without one of these grammar constructors, and no grammar constructor lacks an algorithm equation or a proof-discharge route. The proposition counts coverage obligations, not separately numbered lemmas.

Proof. For each sort, inspect the alternatives of its grammar from Sect. 3 in order and map it to the correspondingly named row of Tables 7–10. The constructor tags are pairwise disjoint; unary/binary primitive and universal/existential rows are explicitly finite two-family schemas rather than catch-all cases. The algorithm listings contain the same alternatives, while Appendix C maps each alternative either to a dedicated argument or to an explicit instance of a reusable lemma. Structural induction supplies the child hypotheses in every mapped branch. Therefore the finite map is total on the grammar inventory; it need not be injective on lemmas because one generic lemma may discharge several constructor branches. $\square$

C Coverage of the Remaining Induction Cases

This appendix discharges the constructor families not expanded in the main text. It is not a catalogue containing one lemma per transformation rule. Rather, the correctness theorem for each full algorithm is proved by structural or mutual structural induction. Repeated algebraic cases are instances of explicit generic lemmas; representation-, collection-, binder-, and bottom-sensitive cases are stated separately. The induction order is source/Core/Q containment height, with expression and plan statements proved mutually where their grammars are mutual.

C.1 Canonical Graph Construction and the Adequacy Bridge

Fix $\text{WF}_{\text{RepSpec}}(MM, \Sigma)$, a finite valid instance M , and $\text{Representable}_{\Sigma}(M)$. Let LP be the injective length-prefixed tuple encoder and define disjoint node/relationship keys by

$$\begin{aligned} \text{objectKey}(o) &= LP(\text{modelKey}, \text{node}, \text{object}, \text{id}_M(o)), \\ \text{slotKey}(o, a) &= LP(\text{modelKey}, \text{node}, \text{slot}, \text{id}_M(o), \text{id}_{MM}(a)), \\ \text{linkKey}(\ell) &= LP(\text{modelKey}, \text{rel}, \text{link}, \text{sig}(\ell), \text{rank}(\ell)). \end{aligned}$$

Here $\text{rank}(\ell)$ is the zero-based occurrence rank inside the finite bucket with the same association, endpoints, association-class object, and ordered encoded qualifier tuple. Equality of keys exposes equal decoded tags and payloads; source-ID injectivity and equality of rank within one bucket then give equality of source elements. Thus all generated semantic keys are injective and node and relationship key families are disjoint.

The builder consists of finite comprehensions for schema declarations, ordinary/aggregate/composite association descriptors, objects, direct type edges, scalar slots and their typing edges, ordinary link occurrences, and the two participant edges of each association-class occurrence. Class declarations retain their abstract flag, while valid instances contribute no object whose dynamic class is abstract. Every lookup is total by $\text{WF}_{\text{RepSpec}}$ and every codec application is total on the used value by representability. A second enumeration yields the same keyed elements and references, hence the result is unique up to key-preserving containment isomorphism.

Lemma C.1 (G4-WF: exhaustive PGMM well-formedness) *The canonical construction satisfies all 56 named invariants of PGMM-Constraints.*

Proof. Partition the executable invariants into catalogue closure and uniqueness; global key/model isolation; contained type and endpoint closure; graph typing; required/key/qualifier property existence and uniqueness; label and physical type conformance; unqualified multiplicity; association-class closure and participant completeness; bounds and qualifier indices; codec/storage/default compatibility; value carrier conformance; and the nine observer-shape constraints. Catalogue groups follow from $WF_{RepSpec}$; key groups follow from injectivity of LP and the family tags; property, endpoint, typing, label, and multiplicity groups follow by inversion of their unique generating comprehension; association-class groups follow from the source link-object bijection; value groups use codec typing and left-inversion; and observer-shape groups follow from the disjoint Rule 03 constructor switch. These groups are an exhaustive partition of the 56 invariant names, so finite conjunction introduction yields $WF_{PGMM}(G)$. $\square$

Table 12: Exhaustive G4 crosswalk for the 56 PGMM invariants.

Invariant block	Count	Construction fact that discharges the block
Catalogue closure and uniqueness	4	One finite Σ entry per semantic signature and declaration key.
Global keys, isolation, and containment closure	5	Injective length-prefixed keys, disjoint family tags, and references created only to contained elements.
Typing roles and canonical endpoints	6	Exactly one direct-type edge per object and the fixed Rule 03 role switch.
Property and qualifier closure	8	One indexed property comprehension, exact required/key components, and ordered qualifier payloads.
Labels, physical types, endpoints, and unqualified multiplicity	6	Descriptor-selected labels/types and one generated occurrence per source link.
Observer references and association-class closure	3	Total descriptor lookup and the source occurrence–association-object bijection.
Bounds, codecs, storage, defaults, and semantic identity	14	Source multiplicities, codec typing/left inversion, separated tags, and stable semantic keys.
Observer shapes	10	The disjoint context, extent, type, attribute, identity, ordinary-navigation, and association-class constructors.
Total	56	The blocks are pairwise disjoint and cover every <code>inv</code> declaration in <code>PGMM-Constraints.ocl</code> .

The object comprehension defines a bijection γ because key injectivity gives injectivity and the no-extra rule gives surjectivity. Slot construction is indexed exactly by stored source slots. Ordinary relationship construction is indexed exactly by source link occurrences and preserves association kind, endpoints, and ordered qualifier payloads; association-class construction composes the source link-object bijection with γ and creates exactly two differently tagged participant edges. Restricting either occurrence bijection to a receiver and qualifier bucket preserves its cardinality. Copied direct-generalization edges have the same reflexive-transitive closure as MM . Abstract declarations remain available to conformance and extent observers but have no direct runtime object. These facts prove A-ID, A-DTYPE, A-TYPE, A-EXTENT, A-ATTR, A-NAV1, A-NAVM, A-AC, A-MULT, and A-BOTTOM, and therefore

$$\text{ValidRep}_{\Sigma}(MM, M, G, \gamma) \implies \text{Adequate}_{\Sigma}(M, G).$$

C.2 Core Construction Well-Formedness and Admission

The N-stage invariant states that declaration binding is injective, every reference resolves inside the current model and lexical scope, types are canonical and closed, containment ownership is unique, every constructor has its declared signature, and every output element carries its source trace. Structural induction establishes this invariant before invoking a validator: leaves use resolved fidelity, binders extend and restore the declaration environment, conditionals use the unique admitted join, collections coerce each item to one flat element type, iterators use the source element type under the body binder, and attribute/navigation/type/operator cases use their resolved declarations and signatures.

The constraint crosswalk partitions all 92 named Core-IR invariants into root and catalogue closure, declaration/type uniqueness, ownership and trace closure, literal fidelity, scope and binding, per-constructor typing, iterator shape, navigation/qualifier conformance, and unit/model assembly. Each group is exactly a conjunct of the construction invariant or one constructor case; therefore the validator is a postcondition check, not a premise of N-WF. The abstract freshness proof uses natural numbers and applies to every finite admitted AS.

Table 13: Exhaustive N3/N5 crosswalk for the 92 Core-IR invariants.

Invariant block	Count	Inductive construction fact
Model header, registries, uniqueness, and canonical catalogues	16	Root initialization creates the exact type/UML/unit registries and uses fresh element, trace, and semantic identities.
Type, trace, ownership, and model-scope closure	17	Canonical type interning, unique containment, closed classifiers, flat element types, and source-trace construction.
Resolved class/property declarations	16	Resolved UML fidelity supplies acyclic inheritance, multiplicity, opposite, qualifier, owner, and canonical OCL-value facts.
Units, variables, contexts, and invariants	8	Binder-environment extension/restoration plus the Boolean, parameter-free invariant-root rule.
Literal, variable, let, if, and coercion nodes	12	Constructor-local typing and canonical literal/coercion witnesses.
Attribute, ordinary-navigation, and association-class nodes	11	Resolved owner/receiver/result types, navigation kind, participant role, and ordered qualifier checks.
allInstances, type test/cast, and primitive operators	8	Closed class references and the unique admitted operation signature.
Collection literal and iterator nodes	4	Homogeneous flat items and source-element/binder/result signature agreement.
Total	92	The blocks correspond to invariant declarations 1–92 in source order and leave no validator clause unmatched.

C.3 Reusable Lemmas

Lemma C.2 (Environment extension) *If $\rho \simeq_{\Sigma} \eta$ and $v \simeq_{\Sigma} w$, then $\rho[x \mapsto v] \simeq_{\Sigma} \eta[x \mapsto w]$ for the same declaration identity x .*

Proof. The two extended maps have equal domains. They agree by the original environment correspondence at identities other than x , and by $v \simeq_{\Sigma} w$ at x . $\square$

Lemma C.3 (Typed unary and binary congruence) *Suppose an admitted source/Core operation and its Q image share the typed operation certified for their carriers. Corresponding operands then yield corresponding results.*

Proof. For a defined primitive result, apply the operation-correspondence certificate. If a strict non-equality operand is typed bottom, both semantic clauses return corresponding typed bottom. For typed equality, split into both-bottom, exactly-one-bottom, and both-defined cases; total typed equality returns the same Boolean result in each case. The Boolean operations follow by exhaustive inspection of the finite three-valued tables. The certified numeric cases follow from the local range/intermediate-operation certificate. $\square$

Lemma C.4 (Occurrence image) *Suppose $C \simeq_{\Sigma} C'$ and for every paired occurrence $v \simeq_{\Sigma} w$, $F(v) \simeq_{\Sigma} F'(w)$. Then occurrence-wise collect produces corresponding Bags with one output per input occurrence. If a separate typed-support operation is subsequently specified, that operation produces corresponding Sets.*

Proof. Adequacy supplies a bijection between indexed source occurrences. Mapping paired indices produces equal Bag output multiplicities, including repeated images of distinct Set members. A later typed-support operation removes exactly duplicate corresponding values. Whole-collection bottom is mapped to whole-collection bottom before the occurrence premise is used. $\square$

Lemma C.5 (Order-erasure invariance) *Let I and I' be two finite enumerations of the same link-occurrence multiset. If a resolved expression context satisfies `orderAdmissible`, replacing an ordered-end observation enumerated as I by the same observation enumerated as I' does not change its denotation. For a unique end both observations use the same typed support; for a non-unique end both use the same Bag multiplicity.*

Proof. The navigation base case follows directly from equality of support for Set and equality of occurrence multiplicity for Bag. Structural induction on the admitted enclosing context completes the proof. Boolean and numeric operations see only scalar child values; size, emptiness, membership, count, Set union/intersection, and certified sum are invariant under a permutation of occurrences. Select and reject restrict the same occurrences by the same three-valued predicate, collect maps them to the same result multiset, and the universal/existential folds depend only on the presence of true, false, and bottom classifications. Let, conditionals, coercions, and type operations use their child induction hypotheses. The admission premise excludes the only remaining observable case, collection equality with ordered-navigation provenance. Hence no admitted context distinguishes I from I' . $\square$

Lemma C.6 (Filter, project, and collect correspondence) *Let $C \simeq_{\Sigma} C'$ be paired finite occurrence collections, and extend paired environments with each paired occurrence. If paired predicates have the same Boolean3 result and paired projection bodies have corresponding values, then: (i) select retains exactly the paired true occurrences; (ii) reject retains exactly the paired false-or-bottom occurrences; (iii) Bag projection and collect preserve one output occurrence per input occurrence; and (iv) Set projection returns corresponding typed supports. Whole-collection bottom is preserved in every case.*

Proof. Lemma C.2 makes the body hypotheses applicable at every paired index. Equality of the three Boolean classifications proves (i) and (ii) by restriction of the same index bijection. Lemma C.4 then proves (iii), including repeated images; applying total typed equality to the paired images identifies the same equivalence classes and proves (iv). The separate whole-bottom semantic clause precedes iteration on both sides. $\square$

Lemma C.7 (Three-valued finite folds) *Paired finite occurrence collections whose paired predicates have equal Boolean3 classifications have equal ForAll3 and Exists3 results.*

Proof. If the source is whole-bottom, both results are bottom. Otherwise the paired predicates contain a false occurrence on one side iff they do on the other; the same biconditional follows separately for true occurrences and for bottom occurrences. Applying the identical priority order gives the same result. When the collections are empty, none of the existential conditions holds, yielding true for universal and false for existential. $\square$

Lemma C.8 (Fresh target extension) *Let $y = \text{leastFresh}(n, F)$ with F finite. Then $y \notin F$. If $\rho \simeq_{\chi, \Sigma} \theta$, $x \notin \text{dom}(\chi)$, and $v \simeq_{\Sigma} w$, then*

$$\rho[x \mapsto v] \simeq_{\chi[x \mapsto y], \Sigma} \theta[y \mapsto w].$$

No pre-existing target binding is captured.

Proof. The ordered target-name domain is infinite, so a least element outside finite F exists. The extended environments agree at the new paired identities by $v \simeq_{\Sigma} w$ and at old identities by the original correspondence. Freshness makes the two cases disjoint and prevents capture. $\square$

Lemma C.9 (Fixed-template observer refinement) *Assume*

$$\text{WF}_Q^{\Sigma}(q), \quad \text{WF}_{\text{RepSpec}}(MM, \Sigma), \quad \text{Adequate}_{\Sigma}(M, G).$$

For every observer reachable from q , the fixed Rule 06 target template selected by its unique catalogue entry equals the corresponding Q graph observer on admitted arguments:

$$\begin{aligned} \text{obs}_a(G, g, \omega_a) &= \text{slot}_G(g, a), \\ \text{obs}_1(G, g, \bar{u}, \omega_r) &= \text{Nav}_G^{\Sigma}(g, r, \bar{u}), \\ \text{obs}_*(G, g, \bar{u}, \omega_r) &= \text{Nav}_G^{\Sigma}(g, r, \bar{u}), \\ \text{obs}_{\text{type}}(G, g, \omega_K, b) &= \text{typeTest}_{K, b}^{\Sigma}(G, g), \\ \text{extent}(G, \omega_K) &= \text{Extent}_G^{\Sigma}(K). \end{aligned}$$

The many-valued equation preserves the certified Set support or every Bag occurrence.

Proof. Apply Theorem 4.8. Its finite case proof obtains lookup existence/uniqueness from $WF_{RepSpec}$ and obtains semantic equality from adequacy plus the actual A5 row-bag calculation of the selected fixed template. In particular, attribute absence is separated from stored bottom, qualified navigation matches qualifier components by codec reflection, upper-one has zero/one rows, Set navigation takes typed support, Bag navigation retains every relationship occurrence, association-class navigation uses the occurrence node as identity, and extent uses direct typing plus inheritance closure. Thus no semantic equality is assumed as a catalogue field. $\square$

Lemma C.10 (Stable-ID projection) *Assume $Adequate_{\Sigma}(M, G)$ with witness γ , and let paired source and graph predicates classify every o and $\gamma(o)$ identically as true, false, or Boolean bottom. Projecting stable IDs from their respective non-true subsets and taking distinct support yields equal ID sets.*

Proof. Adequacy makes γ a bijection between the source extent and graph extent and gives $id_G(\gamma(o)) = id_M(o)$. Equal three-valued classifications restrict this bijection to the two non-true subsets. ID projection therefore has the same image on both sides; taking distinct support changes multiplicity but not membership. $\square$

Lemma C.11 (Canonical production inversion) *For every target constructor C and admitted children t_i , the parser of the canonical text profile satisfies*

$$\text{parse}_5(\text{print}_C(t_1, \dots, t_n)) = C(\text{parse}_5(\text{print}(t_1)), \dots, \text{parse}_5(\text{print}(t_n)))$$

up to alpha-renaming of freshly printed binders.

Proof. The productions in Appendix B have disjoint fixed keywords/delimiters. Identifier, parameter, literal, and descriptor codecs have left inverses. Hence the outer production is unique and recursive parsing is confined to its delimited child fields. A binder identity may be renamed consistently by the parser; freshness and alpha-invariance preserve its binding structure. $\square$

C.4 Remaining Frontend Cases

Variable. Resolution identifies a unique declaration x and the algorithm produces $\text{Var}(x)$. Both semantics return $\rho(x)$.

Conditional. The induction hypothesis equates the guards. A true or false guard therefore selects the same branch, whose equality follows from the corresponding branch hypothesis. A bottom guard produces typed bottom on both sides. The unselected branch is irrelevant in both semantics.

Attribute and upper-one navigation. Resolution records the same declaration key in the Core node. The source and Core semantics both invoke slot_M or Nav_M after applying the receiver induction hypothesis. Upper-one well-formedness makes absence and ambiguity treatment identical. For upper zero, instance validity forces the occurrence bucket to be empty, so both sides return the same target-typed bottom.

Many navigation and collection literal. The receiver hypothesis gives the same source receiver. Both source and Core invoke the same logical navigation and retain the resolved collection kind. When the source end is ordered, Lemma C.5 discharges the admission premise that no enclosing context observes position. For a literal, item hypotheses equate every occurrence; Set construction takes support and Bag construction retains the item sequence.

Unary and binary operation. Apply the child hypotheses and Lemma C.3. This single case covers every primitive operator whose signature and bottom policy are in the admission catalogue.

Select, reject, and collect. The source translation creates a plan and a body under the same declaration identity. The plan induction hypothesis gives the same occurrences. By Lemma C.2, the body hypothesis applies to each occurrence. Select retains exactly the true occurrences; reject retains false and bottom occurrences; both preserve the source kind and multiplicity. Collect follows from Lemma C.4 and produces a Bag.

ForAll and Exists. The plan hypothesis pairs the occurrence domains. Environment extension and the body hypothesis pair every Boolean result. Apply Lemma C.7.

AllInstances and type operations. The source and Core scan both use $Inst_M(K)$. Type tests use the same reflexive-transitive $\preceq_{MM}$ relation. An admitted cast returns the same object when the test succeeds and the same typed bottom when it fails.

These cases exhaust the frontend rows in Table 7; unsupported constructors have no successful case to prove.

C.5 Remaining Core-to- Q Expression Cases

Variable and bottom. Environment correspondence gives $\rho(x) \simeq_\Sigma \eta(x)$. The lifted adequacy map preserves each typed bottom by definition.

Public parameter. A Core parameter declaration becomes a $QParameterExpr$ with the same declaration identity and expected type. The realization contract records it once in the public namespace, disjoint from generated parameters. Both denotations therefore read the same typed runtime binding; a missing or malformed binding is an explicit failure and lies outside successful semantic preservation rather than denoting bottom.

Let. The initializer hypothesis gives paired values. Apply Lemma C.2 and then the body hypothesis. Unfolding the two let clauses proves the result.

Conditional. Boolean correspondence selects the same branch or bottom outcome. Apply the selected branch hypothesis.

Attribute. Let the receiver results be $v \simeq_\Sigma w$. For defined paired objects, the slot clause of adequacy gives

$$\widehat{\gamma}(\text{slot}_M(v, \mathbf{a})) = \text{slot}_G(w, \mathbf{a}).$$

Strict bottom receiver cases agree definitionally.

Typed operations. Apply Lemma C.3 to the child hypotheses.

Coercion. The child hypothesis supplies corresponding input carriers. The unique admitted coercion witness commutes with $\widehat{\gamma}$: Integer-to-Real acts on the primitive payload, class upcast preserves the stable object identity, and collection-element coercion applies the corresponding scalar witness to every occurrence without changing Set/Bag kind or whole-bottom. Hence the Core and Q coercion results correspond.

Type test and cast. For a defined paired object, the type-conformance clause of adequacy makes the source class test and the fixed graph-type observer return the same Boolean. Exact testing additionally compares the preserved direct type. A successful cast returns the same paired object; a failed or bottom cast returns the same typed bottom. This discharges $QTypeTest$ and $QTypeCast$ separately from ordinary primitive operators.

Collection literal and materialization. Item hypotheses establish occurrence correspondence; construction uses the same resolved kind. Materialization invokes the mutual plan hypothesis and changes no value.

ForAll and Exists. The plan hypothesis pairs occurrence domains, and the body hypothesis applies under paired environment extensions. Lemma C.7 finishes both cases.

C.6 Core-to- Q Plan Cases

From/FromCollection. The expression induction hypothesis gives either corresponding collection values or corresponding whole-bottom.

Scan/ScanClass. The extent clause of adequacy gives

$$\gamma[Inst_M(K)] = \text{Extent}_G^\Sigma(K).$$

Since a scan is Set-valued, this is the required plan correspondence.

Many navigation. The receiver expression hypothesis pairs receivers. Navigation adequacy maps the source result occurrence-wise to the selected graph observer. The collection-kind clause preserves Set support or Bag multiplicity.

Select/Reject/Filter. Apply Lemma C.6 to the plan and predicate induction hypotheses.

Collect/CollectQ. Apply the collect clause of Lemma C.6.

LetP/PlanLet. The initializer expression hypothesis and Lemma C.2 establish paired extended environments. Apply the plan hypothesis to the body.

These cases complete the mutual proof of Theorem 4.6.

C.7 Target Realization Cases

The realization induction is over normalized Q syntax plus its admission derivation. Admission supplies the certificate consumed by each case.

Constant. The carrier certificate supplies a canonical target payload c_T with $decode(c_T) = c$. Realization emits $TConst(c_T)$ or a generated parameter whose binding is c_T . The target leaf equation therefore yields c , and the leaf is closed and well typed.

Public parameter. Admission gives one public contract entry with the exact expected type and a name outside the generated namespace. $QParameterExpr$ and $TParam$ both read that binding, so they return the same decoded carrier; absent or ill-shaped bindings fail explicitly.

Typed bottom. $QBottom_\tau$ becomes $TBottom(\tau)$. Both semantic equations yield $\perp_\tau$; admission supplies the same type tag, so defined payloads cannot be confused with bottom.

Variable. Well-formed Q guarantees $x \in \text{dom}(\chi)$. Realization emits the target identity $\chi(x)$. Scope lookup relates the logical environment entry to the target row entry; therefore the values agree.

Coercion. The child hypothesis supplies equal logical input. The coercion certificate states that the guarded target payload conversion implements the same Integer-to-Real, class-upcast, or collection-element map as $QCoerce$, while preserving typed and whole-collection bottom. Thus $TCoerce$ has the declared type and denotation.

Let. The initializer hypothesis supplies a corresponding target value. The least fresh rule and Lemma C.8 extend the paired environments without capture. Applying the body hypothesis in those environments and unfolding $QLet/TLet$ proves equal results.

Conditional. The three child hypotheses pair guard and branch values. True and false select corresponding branches; a bottom guard yields the same typed bottom. Thus QIf and $TCase$ agree in all three guard cases.

Unary primitive. The operand hypothesis gives corresponding values. The unique $\text{prim}_\Omega(op)$ certificate states equality of the Q operator and its target carrier operation for defined and bottom operands. Hence the TOp result corresponds and has the declared result type.

Binary primitive. Apply both operand hypotheses and the binary field of the same certificate. For equality, use the both-bottom, exactly-one-bottom, and both-defined split; for strict numeric operations, use the certified range and bottom clauses; for Boolean operations, inspect the finite Boolean3 table.

Attribute. The receiver hypothesis pairs the receiver values. Lemma C.9 rewrites evaluation of $TObserveAttr$ to the exact Q slot observer. Strict receiver-bottom behaviour is identical.

To-one navigation. Use the receiver hypothesis and the second descriptor-refinement equation. The upper-one certificate makes zero targets bottom, one target that object, and ambiguity an explicit failure on both admitted observations.

Association-class to-one navigation. The same receiver and qualifier induction hypotheses apply, but the selected descriptor uses the certified association-class participant path. R5O maps its zero/one row to the occurrence object identified by the occurrence key, not merely to the endpoint pair, which gives exactly the $Q\text{NavigateAssociationClassOne}$ denotation.

Type test and cast. The child hypothesis pairs the object carrier. R5O makes direct-type equality and finite conformance closure agree with the Q type observer. Consequently exact/conformance tests return the same Boolean3 value; a cast returns the same object on success and the target-typed bottom on failure or bottom input.

Collection literal. Induction on the finite item sequence pairs each item in order. Target list construction applies the same resolved Set-support or Bag-occurrence policy, so the resulting collections correspond.

Materialization. The mutual plan hypothesis gives corresponding whole-bottom or finite occurrence collections. MaterializeQ and TMaterialize expose those values unchanged.

FromCollection. The expression hypothesis pairs the collection values. Both plan equations return that collection, including whole-collection bottom.

ScanClass. The fourth descriptor-refinement equation gives $\text{extent}(G, \omega_K) = \text{Extent}_G^\Sigma(K)$, exactly the two scan denotations.

NavigateMany. The receiver hypothesis and the third descriptor-refinement equation pair all target occurrences. The kind certificate takes support for Set and retains each occurrence for Bag.

Association-class NavigateMany. R5O replaces the ordinary link-row bijection by the association-object occurrence bijection. Qualifier predicates remain componentwise equivalent, and the kind certificate takes typed support only for Set; Bag retains every association occurrence, including equal endpoint pairs.

Filter. Fresh-target extension supplies the paired body environments; the filter clause of Lemma C.6 finishes the case.

CollectQ. Apply fresh-target extension and the collect clause of Lemma C.6.

Project. Apply fresh-target extension and the Bag/Set projection clauses of Lemma C.6.

PlanLet. The initializer hypothesis pairs its values. Extend environments using the fresh-target lemma and apply the mutual plan hypothesis to the plan body.

DistinctQ. The plan hypothesis preserves values and occurrences. Certified total typed equality identifies exactly the same equivalence classes, so support on both sides gives corresponding Sets.

ForAll3. The plan hypothesis pairs the finite domain and fresh-target extension makes the body hypothesis applicable to every occurrence. Lemma C.7 uses false, then bottom, then true priority and also covers empty and whole-bottom inputs.

Exists3. The same reasoning applies with true, then bottom, then false priority. The identity result on empty defined input is false.

Violation root. The scan case yields the Q context extent. Expression preservation gives R1 for every bound context object. The target root retains exactly false and bottom, projects the stable-ID observer, and applies distinct. Therefore

$$\text{Ret}^p(j) \iff \exists g \in \text{Extent}_G^\Sigma(K) : \text{id}_G(g) = j \wedge B_T^p(g) \neq \text{⊥},$$

which is R2. The result contract records the same projection alias and decoder.

These cases are in bijection with every expression, plan, and root row of Table 9; hence the mutual realization induction has no omitted constructor case.

C.8 Serialization Cases

The proof is simultaneous over target expressions, descriptors, clauses, and query roots.

Constant and parameter. Codec left-inversion recovers the typed constant payload. Parameter-name injectivity recovers the same public or generated parameter identity; the public contract or copied π_{gen} , respectively, supplies the same typed payload, and namespace disjointness prevents an ambiguous parse.

Variable. Identifier-codec left-inversion recovers the printed target identity. Under a binder parser this identity may be consistently alpha-renamed; lookup and denotation are alpha-invariant.

Typed bottom. The type tag and reserved bottom payload select the bottom production and cannot parse as a defined constant. Parsing therefore recovers $\text{TBottom}(\tau)$ exactly.

Let. The fixed head($[y \text{ IN } [t_1] \mid t_2]$) delimiters select the bind-once production. The child hypotheses recover t_1, t_2 and binder occurrences; freshness prevents capture. Target let and parsed comprehension both evaluate t_1 once and then t_2 .

Case. The mandatory CASE/WHEN/THEN/ELSE/END delimiters and child hypotheses recover all three children and their order. Both semantics use the same Boolean3 branch equation.

Primitive operation. The certified precedence and delimiter table identifies the unique operator signature and operand boundaries. Child inversion plus the operator token's left inverse recovers the same TOp tree.

Coercion. The fixed guarded carrier production and the coercion-kind token recover TCoerce_h and its child uniquely. Its bottom branch precedes the payload conversion, exactly as in the target equation.

Attribute and to-one observers. The fixed production and the left inverses of every symbolic field recover ω_a or ω_r ; the receiver and qualifier hypotheses recover their terms. Semantic correctness of the produced target pattern is R5O, not a raw template certificate stored in the descriptor.

Type test and cast. Distinct fixed delimiters recover TTypeTest versus TTypeCast , while descriptor and Boolean-mode codecs recover the target class and exact/conformance choice. R5O supplies their graph-type semantics after structural inversion.

List. Brackets and separators recover the finite child sequence. The explicit kind tag recovers Set versus Bag; child hypotheses preserve all item occurrences.

Materialize. The plan induction hypothesis recovers the contained plan. Its fixed subquery/comprehension delimiters recover the expression-plan bridge without changing the collection value.

ForAll3 and Exists3. The two distinct canonical fold templates recover the plan, binder, and body. Their ordered CASE branches are exactly the Boolean3 fold equations, including empty and whole-bottom cases.

From. The collection-expression hypothesis recovers the child and the fixed bridge template reconstructs TFrom.

Scan. The fixed scan production and symbolic-field codecs recover the unique class-extent descriptor from the read-only MATCH syntax.

Many observer. The receiver and descriptor hypotheses recover both arguments; the explicit kind field recovers the Set/Bag row policy.

Filter. Plan and predicate hypotheses recover both children. The scoped predicate alias is recovered up to alpha-renaming, and the explicit policy token recovers whether the fixed template retains exactly true rows (SELECT) or false-or-bottom rows (REJECT).

Project. Plan and body hypotheses recover both children and their binder. The explicit kind token recovers Bag occurrence projection or Set projection followed by typed support, hence the same TProject constructor.

Collect. Plan and body hypotheses recover both children and their binder. The canonical projection/comprehension emits one row per input occurrence, so Bag multiplicity is unchanged.

PlanLet. The same bind-once and freshness argument as target let applies, followed by the plan induction hypothesis.

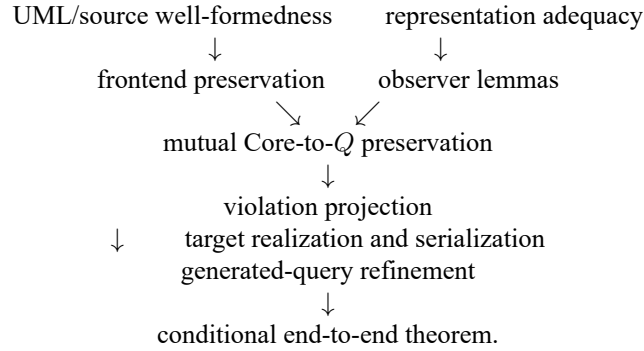
Distinct. The child hypothesis recovers the plan and the mandatory DISTINCT token uniquely recovers TDistinct.

Query root and side channels. The root production fixes the read-only clause order, context descriptor, non-true filter, stable-ID observer, one result alias, and final distinctness. The query parser therefore recovers $\text{Syn}(p)$, the executable syntax of the same target root. Serialization copies Δ and π_{gen} without recomputation, so parameter ownership, generated values, and the decoder are unchanged even though those values are not reconstructed from query text.

These cases prove Theorem 4.10 on the declared $\text{WF}_{\text{Target}, \Sigma} \cap \text{LexWF}_5$ domain.

C.9 Non-Circular Dependency Order

The complete dependency order is



Adequacy is established independently of translation. R3 is supplied only after successful execution and decoding. No local lemma is justified by the end-to-end equality it helps prove.

D Assurance Traceability

This appendix records the epistemic status of each boundary. The compact labels have fixed meanings:

Manual a supplied mathematical argument not fully kernel checked;

Lean the named formal slice is accepted by the Lean kernel;

Partial only the named subset or refinement is established;

External an external premise required by a corollary;

Tested finite implementation evidence;

Live finite database execution evidence.

D.1 Theorem Traceability

Table 14: Formal result and assurance status.

Boundary	Primary result	Main premise	Status
Resolved OCL to Core	Frontend type/scope and semantic preservation	source admission and typing	Manual; Lean slice

Boundary	Primary result	Main premise	Status
Core to Q	Mutual type and semantic preservation	adequacy and environment correspondence	Manual; Lean slice
Canonical graph builder Q to violation IDs	Builder implies adequacy Violation-projection preservation	valid finite representation body, extent, and ID correspondence	Manual; Tested Manual; Lean slice
Q to target	Target well-formedness, lexical well-formedness, and preservation	C5 certificates; R5O derives observer adequacy	Manual; Lean slice
Target to text	Parser/printer and semantic transport	target/lexical well-formedness	Manual; Lean slice
Formal text to Neo4j	Corollary 4.16	$\text{Exec}_{\text{Neo4j}} \sqsubseteq_{\text{Gen5}} \text{TextSem}_5$	External; selected runs Live
End to end	exact formal violation-ID equality	P1–P5	Manual; Lean composition slice
Logical translation existence	OCL-admission to Q existence	independent admission derivation	Manual
Backend existence	C5-realizable to formal text existence	independent certificates	Manual

The registries have different denominators: the formal preservation ledger has 37 hand-proved entries, whereas the transformation coverage registry has 31 rule/refinement rows. Neither number is a count of grammar constructors or machine-checked theorems. Exact rerun dates, script totals, source digests, and build-environment notes belong to the companion artefact record rather than to the mathematical claim.

D.2 Lean-Checked Slice

Table 15: Machine-checked components and exact scope.

Component	Checked scope
Scalar codecs	canonical encoding, successful decoding, injectivity, malformed rejection, and typed-bottom separation for the named carriers
Typed equality	primitive values, flat Set support, Bag multiplicity/permutation, and typed collection bottom
Boolean3	complete connectives and finite fold laws
Numeric kernels	selected signed-INT64 interval properties and exact rational Real lemmas; not unrestricted binary64 refinement
Collection pipeline	separate source/target carriers, empty versus whole-bottom, filter policies, collect order/multiplicity, and Bag result
Type foundation	flat type grammar, selected joins, inheritance conformance, and Set/Bag covariance
Formal Q inventory	closed mutual syntax inventory for the admitted calculus
Vertical slice	literals, variables, Boolean operations, if, let, formal target realization, and exact violation-ID preservation
Serialization slice	canonical token serialization/parser round trip for the admitted slice
End-to-end composition	conditional no-extra/no-missing ID and set equality from explicit refinement witnesses

The current axiom audit reports no project-defined axiom and no sorry/admit. Standard library reasoning may report propositional extensionality, classical choice, and quotient soundness.

D.3 External Assumptions

Table 16: Assumption ledger.

Assumption	Consumed at	Discharge route
Finite valid M	source semantics and iteration	source-model validation

Assumption	Consumed at	Discharge route
Stable-ID injectivity	projection theorem	$WF_{MM}(M)$
Representation adequacy	Core-to- Q theorem	independent witness or builder bridge
Observer/codec catalogue	translation and realization	$WF_{RepSpec}(MM, \Sigma)$; R5O
Numeric/cardinality coverage	target primitive/length operations	derives template adequacy
Lexical admissibility	serializer totality	Ω and runtime coverage judgment
Public/generated parameter disjointness	target execution	C5 admission derivation
Formal executor semantics	generated-query refinement	artifact contract and runtime check
Concrete Neo4j refinement	Corollary 4.16 only	A_5 and parser/text transport
		external proof of
		$Exec_{Neo4j} \sqsubseteq_{Gen5} TextSem_5$; finite
		live runs are supporting evidence
Resource termination	concrete deployment only	outside mathematical semantics

References

- [1] Amine Benelallam, Abel Gómez, Gerson Sunyé, Massimo Tisi, and David Launay. Neo4EMF: A scalable persistence layer for EMF models. In *Proc. 10th Eur. Conf. Model. Foundations Appl.*, volume 8569 of *Lect. Notes Comput. Sci.*, pages 230–241, 2014.
- [2] Jordi Cabot and Martin Gogolla. Object constraint language (OCL): A definitive guide. In *Formal Methods for Model-Driven Engineering*, volume 7320 of *Lect. Notes Comput. Sci.*, pages 58–90. Springer, 2012.
- [3] Gwendal Daniel. Efficient validation of large models using the Mogwai tool. In *Proc. 18th Int. Workshop on OCL and Textual Model. (OCL)*, volume 2245 of *CEUR Workshop Proc.*, pages 187–193, 2018.
- [4] Gwendal Daniel, Frédéric Jouault, Gerson Sunyé, and Jordi Cabot. Gremlin-ATL: A scalable model transformation framework. In *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, pages 462–472. IEEE, 2017.
- [5] Gwendal Daniel, Gerson Sunyé, Amine Benelallam, Massimo Tisi, Yoann Vernageau, Abel Gómez, and Jordi Cabot. NeoEMF: A multi-database model persistence framework for very large models. *Sci. Comput. Program.*, 149:9–14, 2017.
- [6] Gwendal Daniel, Gerson Sunyé, and Jordi Cabot. Mogwai: A framework to handle complex queries on large models. In *Proc. 10th Int. Conf. Research Challenges Inf. Sci. (RCIS)*, pages 1–12. IEEE, 2016.
- [7] Gwendal Daniel, Gerson Sunyé, and Jordi Cabot. UMLtoGraphDB: Mapping conceptual schemas to graph databases. In *Proc. 35th Int. Conf. Conceptual Model. (ER)*, volume 9974 of *Lect. Notes Comput. Sci.*, pages 430–444. Springer, 2016.
- [8] Marina Egea and Carolina Dania. SQL-PL4OCL: An automatic code generator from OCL to SQL procedural language. *Softw. Syst. Model.*, 18(1):769–791, 2019.
- [9] Martin Gogolla, Fabian Büttner, and Mark Richters. USE: A UML-based specification environment for validating UML and OCL. *Sci. Comput. Program.*, 69(1–3):27–34, 2007.
- [10] Yang He, Ruijie Fang, Işıl Dillig, and Yuepeng Wang. Graphiti: Bridging graph and relational database queries. *Proc. ACM Program. Lang.*, 9(PLDI):1–25, June 2025. Article 216.
- [11] Ana León, Maribel Yasmina Santos, Alberto García, Juan Carlos Casamayor, and Oscar Pastor. Model-to-model transformation: From UML class diagrams to labeled property graphs. *Bus. Inf. Syst. Eng.*, 66(1):85–110, 2024.
- [12] Hoang Nguyen Phuoc Bao and Manuel Clavel. OCL2PSQL: An OCL-to-SQL code-generator for model-driven engineering. In *Proc. Int. Conf. Future Data and Security Eng. (FDSE)*, volume 11814 of *Lect. Notes Comput. Sci.*, pages 185–203, 2019.
- [13] Hoang Nguyen Phuoc Bao and Manuel Clavel. Proving correctness for SQL implementations of OCL constraints, 2024. arXiv:2403.18599 [cs.DB].

- [14] Object Management Group. Object constraint language, version 2.4. Technical Report formal/2014-02-03, Object Management Group, February 2014.
- [15] Hendrik Radke, Thorsten Arendt, Jan Steffen Becker, Annegret Habel, and Gabriele Taentzer. Translating essential OCL invariants to nested graph constraints for generating instances of meta-models. *Sci. Comput. Program.*, 152:38–62, 2018.